athenea: assets USD fotorrealistas como gaussianas reiluminables, de la malla al móvil

jesus luque

12 de octubre de 2026

RESUMEN

Describimos athenea, un renderizador de USD e Hydra en el que cada paso que toca datos de la escena se ejecuta como un kernel de cómputo en la GPU, y la cadena construida sobre él que convierte mallas USD de producción en splats de gaussianas 3D que conservan su luz. Una malla se convierte en el dispositivo en gaussianas anisótropas, una por celda de una retícula derivada de una cámara, con opacidad que respeta la cobertura, bordes sujetos donde una superficie termina o se pliega, materiales OpenPBR por capas, vidrio clasificado como sólido o como pared fina, y rugosidad ensanchada por la dispersión de normales bajo cada gaussiana. El propio path tracer de la escena hornea después un transfer en cada gaussiana: armónicos esféricos de grado 3 de la luz directa e indirecta, un campo reflejado y 256 bits de direcciones abiertas. Al dibujar, estos reiluminan la nube bajo cualquier cielo y cualquier luz, con reflejos nítidos girados por píxel mediante un operador de forma almacenado. Los assets con rig conservan su esqueleto, con un jacobiano que incluye los gradientes de los pesos, lóbulos zonales de transfer que giran con un miembro y campos de visibilidad por parte. Árboles de fusión por pieza con un error de pantalla monótono y acotado, culls de caras traseras y de oclusión, y páginas ordenadas por error dan los niveles de detalle. Un PCA por clusters, la visibilidad cuantizada vectorialmente y un registro compacto de 40 bytes llevan el Chevrolet Corvette C7 entero a un reproductor WebGPU en un iPhone. En todo momento, la referencia es un path trace de la malla de origen. Doblar la resolución del horneado de 640 a 1280 px subió el PSNR del coche de 16.55 a 17.88 dB en su recorte y de 22.18 a 23.90 dB en el fotograma. El nivel de detalle cuesta unos 0.1 dB. En el móvil, los datos del coche se mantuvieron en 31–44 MB en movimiento. Para las vistas de cerca se construyó un nivel de primer plano a 5120 px de 74.5 millones de hojas en 24 árboles.

Palabras clave: splatting de gaussianas 3D; USD; Hydra; transferencia de radiancia precalculada; armónicos esféricos; de malla a gaussianas; nivel de detalle; PCA por clusters; WebGPU; render en móvil; path tracing; cadenas solo en GPU.

ÍNDICE

1. Introducción

1.1 El problema

1.2 Objetivos y restricciones

1.3 Contribuciones

1.4 Organización
1.5 Trabajo relacionado
1.6 Convenciones

2. El motor y el renderizador athenea

2.1 Reglas de diseño
2.2 Arquitectura de módulos
2.3 Fundamentos de GPU
2.4 La representación gaussiana
2.5 El rasterizador por teselas
2.6 El trazador de rayos de gaussianas
2.7 Puntos
2.8 Mallas: geometría, escena y visibilidad
2.9 Materiales
2.10 Luces
2.11 El trazador de caminos
2.12 Visualización y color
2.13 USD e Hydra
2.14 Hosts y front ends
2.15 Verificación sin oráculo en CPU
2.16 Limitaciones

3. De mallas USD a gaussianas (mesh2splat)

3.1 Origen, y dónde se ejecuta la conversión
3.2 Lectura del stage
3.3 El tamaño de una celda
3.4 Una gaussiana por celda
3.5 Donde una superficie termina o se pliega
3.6 Captura del material
3.7 El filtro especular (ensanchamiento de Toksvig)
3.8 Vidrio
3.9 Captadores de sombra y de luz
3.10 Validación frente al path trace
3.11 Resultados medidos sobre el Corvette
3.12 Limitaciones

4. Transporte de luz horneado en gaussianas (transfer) y reiluminación

4.1 Notación
4.2 Qué guarda una gaussiana
4.3 El entorno en el momento de dibujar
4.4 El horneado
4.5 El cuerpo
4.6 Reflexión
4.7 Vidrio, transmisión y paredes finas
4.8 La reflexión a lo largo de la huella
4.9 Deshacer el ensanche del horneado
4.10 Medidas frente al path trace

4.11 Limitaciones

5. Assets animados: esqueletos, skinning y visibilidad por partes (el peón y el gorrión)

5.1 Principio de diseño: llevar el rig, no los frames
5.2 Lectura de un stage con rig
5.3 Influencias por gaussiana, y sus gradientes
5.4 Skinning de una gaussiana en el dispositivo
5.5 El fichero: el rig como USD
5.6 El motor: pose, límites, reajuste y desenfoque de movimiento
5.7 Luz que gira con un miembro: lóbulos zonales de transfer
5.8 Campos de visibilidad por partes: horneados una vez, un producto de lecturas de tabla por frame
5.9 Niveles de detalle y diezmado de nubes con skinning
5.10 El gorrión
5.11 El peón: el control rígido
5.12 Reproducción en la web y en un teléfono
5.13 Lo que no está hecho

6. Niveles de detalle: árboles de fusión, cortes por error, páginas y oclusión

6.1 El nivel de detalle original del motor
6.2 Árboles de fusión por pares por pieza
6.3 El corte por error, con cotas
6.4 La opacidad de una fusión: cobertura por mínimos cuadrados
6.5 La métrica de derrame, y por qué falló en la rejilla
6.6 Descarte de caras traseras en cascos cerrados
6.7 Máscaras de oclusión
6.8 Páginas por error
6.9 El nivel de primer plano
6.10 Resultados medidos
6.11 Límites y cuestiones abiertas

7. Compresión y el reproductor WebGPU en el iPhone

7.1 El contenedor: `.athc` versión 3
7.2 Compresión del transfer por PCA agrupada
7.3 Los bits de direcciones abiertas por cuantización vectorial
7.4 El registro compacto
7.5 Páginas por bandas de error y el tier compacto
7.6 Streaming y el corte de cada frame
7.7 El dibujado
7.8 Reiluminado en el móvil
7.9 La sombra de contacto
7.10 Control por tiempo de frame y el frame quieto
7.11 Contabilidad de memoria
7.12 Resultados medidos
7.13 Limitaciones y trabajo pendiente

8. Evaluación

8.1 Marco
8.2 La corrección y la velocidad del motor

8.3 La conversión frente al path trace
8.4 El coche entero a lo largo de la cadena
8.5 Niveles de detalle
8.6 Compresión
8.7 El móvil
8.8 Memoria
8.9 Costes
8.10 Donde las fuentes difieren

9. Limitaciones y trabajo futuro

9.1 Defectos que quedan en el Corvette
9.2 La representación y el transfer
9.3 Assets animados
9.4 Niveles de detalle, streaming y el reproductor
9.5 El motor
9.6 Huecos en la validación
9.7 Trabajo futuro

10. Referencias

10.1 Splatting de gaussianas y render basado en puntos
10.2 De mallas a gaussianas
10.3 Transporte de luz, transfer precalculado y compresión
10.4 Materiales y sombreado
10.5 Geometría, estructuras de aceleración y niveles de detalle
10.6 Color
10.7 Animación y visibilidad
10.8 Sistemas y estándares

Apéndice A. Repositorios, comandos y figuras

A.1 Repositorios
A.2 Comandos y opciones
A.3 Figuras

1. Introducción

1.1 El problema

Los assets 3D de producción se crean como mallas: polígonos, materiales físicos por capas, texturas, luces y, en los personajes, esqueletos, todo compuesto en USD. Un path tracer los renderiza con fidelidad, pero es demasiado lento para un móvil; un rasterizador los renderiza de forma interactiva, pero necesita residentes toda la geometría y todas las texturas del asset y aun así aproxima su transporte de luz. El splatting de gaussianas 3D (3DGS) ofrece una tercera vía: una nube de gaussianas anisótropas, ordenadas y mezcladas en el espacio de pantalla, dibuja detalle fotográfico a ritmo interactivo en hardware modesto. Sin embargo, una nube de gaussianas normalmente se *entrena* a partir de fotografías de una escena real. Contiene la luz de esa captura, no se puede reiluminar, y no tiene material, ni esqueleto, ni relación con un asset creado por un artista.

Este artículo aborda el problema inverso. Dado un asset USD fotorrealista (el caso que se sigue de principio a fin es un Chevrolet Corvette C7 de 192 mallas con pintura de coche OpenPBR, cromo, vidrio, goma y faros), producir una representación en gaussianas que

1. sea **fiel a un path trace** de la malla de origen, juzgada imagen contra imagen y región por región;
2. **se reilumine**: que responda a cualquier cielo y cualquier luz, no solo a la iluminación con la que se hizo;
3. conserve el rig cuando el asset lo tiene, de modo que un asset animado siga moviéndose;
4. **se reproduzca en un móvil** dentro de una página web, con un presupuesto de memoria fijo, tanto de cerca como de lejos.

Ninguna de las vías evidentes cumple las cuatro. Entrenar una nube a partir de vistas del asset con path tracing da una iluminación fija y gasta horas de optimización por asset. Rasterizar la malla en el móvil está limitado por la memoria y por el sombreado por píxel que puede permitirse una GPU móvil. La vía que se sigue aquí convierte la malla directamente en gaussianas, hornea el transporte de luz del asset en cada gaussiana con el propio path tracer del renderizador, y después reduce, comprime y transmite el resultado.

1.2 Objetivos y restricciones

El trabajo se hizo bajo un pequeño conjunto de restricciones, fijadas por el usuario y mantenidas en todo momento. Determinan casi todas las decisiones de diseño de las secciones 2–7.

restricción	**enunciado**	**dónde actúa**
Aritmética solo en GPU	Ninguna aritmética sobre datos de la escena se ejecuta en la CPU: ni renderizador, ni alternativa, ni oráculo de pruebas en CPU. El host lee ficheros y lleva las cuentas (recuentos, huecos, colas).	Todas las secciones; §2.1 enuncia la regla
3DGS puro en la reproducción	Lo que dibuja el reproductor son solo gaussianas. El suelo no es una malla: su interacción con el objeto se convierte en gaussianas (un shadow catcher). Una malla solo es una entrada del horneado.	§3.9, §7.9
Fidelidad ante todo	La referencia es el path trace de la malla. Donde la calidad y el frame rate chocan, gana la calidad («aunque empeore el rendimiento quiero calidad», 12 de octubre de 2026).	§4.10, §7.10
Presupuesto de memoria	50–60 MB de datos del coche residentes mientras la cámara se mueve (unos 55 MB en el reproductor actual), unos 100 MB en reposo; el estado de render se cuenta aparte.	§6.8, §7.10, §7.11

restricción	enunciado	dónde actúa
Automático, no manual	Las decisiones por asset se detectan a partir de los datos: el tamaño de celda a partir de una cámara, el vidrio sólido frente al fino, las cáscaras cerradas, el suelo para el shadow catcher. Las opciones solo lo sobrescriben.	§3.3, §3.5, §3.8, §3.9
Las piezas pequeñas deben verse bien	Emblemas, rejillas, molduras y faros se juzgan con el mismo cuidado que los paneles, tanto de cerca como de lejos.	§3.11, §6.9, §7.12

1.3 Contribuciones

Las contribuciones que describe este artículo son las siguientes.

1. **Un renderizador USD solo en GPU** (athenea, sección 2) que dibuja mallas, puntos y splats de gaussianas a través de Hydra, con un rasterizador por teselas y un trazador de rayos de gaussianas que comparten una única disposición de gaussianas, un compilador de materiales de MaterialX a Slang comprobado frente al propio MaterialX, un path tracer con muestreo por importancia múltiple usado como verdad de referencia, y una disciplina de verificación en la que toda comparación de imágenes es a su vez un kernel de GPU.
2. **Una conversión de malla a gaussianas** (mesh2splat, sección 3), ejecutada como efecto AOFX en el dispositivo, con celdas derivadas de la huella de un píxel de una cámara, una opacidad que reproduce la cobertura de una superficie, bordes duros y cáscaras cerradas detectados en el dispositivo, ensanche de Toksvig de la rugosidad por la dispersión de las normales, captura de OpenPBR por capas y clasificación automática del vidrio.
3. **Un transfer reiluminable horneado en cada gaussiana** (TX, sección 4): armónicos esféricos de grado 3 del transfer directo e indirecto, un campo reflejado normalizado sobre las direcciones cerradas y acoplado al cielo al dibujar, y 256 bits de direcciones abiertas que cada lóbulo de reflexión lee como conos; lóbulos nítidos por píxel guiados por un operador de forma almacenado, con el ensanche del horneado deshecho y un tubo distinguido de un pliegue.
4. **Nubes de gaussianas con skinning** (sección 5) que llevan el rig y no los fotogramas: un kernel de skinning que usa el jacobiano completo, incluidos los gradientes de los pesos, lóbulos zonales de transfer que giran con la gaussiana, campos de visibilidad por parte leídos como un producto de consultas a tablas, y una decimación que conserva el rig.
5. **Árboles de fusión por pieza con un corte por error acotado** (sección 6): fusión por pares bajo guardas que respetan piezas y materiales, una opacidad de cobertura por mínimos cuadrados para las fusiones, un error hecho monótono con esferas envolventes para que una sola prueba paralela seleccione el corte, cull de caras traseras de cáscaras cerradas, un codebook de oclusión compartido y conservador, y páginas ordenadas por el error del padre.
6. **Un camino de compresión y streaming hasta un móvil** (sección 7): un contenedor `.athc` v3 con codificaciones por planos de bytes, PCA por clusters del transfer mantenido comprimido en la GPU, bits de direcciones abiertas cuantizados vectorialmente, un registro compacto de 40 bytes, registros de recorrido cuantizados de 5–11 bytes, y una política de tiempo de frame y de fotograma quieto que mantiene los datos del coche dentro de su presupuesto.
7. **Un relato medido** (sección 8) de lo que aporta cada paso frente al path trace, sobre un asset de producción, desde el horneado hasta un iPhone real.

1.4 Organización

La sección 2 describe el motor y el renderizador. La sección 3 describe la conversión de mallas USD a gaussianas, y la sección 4 el transporte de luz horneado en ellas y cómo se reiluminan. La sección 5 trata los assets animados: el skinning, el gorrión y el peón. La sección 6 trata los niveles de detalle, y la sección 7 la compresión y el reproductor WebGPU en el iPhone. La sección 8 consolida la evaluación de las secciones 2–7, la sección 9 reúne las limitaciones y el trabajo pendiente, y la sección 10 enumera las referencias, seguidas de un apéndice sobre los repositorios, los comandos y las figuras.

1.5 Trabajo relacionado

Splatting de gaussianas. Kerbl et al. (2023) introdujeron el splatting de gaussianas 3D: una escena como gaussianas anisótropas con color en armónicos esféricos, optimizadas a partir de fotografías y dibujadas por un rasterizador por teselas ordenado en profundidad. Su proyección es el splatting EWA de Zwicker et al. (2001, 2002), una aproximación afín local de la proyección perspectiva. Mip-Splatting (Yu et al., 2024) añade un filtro de suavizado 3D y un filtro de pantalla 2D contra el aliasing; la dilatación de 0.3 px^2 de athenea, con su compensación de opacidad, pertenece a esa familia (§2.5.1). El splatting de gaussianas 2D (Huang et al., 2024) aplana cada gaussiana en un disco orientado que se evalúa por intersección rayo–plano, en la línea de los surfels (Pfister et al., 2000); los surfels de athenea lo siguen (§2.5.2). El trazado de rayos de gaussianas 3D (Moënne-Loccoz et al., 2024) traza partículas a través de proxies envolventes en una BVH, y la ruta por hardware de athenea lo sigue (§2.6).

De mallas a gaussianas. El mesh2splat de Electronic Arts convierte una malla en gaussianas rasterizando cada triángulo en el plano de su eje dominante, una gaussiana por texel cubierto; la conversión de athenea parte de él (§3.1). Mesh2GS (arXiv 2606.21898) deriva el tamaño de una gaussiana de la huella de un píxel de una cámara, y `--cell-from-camera` de athenea lo sigue (§3.3).

Transporte de luz precalculado. La transferencia de radiancia precalculada (Sloan, Kautz y Snyder, 2002) guarda, por punto, la proyección de la visibilidad por el coseno sobre armónicos esféricos, de modo que reiluminar es un producto escalar con los coeficientes del entorno; Ramamoorthi y Hanrahan (2001) mostraron que nueve coeficientes representan bien la irradiancia. El PCA por clusters (Sloan et al., 2003) comprime esos transfers agrupándolos y con una base principal por cluster, y athenea lo usa tanto en el fichero como en la GPU del móvil (§7.2). Los armónicos zonales, que giran girando un eje, se ajustan según Sloan (2008) (§4.4.5, §5.7). El muestreo por importancia filtrado (Křivánek y Colbert, 2008) prefiltra el entorno (§4.3), y Lagarde y de Rousiers (2014) dan el ajuste de oclusión especular que usaba el primer transfer (§4.6.1).

Antialiasing especular. Toksvig (2005) ensancha una distribución de microfacetas por la varianza de las normales que promedia; athenea aplica la misma idea en la conversión, a la dispersión de normales bajo una gaussiana (§3.7, §4.9).

Niveles de detalle para gaussianas. Octree-GS (Ren et al., 2024) organiza anclas en un octree seleccionado por la distancia de la vista. Las gaussianas 3D jerárquicas (Kerbl et al., 2024) construyen una jerarquía de fusión a partir de los momentos de las gaussianas y la cortan por tamaño en pantalla; el nivel de detalle original de athenea sigue sus momentos (§6.1). NanoGS (Xiong et al., 2026) fusiona por pares vecinos más cercanos mutuos, y los árboles por pieza de athenea lo ejecutan en el dispositivo (§6.2). El coste usa la cota de Runnalls (2007) para la reducción de mezclas de gaussianas, y las páginas por bandas de error siguen el uso que hace Nanite (Karis et al., 2021) de un error monótono para un corte paralelo (§6.3, §6.8).

Motores y formatos. El motor está construido sobre OpenUSD e Hydra, materiales MaterialX y OpenPBR, shaders Slang compilados mediante slang-rhi a Metal, Vulkan y CUDA, Intel Open Image Denoise y OpenColorIO. El reproductor es un fork de Spark con backend WebGPU.

1.6 Convenciones

Salvo que se diga otra cosa, las medidas se tomaron en un Apple M5 Pro con Metal. Las imágenes se comparan en la GPU (§2.15). El **PSNR** está en dB sobre imágenes codificadas para pantalla; el **relMSE** es el error cuadrático medio relativo sobre valores lineales. **El path trace** (PT) del Corvette es la malla trazada por caminos desde `/World/Camera` a 1920 × 1080 bajo el cielo `autoshop_01_4k`. El **recorte del coche** es el PSNR sobre el recorte de 900 × 420 del coche, el **fotograma** sobre el fotograma entero; un **cociente por clase** es la media de una región sobre la del path trace. El arnés validate-corvette del reproductor mide otras regiones (coche, puerta, umbral, parachoques, capó), que no son el recorte del coche (§7.12.3). Los identificadores de prueba (T15–T75) son los del registro del trabajo de pieces, y los hashes de commit son los del repositorio que se nombra (apéndice A). Las palabras *gaussiana* y *splat* se usan indistintamente; una *hoja* es una gaussiana horneada, una *fusión* (merge) un nodo de un árbol de fusión, y un *corte* el conjunto de nodos que se dibujan para una vista. *Transfer* (transferencia) es el transporte de luz horneado (§4), *TX* su forma actual. Las figuras que las secciones nombran como rutas de fichero se enumeran en el apéndice A.3.

2. El motor y el renderizador athenea

athenea es un renderizador de USD construido para que todo paso que toca datos de la escena se ejecute en la GPU. Dibuja tres clases de primitiva: mallas, con materiales MaterialX y luces UsdLux; puntos; y splats gaussianos 3D. Ofrece dos técnicas, un raster interactivo y un trazador de caminos que sirve de verdad de referencia. Hydra alimenta ambas a través del delegado de render `hdAthenea`. Esta sección cubre la arquitectura del motor, los algoritmos que ejecuta cada subsistema, cómo se establece la corrección sin un oráculo en CPU y lo que se ha medido. Salvo que se indique otra cosa, cada número se cita de `docs/decisions.md`, con su unidad y sus condiciones, y se midió en un Apple M5 Pro con Metal.

2.1 Reglas de diseño

El código se atiene a un número reducido de reglas. Cada una se hace cumplir en la revisión y, donde es posible, con una prueba.

1. **Ninguna aritmética de CPU sobre los datos.** athenea no tiene renderizador de referencia en CPU, ni alternativa en CPU, ni oráculo de pruebas en CPU. El host lee ficheros, analiza cabeceras, descomprime y lleva las cuentas: recuentos, ranuras y colas. La decodificación, la ordenación, la fusión, el descarte, la generación de mips, los prefijos de potencia de las luces y la comparación de imágenes son todos kernels de cómputo. Cuando apareció un paso que rompía esta regla, se llevó al dispositivo y el traslado quedó registrado. Son ejemplos el entrelazado por elemento de los arrays de `ParticleField`, la conversión de half a float, la conversión por píxel de los búferes de render de Hydra y la parte acumulada de cada luz en la potencia del fotograma (`light_prefix.slang`). La única excepción documentada es la teselación de Dear ImGui, que procesa la interfaz del visor y ningún dato de la escena.
2. **Enlace por nombre.** Todo parámetro de shader se enlaza por reflexión (`cursor["name"].setBinding(...)`, `ComputeKernel::dispatch`) y nunca por ranura. La regla viene de errores de recuento de ranuras sufridos en el compositor que precedió a este motor.
3. **Errores como valores.** Los errores se devuelven como `Result<T>` con `ATHENEA_TRY`, y nada lanza excepciones. Las llamadas al sistema operativo viven solo en `modules/core/Platform`, que es también donde empezaría un port a Windows.
4. **Un Slang, un slang-rhi, un TBB por proceso.** OpenUSD, OIDN y el motor comparten un único TBB, y la prueba `single_tbb` comprueba el recuento. Por la misma razón OIDN se compila desde el código fuente: sus binarios publicados traen un segundo `libtbb.12`.
5. **Solo dispositivos GPU, en todas partes.** OIDN se compila solo para dispositivos GPU, porque un eliminador de ruido que recurre a la CPU es una alternativa en CPU. Una prueba cuya capacidad falta en un dispositivo se omite y nombra la capacidad que falta.

Los backends son los de slang-rhi: Metal (macOS, iOS), Vulkan y CUDA (Linux, NVIDIA L4). Todos los kernels están escritos en Slang.

2.2 Arquitectura de módulos

Cada módulo bajo `modules/<name>` es una biblioteca estática, `athenea::<name>`. Los módulos se listan en orden de dependencia y cada uno enlaza solo con los anteriores (`modules/CMakeLists.txt`).

Módulo	Contenido
core	`Result`, registro, `Platform` (todas las llamadas al SO)

Módulo	Contenido
ui	los paneles del visor como descripción (`Controls.h`, `GaussianPanel.h`), recorridos por `athenea view` y la app de iOS
sched	`FrameClock`: genlock PTP y alineación ST 2059-1, timecode
image	`Image`: float32 RGBA premultiplicado, origen abajo a la izquierda; el tipo que intercambian gpe y AOFX
io	lectores de ficheros en CPU (cabecera PLY, SPZ, SOG zip/WebP, IES), EXR con atributos
gpu	dispositivo slang-rhi, `ShaderLibrary`, `ComputeKernel`, `CommandBatch`, `Buffer`, `AsyncReadback`; `gpu/algo` (PrefixSum, RadixSort)
gpu_host	gpe adoptando el dispositivo de slang-rhi: un `MTLDevice` o contexto CUDA, búferes compartidos sin copias
colour	`ColourCompiler` (OpenColorIO como compilador a Slang), `ColourNames`
scene	`CloudLoader`: registros en bruto subidos y decodificados en la GPU en `GpuSplats` / `GpuPoints`
render	`TileRasterizer`, `GaussianRayTracer`, `PointRasterizer`, `ReferenceRenderer`, `Camera/Projection`, `SplatEdit`
geom	`MeshBuilder`: mallas de Hydra trianguladas, con normales suaves y primvars expandidos en la GPU
material	`TextureStore`, `MaterialCompiler` (MaterialX → Slang), la biblioteca de lóbulos
light	luces UsdLux en el dispositivo: un registro por luz, muestreo, densidades, BVH de luces
world	`GpuScene`, `Instancing`, `RayTracingScene` (BLAS/TLAS), `BvhScene` (LBVH de cómputo en dos niveles)
technique	`VisibilityRaster/Trace/Bvh`, sombreado, `AovShading`, trazador de caminos, `Denoiser`, `DisplayTransform`, `SplatVisibility`
lod	`LodBuilder`, `CutSelector`, el lector/escritor de `.athc` y `StreamingPool` (Sección 6)
usd	`Engine`, `StageRenderer`, `Export`, el plugin `hdAthenea`, esquemas sin código
mcp	el motor como servidor MCP (JSON-RPC 2.0)
aofx	el lado de este motor del host AOFX (`host/` de aopenfx), `renderEffect`, los bundles
view	`athenea view`: ventana GLFW, Dear ImGui en el dispositivo del motor

El árbol de shaders `shaders/athenea/` refleja los módulos (common, algo, scene, splat, rt, points, reference, lod, geom, world, material, light, technique, usd, volume, view). Los shaders se copian al árbol de compilación y se compilan en tiempo de ejecución.

2.3 Fundamentos de GPU

Dispositivo y kernels. `gpu::Device` envuelve un dispositivo slang-rhi. `ComputeKernel` es un punto de entrada de cómputo compilado como pipeline. Se despacha en hilos, no en grupos: el número de grupos se redondea hacia arriba en todos los backends y cada kernel comprueba sus propios límites. `RasterKernel` es un pipeline de vértices y fragmentos que toma sus datos de `StructuredBuffers`, sin búferes de vértices ni layouts de entrada. `RayTracingKernel` cubre los pipelines de OptiX y de Vulkan RT. En Metal, slang-rhi no ofrece pipelines de trazado de rayos, así que el trazado usa `RayQuery` en línea dentro de kernels de cómputo.

ShaderLibrary. La biblioteca carga módulos Slang. Un módulo puede declarar constantes de enlace (`extern static const uint kName;`), y entonces se enlaza contra un módulo de exportaciones generado, lo que da un programa por cada conjunto distinto de valores. También compila módulos que solo existen como código generado, como los materiales y las funciones de color; se rechaza cargar un nombre existente

con otro código. Una `DiskShaderCache` persistente escribe un fichero por programa compilado, con la clave del hash que slang-rhi da al programa enlazado. Con la caché caliente, la batería de pruebas completa pasó de 89 s a 26 s.

Primitivas paralelas. `PrefixSum` y `RadixSort` son las únicas ordenaciones y barridos del motor. La ruta radix por bloques (`radix_block.slang`) es una forma reducir-y-luego-barrer de la pasada LSD estable. Un grupo de 256 hilos cuenta un bloque de 1024 claves en memoria de grupo, una suma prefija ordenada por dígito sitúa la parte de cada bloque en cada dígito, y el grupo ordena su bloque con ocho divisiones de un bit. Ordena 10 M de pares de 32 bits en **12.4 ms**, frente a 39.9 ms de las pasadas por trozos a las que sustituyó a partir de 65 536 elementos. No usa operaciones de onda ni atómicas de dispositivo.

Mips. slang-rhi no genera cadenas de mips, así que `MipGenerator` es un kernel. Cada texel de un nivel es la media ponderada por área de los texels de encima. Un borde impar $2n+1$ se pliega en n con pesos $(n-x,\ n,\ x+1)/(2n+1)$, de modo que cada texel de origen da exactamente $n/(2n+1)$ de sí al nivel inferior y la cadena conserva la media del nivel 0. En tamaños 64^2, 37×23, 1×9 y 128×5, las medias coinciden a $2\cdot10^{-6}$.

La memoria como error de primera clase. Un command buffer de Metal que se quedaba sin memoria provocaba una aserción dentro de slang-rhi en el hilo de finalización de Metal, que terminaba el proceso. Un parche registra ahora el error en la cola (`SLANG_E_OUT_OF_MEMORY`) y lo devuelve en el siguiente `submit()` o `waitOnHost()`. `Device::admit` rechaza toda reserva que exceda el presupuesto: `ATHENEA_GPU_BUDGET` o, en su defecto, el `recommendedMaxWorkingSetSize` de Metal, que es 18 186 MiB en el M5 Pro. Una reserva debe caber además en la memoria física libre menos 1.5 GiB. En memoria unificada, cuatro trabajos que estaban cada uno por debajo de la cifra de Metal hicieron paginar juntos a la máquina hasta que el servidor de ventanas perdió su watchdog. Cuando un fotograma se queda sin memoria, `Engine::relieveMemory` baja escalones: primero suelta las sombras de splats, después pasa hasta cuatro niveles de LOD más gruesos y por último reabre los streams a la mitad de su presupuesto.

2.4 La representación gaussiana

Todos los renderizadores, los cargadores y el LOD comparten un único layout en GPU, `GpuSplats`. Cada splat ocupa **32 bytes antes de sus armónicos**, definidos en `common/packing.slang`:

Campo	Codificación	Razón
posición, opacidad	4 × float32	posiciones en fp16 saltarían dos píxeles a 4K de cerca
rotación	cuaternión smallest-three, 3 × 10 bits + índice omitido	dentro de una décima de grado
escala	f16 de log*s* por eje	0.1 % de precisión relativa a cualquier tamaño
color base	3 × f16	puede salir de [0, 1]
resto de SH	`shWords` × u32, f16 por coeficiente	con signo
normal (opcional)	octaédrica 2 × 16 bits	normal de sombreado separada del marco
emisión (opcional)	RGB9E5	radiancia que emite la superficie

`CloudLoader` lee PLY, SPZ, SOG, `.splat` y prims `ParticleField` de USD. Sube los registros en bruto y los decodifica en la GPU. Para SOG, el único trabajo del host es descomprimir el zip y decodificar las imágenes WebP.

Luz lineal. Toda gaussiana se mezcla en Rec.709 lineal, y el sRGB solo aparece donde se muestra una imagen. Una captura conserva los armónicos sRGB con los que se entrenó: cada proyección los evalúa, los recorta en cero y los convierte a lineal (`cloudLight`) antes de mezclar. Una nube escrita por athenea declara que contiene luz (`primvars:athenea:splat:linear`) y se usa tal como está almacenada. Sobre dos discos semiopacos solapados, el rasterizador, la referencia y el trazador de rayos coinciden cada uno con el operador over analítico a p99 0. Con la mezcla codificada anterior, los tres leían p99 26.

2.5 El rasterizador por teselas

`render::TileRasterizer` es la ruta interactiva para splats. Es un rasterizador por teselas determinista, basado en ordenación, de la familia de 3DGS. Siete etapas se ejecutan en el dispositivo:

Etapa	Trabajo
project	por splat: covarianza, cónica, color, clave de profundidad de 24 bits, huella, número exacto de teselas tocadas
sort	splats por profundidad (RadixSort)
gather	recuentos de teselas tocadas en orden de profundidad, suma prefija
emit	un par (tesela, splat) por tesela tocada, en orden de profundidad
sort	pares por tesela, de forma estable, solo con los bits de tesela
ranges	inicio y fin de los pares de cada tesela
blend	por píxel: los pares de su tesela, de delante hacia atrás

El motor ordena los splats por profundidad una sola vez (N claves de 32 bits) y después ordena los pares de forma estable por tesela (P claves de unos 15 bits). Así cada tesela recibe sus splats en orden de profundidad, con los empates resueltos por índice de splat, exactamente como lo haría una única ordenación de 64 bits por (tesela, profundidad). La construcción es determinista. Las teselas son de 16×16 píxeles.

2.5.1 Proyección

Un splat con rotación R y escalas $S = \mathrm{diag}(s)$ tiene la covarianza 3D $\Sigma = RSS^{\top}R^{\top}$, construida en el espacio propio de la nube. Con W la parte lineal de objeto a vista y J el jacobiano de la aplicación perspectiva en el centro en espacio de vista t, la covarianza en pantalla es la aproximación afín local EWA

$$\Sigma_{2D} = (JWM)(JWM)^{\top}, \qquad M = RS, \qquad J = \begin{pmatrix} f_x/t_z & 0 & -f_x t_x/t_z^2 \\ 0 & f_y/t_z & -f_y t_y/t_z^2 \end{pmatrix}.$$

Antes de construir J, t_x/t_z y t_y/t_z se acotan al frustum ensanchado a $[-0.15, 1.15]$ de la imagen. Un splat muy fuera de pantalla se extendería si no por todo el fotograma, y 3DGS acota de la misma manera. Las cámaras ortográficas usan $J = \mathrm{diag}(f_x, f_y)$.

Obturador y lente, como convoluciones. Una gaussiana arrastrada a lo largo de un camino recto en pantalla d bajo un obturador uniforme es esa gaussiana convolucionada con un segmento. Igualar segundos momentos convierte esto en una actualización de rango uno. Una lente delgada convoluciona con el disco del círculo de confusión:

$$\Sigma' = \Sigma + \frac{1}{12} d\, d^{\top}, \qquad \Sigma' = \Sigma + \frac{R^2}{4} I, \quad R = f A \left| \frac{1}{z} - \frac{1}{z_f} \right|.$$

El desplazamiento d tiene dos fuentes: lo que un esqueleto movió el splat durante el obturador (un búfer por splat) y un paso en espacio de vista que combina el movimiento de la cámara y el del prim (`viewStep`). Ambos se proyectan con las mismas filas JW que la covarianza. La actualización se comprueba frente al

mismo rasterizador ejecutado en N instantes y promediado en el dispositivo (§2.5.3).

Filtro antialiasing y la opacidad que devuelve. A continuación la covarianza se dilata en 0.3 px², como en el filtro 2D de Mip-Splatting. Cada convolución ensancha el splat, así que su pico debe bajar en $k = \sqrt{\det\Sigma_{\text{still}}/\det\Sigma'}$ para conservar su energía. Una sola raíz cuadrada cubre a la vez el filtro, el obturador y la lente. Escalar sin más pierde masa, sin embargo, porque un splat solo se dibuja hasta donde cae por debajo de 1/255, y el ensanchamiento agrandó esa área en $1/k$. Un plano convertido que debía cubrir 0.25 cubría 0.18 a una vigésima de la pantalla. Por eso la opacidad se devuelve como

$$\alpha' = \begin{cases} k\alpha + (1-k)/255 & k\alpha \geq 1/255 \\ k\alpha & \text{en otro caso.} \end{cases}$$

Esto conserva el área cubierta del splat nítido (`paidOpacity`). Devolverla a todos los splats dibujaría también los que el filtro había dejado demasiado tenues para conservarse. Eso subía un peón convertido lejano de 8.7 a 29.8 ms (compilación debug), y la regla de arriba cuesta allí 11.0 ms.

Huella y teselas. Un splat se corta en el radio de Mahalanobis al cuadrado

$$r^2_{\max}(\alpha) = \min\big(2\ln(255\,\alpha),\ 9\big),$$

que es donde su propia contribución cae por debajo de 1/255, y nunca más allá de 3σ. El recuento de teselas es exacto en lugar de tomarse de una caja envolvente. Para cada tesela de la extensión alineada con los ejes, el mínimo de la forma cuadrática $q(\delta) = \delta^\top \Sigma'^{-1}\delta$ sobre la caja de los centros de píxel de la tesela se calcula en forma cerrada: como q es convexa, es 0 o está en una arista, donde es una cuadrática 1D acotada. La etapa emit repite la misma prueba. Como la mezcla corta en el mismo radio, ningún píxel fuera de una tesela contada pudo ser tocado, así que los bordes de las teselas no dejan costuras. Una ranura descartada escribe el motivo del descarte en su palabra de clave de profundidad sin usar (edit, profundidad, sin área, tenue, fuera de pantalla, sin tesela), y el panel Gaussians del visor cuenta esos motivos sin añadir un contador a la proyección.

Clave de profundidad. La clave es `bit_cast<uint>(z) >> 7`, que conserva el orden para floats positivos y tiene 24 bits.

2.5.2 Mezcla

Un grupo de 16 × 16 hilos por tesela carga los registros de la tesela en tandas de 256 en memoria de grupo, y cada píxel los recorre desde ahí. Para el píxel p y el splat i con cónica $Q_i = \Sigma_i'^{-1}$, centro μ_i y color c_i:

$$\alpha_i(p) = \min\Big(0.99,\ \alpha_i' \exp\big(-\tfrac{1}{2}(p-\mu_i)^\top Q_i (p-\mu_i)\big)\Big), \qquad C = \sum_i c_i\,\alpha_i \prod_{j<i}(1-\alpha_j) + T\,C_{\text{behind}}.$$

Las contribuciones por debajo de 1/255 se saltan. Un píxel se detiene en cuanto $T = \prod(1-\alpha_j) < 10^{-4}$, y el grupo deja de cargar cuando sus 256 píxeles han terminado. C_{behind} es el fondo o la capa opaca *under* de mallas y puntos, compuesta por z de vista: el recorrido termina en el primer registro detrás de la profundidad de la capa. La profundidad se escribe como la media ponderada por alfa o como el primer cruce de un umbral de cobertura. El mismo recorrido acumula también un Cryptomatte de seis rangos, por cobertura de id, y, para la poda, la contribución por splat $T\alpha$ o el error cuadrático que dejaría quitar el splat (la puntuación de GaussianPOP). Puntos de entrada especializados eliminan en compilación las ramas de composición y de mate, porque dejarlas vivas en un fotograma que no tiene ninguna de las dos costaba a la mezcla la mitad más.

Surfels y núcleos más duros (fork `athenea-pieces`). Una gaussiana con una escala exactamente 0 es un *surfel*. El rasterizador lo dibuja intersecando el rayo de cada píxel con el plano del disco (Huang et al. 2024): los ejes del disco se guardan en coordenadas homogéneas de pantalla y se llevan de vuelta a través de los dos planos del píxel. El filtro de pantalla es un máximo, no una convolución: el mayor entre el disco y una gaussiana redonda de varianza ½ px². Su opacidad se devuelve por área,

$$\alpha' = \alpha \cdot \min\left(1,\ \frac{\sqrt{\lambda_1\lambda_2}}{\sqrt{\lambda_1\lambda_2}\ + \frac{1}{2} - \sqrt{\min(\lambda_1, \frac{1}{2})\min(\lambda_2, \frac{1}{2})}}\right),$$

donde $\lambda_{1,2}$ son los autovalores de la covarianza en pantalla sin dilatar. Una gaussiana fusionada puede llevar una opacidad $D > 1$. Ese valor selecciona el núcleo más duro de Spark, $1-(1-g)^A$ con $A = \exp((D^2-1)/e)$ y D limitado a 5. Es la composición de A gaussianas coincidentes, y su corte se ensancha en $0.7(D-1)\sigma$. Un *surfel duro* usa $\exp(-(|u|^p + |v|^p)/2)$, con p ajustado a la curtosis de sus miembros. La Sección 6 cubre ambos, junto con las fusiones que los producen.

2.5.3 Medidas

Cobertura de superficies convertidas (`athenea_coverage_tests`, 256 × 256, alfa de planos convertidos por mesh2splat, frente al alfa de la malla; los tamaños son fracciones de la pantalla completa):

Plano	Malla	Raster 1× / 0.25× / 0.05×	Trazado 1× / 0.25× / 0.05×
constante 0.25	0.250	0.251 / 0.250 / 0.248	0.251 / 0.251 / 0.251
constante 0.5	0.500	0.501 / 0.499 / 0.493	0.501 / 0.501 / 0.500
constante 0.75	0.750	0.752 / 0.749 / 0.736	0.752 / 0.752 / 0.752
opaco	1.000	1.000 / 1.000 / 0.999	1.000 / 1.000 / 1.000

Sin `paidOpacity`, el raster leía 0.18 / 0.455 / 0.73 a 0.05×.

Desenfoque de movimiento y lente (`[motion]`, `[lens]`; verdad = el mismo rasterizador en N instantes, o en 64 posiciones de lente, promediado en el dispositivo):

Caso	Resultado
paso de obturador 0.10 (muchos anchos de splat)	p99 8, máx 24
paso de obturador 0.03	p99 2, máx 9; razón de energía 1.0000
paso de obturador 0.9	razón de energía 0.9030 (colas bajo el corte de 1/255)
obturador nulo frente a fotograma quieto	máx 0, bit a bit
f/2, 12.7 px de confusión	p99 17 (sin desenfocar: p99 113)
f/11, 2.3 px de confusión	p99 9 (sin desenfocar: p99 33)
a la distancia de enfoque	máx 0, bit a bit

Coste del fotograma por etapa (`ATHENEA_STAGES=1`, esperando a cada etapa; los clips del gorrión, 5 887 323 gaussianas a 1600 × 1000, en reproducción):

Etapa	Antes	Después	Cambio
ordenación por profundidad (24 bits)	11.3 ms	6.6 ms	radix por bloques
ordenación por tesela (8.8 M pares)	8.4 ms	6.0 ms	radix por bloques
mezcla	13–25 ms	8–13 ms	registros de la tesela compartidos en memoria de grupo

Etapa	Antes	Después	Cambio
proyección, emisión, recuentos, commit	13.6 ms	13.4 ms	–
fotograma, en reproducción	102 ms	49 ms	la nube ya no se resube en cada paso

Un fotograma de seis millones de gaussianas gasta su tiempo en las gaussianas y no en los píxeles: cambiar la escala de render de 800 × 500 a 3200 × 2000 deja el tiempo igual.

2.6 El trazador de rayos de gaussianas

`render::GaussianRayTracer` evalúa cada partícula en 3D en su respuesta máxima a lo largo del rayo, sin aproximación EWA y sin la dilatación de 0.3 px. Las partículas se mezclan de delante hacia atrás en el orden de su pico a lo largo del rayo. En el marco blanqueado de la partícula (origen o, dirección d, con $\Sigma^{-1/2}$ aplicado), el pico y la entrada en el elipsoide de corte son

$$t^* = -\frac{o \cdot d}{d \cdot d}, \qquad \rho = \left\| o + t^* d \right\|^2, \qquad t_{\text{entry}} = t^* - \sqrt{(r_{\max}^2 - \rho)/(d \cdot d)}\,.$$

El discriminante de libro $b^2 - ac$ se cancela catastróficamente en partículas planas y perdía 176 de 400 capas finas, por eso se usa la forma del pico. El color es el del rasterizador: los armónicos evaluados una vez por fotograma en la dirección del ojo al centro.

Dos rutas, un integrador. Ambas rutas alimentan el mismo `Integrator` (`rt_integrate.slang`). La ruta *Hardware* da a cada partícula un proxy de icosaedro estirado (el de 3DGRT) en una BVH del dispositivo consultada con `RayQuery` en línea. La ruta *ComputeBvh* es una LBVH de Karras construida en la GPU (códigos de Morton, radix sort, jerarquía, reajuste hasta que una pasada no cambia nada) y recorrida en cómputo. Cada recorrido registra hasta 256 entradas sin ordenar, que se evalúan y ordenan tras el recorrido con una ordenación por conteo en 128 cubetas y después una ordenación por inserción sobre una permutación de índices. Cuando un rayo encuentra más proxies que esos, el segmento se cierra en la entrada descartada más cercana y el siguiente recorrido se reanuda un relativo 10^{-5} antes de ella. Las partículas cuyo pico cae más allá del límite se arrastran al siguiente segmento; solo cuando se arrastran más de 64 se mezclan antes las más cercanas, que es la única aproximación de la ruta.

train_7k (742 k splats), 1080p	Fotograma	Construcción
Hardware (Metal)	435 ms	731 ms
ComputeBvh (Metal)	343 ms	168 ms
rasterizador por teselas, mismo fotograma	13 ms	–

En Metal, cada candidato no opaco cuesta un viaje de ida y vuelta fuera del recorrido por hardware. Una consulta opaca de impacto más cercano sobre los mismos 15 M de triángulos proxy tarda 6 ms, pero enumerar todos los candidatos tarda 255 ms, así que `Auto` elige ComputeBvh en Metal. La historia de optimización de este fotograma fue de 2.52 s (un k-buffer de 16 entradas al estilo de 3DGRT) a 343 ms. Los pasos mayores fueron registrar entradas sin ordenar y evaluar tras el recorrido (1.94 s → 658 ms) y la ordenación por cubetas (−85 ms); el tamaño de registro de 256 se eligió de un barrido de 64 a 512.

Consultas de sombra y secundarias. La transmitancia es un producto, así que `rt_shadow.slang` toma los proxies en el orden en que llegan, sin k-buffer, y se detiene cuando el producto cae por debajo de su corte. No descarta caras traseras, porque un rayo de sombra nacido dentro de un proxy debe seguir viendo la partícula en la que empieza. Tres formas cerradas lo comprueban: $1 - \alpha$ a través de un centro, medido 0.50000; $(1 - \alpha)^4$ a través de cuatro, medido 0.06250; y la terminación temprana tras tres de cuatro. El in-

tegrador responde a una consulta `rtTraceSegment(ray, [a, b))` en lugar de escribir un píxel, y `segmentOver` compone dos segmentos: una nube dispersa partida a cualquier profundidad es exacta bit a bit con la consulta única.

2.7 Puntos

`PointRasterizer` dibuja nubes de puntos con un pipeline raster con z-buffer y sin ordenación, seguido opcionalmente de surface splatting y de eye-dome lighting. En dispositivos sin rasterización (CUDA), los puntos se dibujan como discos en el pipeline de splats. En ese caso un `colour.w > 0.5` en un registro lo marca como disco opaco, que termina el recorrido exactamente como lo hace la capa under.

2.8 Mallas: geometría, escena y visibilidad

En el dispositivo, en el orden de Hydra. `geom::MeshBuilder` triangula la topología de Hydra en el orden de abanico de `HdMeshUtil`, con agujeros y orientación levógira incluidos. Calcula normales suaves con la fórmula de `Hd_SmoothNormals`, como productos vectoriales por esquina dispersados a los puntos mediante un radix sort, y expande primvars de toda interpolación por esquina, resolviendo los primvars indexados y decodificando los dobles en el dispositivo. Las normales de un campo de alturas quedan a $1.1{\cdot}10^{-7}$ rad de una suma por fuerza bruta. `world::GpuScene` agrupa posiciones, índices, esquinas, caras y valores de primvars, con un registro de 64 bytes por malla y un `InstanceRecord` de 176 bytes por copia dibujada. `world::Instancing` compone las cadenas de instanciadores exactamente como Storm, instancer $\cdot\, T \cdot R \cdot S \cdot$ instanceTransform, anidadas como parent$[i] \cdot$ level$[j]$, con un único dispatch para todos los conjuntos instanciados. En los 1462 conjuntos de Kitchen_set_instanced esto llevó la grabación de comandos de 32 ms a 0.4 ms.

Tres rutas, un búfer de visibilidad. Cada ruta escribe (instancia + 1, triángulo) por píxel. El sombreado y los AOVs reconstruyen el impacto a partir de esos dos números con Möller–Trumbore en espacio de vista (`technique/surface.slang`), así que todas las rutas sombrean igual.

- `VisibilityRaster` usa Z infinita invertida (profundidad $=$ near$/z$, D32Float), un draw por malla con `instanceCount` y un único objeto raíz compartido.
- `VisibilityTrace` construye una BLAS por malla y una TLAS por fotograma. Un kernel escribe los descriptores de instancia en el layout de cada backend (64 B genérico, 80 B OptiX, 68 B Metal).
- `VisibilityBvh` construye una LBVH de Karras por malla y otra sobre las instancias, para dispositivos sin hardware de RT.

El descarte de cara única coincide con Storm en las tres rutas.

Storm como oráculo (Kitchen_set, 480 × 270; ids comparados como segmentación, porque Storm numera los prims de otra forma):

Ruta	Cobertura	Segmentación	Profundidad, peor	Neye > 6/255
raster	0 difieren	18 de 57 219	$1.2{\cdot}10^{-6}$	17
rays	4 difieren	41 de 57 236	$6.3{\cdot}10^{-6}$	17
bvh	4 difieren	36 de 57 233	$6.3{\cdot}10^{-6}$	17

Tiempos de visibilidad (`athenea stage --frames 40`, mediana de los fotogramas tras el primero, sombreado con luz de cámara, lectura incluida):

Escena	Tamaño	raster	rays	bvh
Kitchen_set (1788 mallas)	480 × 270	18.0 ms	4.8 ms	8.9 ms
Kitchen_set	1920 × 1080	31.7 ms	19.2 ms	29.2 ms
Kitchen_set_instanced (1462 conjuntos)	1920 × 1080	29.1 ms	19.2 ms	28.6 ms

El raster pierde sobre todo en el lado del host: grabar unos 1800 draws cuesta unos 8 ms, porque slang-rhi escribe el estado de render por draw. Por eso el ajuste automático prefiere rayos dondequiera que el dispositivo tenga consultas de rayos.

2.9 Materiales

Texturas. El `TextureStore` usa Hio solo para leer los bytes de un fichero. Los bytes se suben en bruto y un kernel los decodifica al espacio de trabajo. Los mips son un kernel que decodifica el sRGB, filtra en luz y vuelve a codificar. UDIM es una tabla de indirección. Todas las texturas están en un único `ParameterBlock` de 1024 ranuras, que Metal toma como argument buffer. El filtrado se elige por destino con `__target_switch`: `SampleGrad` donde el cómputo lo permite, y un nivel explícito a partir del lado mayor de la huella en otro caso (CUDA). En Metal, donde existen ambos, eligen los mismos niveles. Una textura decodificada es el fichero hasta el último bit (0 de 3404 componentes difieren). Los espacios de color de entrada se resuelven con `colour::ColourNames`, y las transformaciones las genera OpenColorIO (§2.12).

De MaterialX a Slang. El generador deriva del `SlangShaderGenerator` de MaterialX 1.39.5. Cada nodo conserva su implementación genglsl o genslang salvo cuatro clases cuyo significado de rasterizador no puede valer aquí:

- el nodo de superficie, que aquí no tiene bucle de luces: su grafo de BSDF se ejecuta una vez, empujando lóbulos;
- los nodos BSDF y EDF, que empujan lóbulos en lugar de responder a una luz;
- los nodos de imagen, que muestrean la tabla de texturas;
- `heighttonormal`, que necesita derivadas de pantalla.

Un valor de BSDF es un *peso por lóbulo construido*, así que `mix`, `layer`, `add` y `multiply` combinan pesos. Los uniforms no se hornean en el código: cada entrada se lee de un blob de floats, y el módulo se nombra por un hash de su código, de modo que los materiales que solo difieren en valores comparten un módulo compilado. Entre los tamaños generados están UsdPreviewSurface con 580 líneas y 23 palabras de blob, standard_surface con 821 y 59, y OpenPBR con 945 y 55. `technique::MaterialPrograms` genera un módulo que importa todos los materiales compilados y despacha según la fila de material. Un kernel que evalúa materiales recorre sus píxeles en orden de quad (`atheneaQuadPixel`), porque en este dispositivo los lanes van a lo largo de las filas y todas las derivadas verticales habían salido mal (15 939 de 15 939 hilos).

La biblioteca de lóbulos. Contiene Oren–Nayar y EON con compensación de energía, Burley, translúcido, dieléctrico (reflexión, transmisión, ambas), conductor, Schlick generalizado con tinte F82, y sheen en sus formas de Imageworks y de Zeltner. Cada lóbulo tiene `eval`, `sample` y `pdf`. Los lóbulos de microfacetas muestrean la distribución de normales visibles y la transmisión sigue a Walter. Los lóbulos de un material forman una `LobeStack`, que se muestrea con MIS de una muestra según los albedos direccionales ponderados de los lóbulos.

Comprobaciones contra el propio MaterialX. Los mismos grafos se compilan una segunda vez en una variante de referencia cuyos closures son las propias respuestas genglsl de MaterialX, y un kernel evalúa ambas para 65 536 direcciones de luz. La peor diferencia por componente sobre once grafos es $\mathbf{3.2 \cdot 10^{-5}}$. En

el dispositivo, los lóbulos pasan una prueba χ^2 de las direcciones muestreadas frente a la pdf (393.2 con 399 grados de libertad para los lóbulos difusos) y un horno blanco que coincide dentro del 3 %.

Recortes y bump. La visibilidad evalúa `opacityThreshold`: el raster usa un fragment shader generado que descarta, y las rutas de rayos llevan el rayo más allá de una muestra recortada hasta dieciséis veces. Solo las filas marcadas como recortes pagan por ello. Un cuadrado recortado muestra exactamente lo que hay detrás (8464 píxeles, 0 discrepancias) en las tres rutas.

2.10 Luces

Registros y unidades. Cada luz UsdLux se convierte en un registro en espacio de mundo, en las unidades que el stage escribió. La exposición, el cuerpo negro de una temperatura de color (ajuste de Krystek, normalizado a luminancia 1) y `normalize` se derivan en el shader. Las clases admitidas son esfera, disco, rectángulo, distante con ángulo, cúpula y cilindro, además de conos de shaping y perfiles IES. Un perfil IES se lee tal como está escrito, se muestrea bilinealmente con la simetría que declara y modula la radiancia pero nunca la densidad. Las luces bajo instanciadores se componen en el dispositivo como las mallas.

Muestreo donde está la luz. Una esfera o un sol se muestrean uniformemente sobre el cono que subtienden. Un disco, un rectángulo o un cilindro se muestrean uniformemente por área y se pasan a ángulo sólido con $p_\omega = d^2/(\cos\theta_l A)$. Una cúpula se muestrea con peso coseno alrededor de la normal de sombreado o deformando su cadena de mips por luminancia. La densidad de la deformación no necesita tablas:

$$p(\omega) = \frac{L(u,v)}{\bar{L}\, 2\pi^2 \sin\theta},$$

donde $\bar{L}$ es la media del nivel de mip de 1×1. La variación de la imagen elige entre las dos estrategias: con un cielo plano la deformación se desvía un 20 % donde el coseno se desvía un 0.08 %. `lightPdf` devuelve la densidad de cualquier dirección y `lightHit` responde a la pregunta inversa, qué luz encuentra un rayo y con qué radiancia. Juntas son lo que necesita el MIS. Con `athenea:chooseLights`, una muestra toma una luz elegida mediante una **BVH de luces** (`light_bvh.slang`). Cada nodo es una caja, un cono de orientación (Conty y Kulla 2018) y la potencia que hay debajo, y la BVH se construye con los mismos kernels de LBVH de cómputo que la geometría.

Comprobaciones. Cada luz se contrasta con una forma cerrada que no comparte código con el renderizador: un plano lambertiano bajo una esfera, un disco, un rectángulo (fórmula de aristas de Lambert), un sol y una cúpula. En cada caso 0 de 8281 píxeles superan el 2 %; el peor es un 1.6 % para el disco. Cada luz ejecuta además una prueba χ^2 con 10^6 muestras, agrupadas en el soporte propio de la luz, frente a `lightPdf` integrada por intervalo: esfera $z = -0.23$, disco 0.14, rectángulo -0.27, sol -0.29, cúpula -1.61, y toda pdf integra 1.0000. La deformación de la cúpula necesitó dos correcciones, ambas halladas con χ^2 en espacio de imagen. El error llegó a $z = 61\ \ 543$ y después a $1\ \ 164\ \ 589$, frente a 0.84 tras las correcciones. La comprobación de consistencia por muestra había pasado todo el tiempo, porque compara una densidad consigo misma.

Medidas (Kitchen_set con cuatro luces escritas, 1600×900, `athenea view --frames 200`, mediana de dibujo):

Configuración	Dibujo
solo luz de cámara, rays	10.61 ms
cuatro luces, 1 muestra, rays	27.77 ms
lo mismo, BVH de cómputo	35.14 ms

Configuración	Dibujo
lo mismo, raster	40.94 ms
cuatro luces, 16 muestras, rays	275.40 ms

2.11 El trazador de caminos

El trazador de caminos es la verdad de referencia del motor. Sus caminos parten del mismo búfer de visibilidad que lee el sombreado raster, así que las dos técnicas difieren exactamente en una cosa: los rebotes. Con una lente, el trazador lanza además sus propios rayos primarios. Un impacto lo reconstruye `material_surface.slang`, su material se evalúa en la misma pila de lóbulos, y la luz directa se recoge con estimación del siguiente evento (NEE). El rebote muestrea la pila, y el throughput del camino se multiplica por `LobeSample.weight` $= f|\cos\theta|/p$.

MIS. En cada vértice que continúa el camino muestreando su material, la muestra de NEE de la luz k y los impactos en luces de la muestra del material (`lightHit`, incluidos los rayos que escapan y encuentran cúpulas y soles) se ponderan con la heurística de potencia,

$$w_{\text{light}} = \frac{p_l^2}{p_l^2 + p_m^2}, \qquad p_l = P(k \mid x)\; p_k(\omega),$$

donde $P(k \mid x)$ es la probabilidad de elección de la luz (por potencia, o mediante la BVH de luces) y p_m es `stackPdf`. Las luces delta, los lóbulos delta y el último vértice conservan peso 1. **Los triángulos emisores** forman una tabla de luces (`technique::EmissiveTable`): la emisión de cada fila de material se sondea una vez, los triángulos se ponderan por área en mundo por esa luminancia, y la elección entre luces y triángulos se hace por potencia, con la densidad de cada lado llevando la parte del otro. Sobre un suelo de rugosidad 0.2 a 32 caminos, el MIS redujo el error frente a un fotograma profundo 4.5× bajo un rectángulo, 57.7× bajo una cúpula sin imagen y 2.4× bajo una esfera. Un quad emisor y un rectángulo UsdLux equivalente coinciden a relMSE $4.1{\cdot}10^{-7}$ con 8192 caminos.

Acumulación, muestreador, adaptatividad. Un fotograma es una media móvil de color × opacidad. Una implementación anterior tomaba el producto de las medias, que solo es insesgado mientras toda muestra es opaca. El muestreador pliega píxel, flujo, índice absoluto de camino, rebote y dimensión mediante un hash PCG, así que 256 caminos en una pasada y 4 × 64 difieren en relMSE $3.9{\cdot}10^{-15}$. Entre pasadas la cámara se desplaza dentro del píxel con un desplazamiento de Halton (2, 3); la primera pasada no toma desplazamiento. Es antialiasing sin coste. El muestreo adaptativo detiene un píxel en cuanto

$$\frac{\sqrt{(\mathbb{E}[l^2] - \mathbb{E}[l]^2)/N}}{\mathbb{E}[l]} < \varepsilon$$

tras un número mínimo de caminos. La varianza usada es la mayor entre la del propio píxel y la media de la de sus vecinos 3 × 3, lo que redujo los píxeles detenidos a más de tres de sus propias σ de 526 a 100 de 4212. **La lente delgada y la distorsión radial** vienen de `UsdGeomCamera`: el radio de la lente es $f/(2N)$, y los rayos se escalan por $1 + k_1 r^2 + k_2 r^4$.

Verificación. Cada comprobación es un kernel de GPU frente a una forma cerrada o frente a un segundo estimador independiente:

Comprobación	Resultado
esfera emisora cerrada, $E(1 + \rho + \ldots + \rho^N)$, N = 0, 1, 2, 3, 6	0 de 3072 píxeles por encima de 10^{-4}, peor $1.7{\cdot}10^{-7}$

Comprobación	Resultado
horno blanco bajo una cúpula sin imagen frente a NEE sin ponderar	p99 relativo 0.0000, máx 0.0003
un rebote frente a la luz directa del raster (nada en que rebotar)	p99 1, máx 1 (códigos de 8 bits), 4096 caminos
error frente a N, pares de estimaciones independientes, 64–1024 caminos	exponente -0.499 (ventana -0.42 a -0.58)
enfocado frente a estenopeica (radio de lente 0.2)	relMSE 0, bit a bit
desenfocado frente al perfil de segmento circular	0 de 4235 píxeles por encima del 4 %
OIDN, 16 caminos frente a referencia de 4096 caminos	relMSE $3.65{\cdot}10^{-4} \rightarrow 7.17{\cdot}10^{-5}$

El horno destapó un peso de MIS de un solo lado. Con una cúpula frente a un lóbulo lambertiano las dos densidades son la misma función, así que el peso valía exactamente ½, y el horno leía 0.5453 en lugar de 1. La esfera destapó una doble división, $\rho\pi/\cos$ en lugar de ρ, cuyo segundo momento infinito había mantenido el exponente de convergencia en -0.43 a -0.45. Antes se descartaron por medida otras catorce hipótesis.

Eliminador de ruido. `technique::Denoiser` ejecuta el filtro `RT` de OIDN 2.5.1 en la propia cola del motor, guiado por el albedo y la normal del primer impacto. En Metal, OIDN solo comparte búferes con seguimiento de riesgos, que slang-rhi prohíbe. Por eso el motor copia a través de búferes intermedios con seguimiento mediante un kernel, ya que el blit de slang-rhi no hacía nada, en silencio, con un búfer envuelto (4095 de 4096 palabras intactas). En CUDA los búferes del motor se comparten directamente. En Vulkan, OIDN se ejecuta a través de CUDA sobre memoria importada (`oidnNewSharedBufferFromFD`).

Medidas (Kitchen_set, 1920 × 1080, `athenea stage --frames 11`, mediana, release):

Stage	raster	rt (1 camino, 1 rebote)
sin luces (luz de cámara)	43.1 ms	333.3 ms
iluminado (cúpula, rectángulo, dos esferas)	96.0 ms	358.9 ms

A través de MCP, el OpenChessSet trazado por caminos a 640 × 360 con 64 caminos, un rebote y ACES 2.0 tarda 25.7 s.

2.12 Visualización y color

`technique::DisplayTransform` escribe directamente la textura de superficie de la ventana. Admite Standard; AgX, en el ajuste analítico de Wrensch (log2 sobre $[-12.47, 4.03]$ pasos, una sigmoide de sexto orden); y ACES 2.0. La transformación de salida ACES 2.0 (`aces2.slang`) está portada función a función desde el CTL de la Academia, incluidos el modelo de apariencia de Hellwig 2022, la escala tonal y la compresión de gama. Las tablas que la referencia calcula al iniciar (362 entradas por matiz) las construyen en el dispositivo tres kernels. Comprobaciones: 256 grises sobre dieciséis pasos permanecen neutros a $5{\cdot}10^{-6}$; el gris del 18 % lee 0.10000 del blanco de referencia a 100 nits; inversa y después directa sobre 8192 colores vuelven a $6{\cdot}10^{-6}$ del pico. Las fórmulas de visualización, derivadas de nuevo en un segundo kernel, coinciden a $3.5{\cdot}10^{-6}$. OpenColorIO se usa **como compilador**: `colour::ColourCompiler` pide a OCIO el HLSL, lo carga como un módulo Slang `athenea_cs_<hash>` y llena sus LUTs en el dispositivo. El host calcula el texto del shader y los valores de las LUTs, pero nada por píxel.

2.13 USD e Hydra

El plugin `hdAthenea` implementa `HdRenderDelegate` con rprims para mallas, basis curves, puntos, `ParticleField` (splats gaussianos) y volúmenes; sprims para cámaras, materiales, los seis tipos de luz UsdLux, ext computations y sistemas de coordenadas; y bprims para búferes de render y render settings. Las superficies implícitas (esfera, cubo, cono, cilindro, cápsula, plano) llegan al delegado como mallas, convertidas por los scene indices de hdsi. **Hilos:** `Sync`, que Hydra puede ejecutar en cualquier hilo, solo entrega registros de CPU a `Engine` bajo un cerrojo. El hilo del render pass sube los datos, abre los ficheros `.athc` y compila los materiales, y después renderiza, así que el dispositivo tiene un solo llamador. Una nube con esqueleto cuyos arrays no llevan muestras de tiempo se conserva entre pasos (`_held`) y no se resube; esto llevó el fotograma en reproducción del gorrión de 102.4 a 71.7 ms antes del trabajo de ordenación y mezcla de §2.5.3. **Salidas:** color, profundidad, primId, instanceId, elementId, Neye, normal, `primvars:*`, Cryptomatte y grupos de luces. Se convierten en el dispositivo (`usd/aov_convert.slang`) solo cuando un host mapea un búfer, y se guardan con la fila inferior primero, como las guarda Storm. `StageRenderer` es el punto de entrada programático y dibuja hasta que un fotograma trazado por caminos alcanza su total.

El motor lee esquemas sin código propios (`AtheneaSplatLightingAPI`, `AtheneaStreamedAssetAPI` y otros). Los ajustes de render llegan como ajustes `athenea:*`: técnica, ruta de visibilidad, muestras de camino, rebotes, totales, MIS, eliminación de ruido, antialiasing y lente.

2.14 Hosts y front ends

- **`athenea view`** mantiene los fotogramas en el dispositivo: `StageRenderer::draw`, después `DisplayTransform` escribe la textura de superficie y Dear ImGui dibuja en el mismo dispositivo. Medido con materiales a 1600×900, Kitchen_set se dibuja en 9.27 ms con rays. Los paneles se describen una vez en `modules/ui`, así que la app de iOS dibuja los mismos.
- **Host AOFX.** El SDK y el host de AOFX son los de aopenfx (`third_party/aopenfx`), y el motor solo declara lo que aporta (`Host.cpp`). Cambian solo de forma aditiva bajo la regla de la ABI, así que los bundles del compositor cargan sin cambios. `aofx_sdk_manifest` falla ante cualquier cambio de cabecera. Los pasos de procesamiento de datos como mesh2splat (Sección 3) y `athenea compare` son bundles AOFX.
- **Servidor MCP** (`athenea-mcp`): JSON-RPC 2.0 sobre stdio, con el dispositivo y el stage calientes. Las herramientas son `open_stage`, `render` (que devuelve un bloque de imagen PNG), `pick`, `bounds`, `convert`, `timings` y `settings`. Una batería de pruebas de protocolo comprueba el saludo inicial, las notificaciones y los códigos de error.

2.15 Verificación sin oráculo en CPU

Toda comparación de imágenes es un kernel. `ReferenceRenderer` es la verdad de referencia para splats: cada píxel considera cada splat, ordena exactamente lo que le llega (profundidad, luego índice, la regla de desempate del rasterizador) y mezcla de delante hacia atrás, sin teselas, rangos ni pares. Una comparación solo es válida cuando ningún píxel desborda sus 1024 contribuyentes. `renderPeaks` es la referencia del trazador de rayos, que ordena exactamente por pico. Las métricas que se leen de vuelta en el host son `compareImages` (p99, máximo y número de píxeles por encima de 2, en códigos sRGB de 8 bits), `compareHdr` (relMSE $= (a-b)^2/(\bar{b}^2 + 10^{-2})$ y el p99 relativo) y `countDifferent` para búferes de ids. El trazador de rayos se mantiene frente a su referencia a p99 ≤ 1 con como mucho un 0.05 % de píxeles por encima de 2.

Frente al rasterizador, la tolerancia solo se cumple donde las dos definiciones coinciden: p99 2 en escenas ortográficas dispersas, donde EWA es exacto, y 4 en escenas en perspectiva con partículas de pocos píxeles. Las escenas densas difieren en decenas de códigos, lo que refleja el orden y no un error.

Tres dispositivos (commit 2e6e183; cada máquina calcula ella misma la misma métrica de GPU, y ningún píxel viaja entre máquinas):

	Metal (M5 Pro)	**Vulkan (L4)**	**CUDA (L4, OptiX 9, sin rayos en línea)**
Casos	188	188	188
Superados	188	180	96
Fallidos	0	0	1
Omitidos (capacidad nombrada)	0	8	91

2.16 Limitaciones

- Los splats aún no están dentro de los rayos secundarios del trazador de caminos. Un stage solo de splats lo traza entero `GaussianRayTracer`. Con mallas, los splats se componen por profundidad sobre las superficies trazadas por caminos; proyectan sombras sobre las mallas, pero un rayo de rebote no los ve.
- El desenfoque de movimiento y de lente del rasterizador son gaussianas con momentos igualados, no mesetas ni discos de bokeh: hasta un 38 % demasiado picudas en estelas largas.
- Las nubes cuyas gaussianas miden unas centésimas de píxel desaparecen del raster ($5{\cdot}10^{-6}$ del fotograma frente a $3{\cdot}10^{-4}$ de la malla y del trazador); los niveles de detalle son el remedio (Sección 6).
- La visibilidad raster de mallas paga unos 8 ms de tiempo de host por cada 1800 draws.
- La compilación de materiales es síncrona.
- La TLAS se reconstruye en cada fotograma.
- No se aplica la distorsión de lente más allá de k_1, k_2.

3. De mallas USD a gaussianas (mesh2splat)

La conversión transforma un stage USD de mallas poligonales y sus materiales en una nube de gaussianas 3D anisótropas que el resto del sistema dibuja, hornea, diezma y transmite. Se invoca como `athenea mesh2splat STAGE` y es el núcleo del producto: todo lo que el teléfono acaba reproduciendo fue antes un triángulo de una malla. Esta sección describe cómo se lee el stage, qué tamaño tiene la celda de una gaussiana, cómo se coloca, se da forma y se colorea cada gaussiana, cómo trata la conversión los lugares donde una superficie termina o se pliega, cómo captura un material físico por capas, cómo trata el vidrio y qué se midió sobre el asset del Chevrolet Corvette C7 que guía la mayor parte del proyecto. El horneado por trazado de caminos que sigue a la emisión (la *transferencia*) es objeto de la sección 4 y aquí solo se toca donde condiciona la conversión.

Las cifras de esta sección proceden de `docs/decisions.md` y `docs/pieces.md` del repositorio, de los registros de conversión en `~/luc/athenea-renders/` y del código fuente (`plugins/mesh2splat/mesh2splat.slang`, `apps/athenea/src/CmdMesh2Splat.cpp`, `shaders/athenea/usd/mesh2splat_*.slang`). Todas las medidas se tomaron en un Apple M5 Pro (Metal) salvo que se indique otra cosa.

3.1 Origen, y dónde se ejecuta la conversión

El algoritmo de emisión es el de *mesh2splat* de Electronic Arts (BSD-3): cada triángulo se proyecta sobre el plano de ejes hacia el que más apunta su normal, la proyección se rasteriza a una resolución fija y cada téxel cubierto se convierte en una gaussiana tendida en el plano del triángulo, tan ancha como un téxel medido en el mundo. El original es un par de shaders de geometría y de fragmentos en OpenGL; aquí es un único kernel de cómputo en el que cada hilo posee un triángulo y recorre las celdas de su caja proyectada.

El kernel no es un módulo del motor sino un efecto AOFX, el bundle `plugins/mesh2splat`, cargado mediante el mismo registro de host que comparten todos los programas (la ABI de AOFX, sección 2.14). La conversión es un kernel sin estado sobre triángulos, que es exactamente la forma de un efecto AOFX, y hacerla uno mantiene un conversor fuera de los módulos del renderizador y permite que el mismo bundle se ejecute en otros hosts AOFX. El coste es una frontera: a un efecto se le entregan imágenes, de modo que una malla viaja como una *imagen de números*, seis `float4` por triángulo — $(\boldsymbol{p}, u)$ y $(\boldsymbol{n}, v)$ para cada vértice, en espacio de mundo (`mesh_pack.slang`) — y cada textura que nombra un material viaja como filas de `float4` lineales muestreadas de la tabla de texturas del motor (`texture_rows.slang`). Ninguna de las dos es una copia: las imágenes viven en el almacenamiento de imágenes de gpe, que adopta el dispositivo Metal del motor, y `Context::renderView` da una vista de los mismos bytes, así que el kernel de empaquetado escribe donde el efecto lee. Los registros que escribe el efecto se reúnen en los búferes de dispositivo de la nube (`mesh2splat_gather`), los límites de la nube se pliegan en el dispositivo (`mesh2splat_span`) y el horneado los lee en su sitio. Solo los contadores y los arrays USD finales cruzan al procesador.

El comando es, por tanto, una cadena de pasadas en el dispositivo orquestada por el host:

1. leer el stage (`usd::MeshStage`) y dividir las mallas en *piezas*;
2. empaquetar cada pieza en una imagen de malla, plegar las cajas y elegir la celda de cada pieza (`mesh2splat_cells`);
3. marcar las aristas duras y las cáscaras cerradas por malla (`mesh2splat_edges`) y clasificar el vidrio (`mesh2splat_sheet`);
4. ejecutar el efecto Mesh2Splat por pieza — contar, explorar, emitir — hacia registros;

5. opcionalmente, hornear (radiancia, o una transferencia) en cada gaussiana con el trazador de caminos;
6. escribir un stage `ParticleField3DGaussianSplat` (o un `.athc`) de forma atómica.

3.2 Lectura del stage

`usd::MeshStage` recorre el stage directamente con `UsdShade` en lugar de a través de Hydra: la conversión no quiere ningún índice de render, y el lado de los materiales se apoya en los propios bindings del stage.

Selección. `--prim P` restringe la conversión a las mallas en `P` o bajo él; `--hide P` (repetible) escribe invisibilidad de sesión en `P` y en todo lo que cuelga de él antes de la lectura. Una malla que resulta invisible en el código de tiempo de la lectura se omite, de modo que la conversión convierte lo que dibuja el renderizador. `--hide` afecta solo a la conversión: el horneado traza el stage entero, así que una parte convertida con todas las demás ocultas sigue recibiendo sus sombras e interreflexiones. Esto es lo que permite convertir el Corvette por grupos (sección 3.11).

Instanciado. La malla de un prototipo de `PointInstancer` se emite una vez por instancia del instanciador más interno que la contiene, en (malla relativa a la raíz de su prototipo) × (transformación de la instancia, raíz del prototipo incluida) × (instanciador a mundo); los búferes de la malla construida se comparten entre las copias. Antes de esto, los dieciséis peones del ajedrez se convertían en dos. Los instanciadores anidados solo se expanden en el nivel más interno.

Tiempo y pose. `--time T` fija el código de tiempo en que se leen posiciones, transformaciones y primvars (sus índices incluidos) y en que traza el horneado. Los `points` de una malla con piel no se animan en USD, así que el stage se posa primero con `UsdSkelBakeSkinning` en la capa de sesión sobre $[T, T]$, lo que cuesta una pose y deja el fichero intacto; un stage sin `SkelRoot` queda igual y se convierte idéntico byte a byte. Con `--skinned` las gaussianas se construyen en la pose de enlace y cada una conserva cuatro articulaciones y los gradientes de sus pesos; ese camino se describe en la sección 5.

Piezas. Una malla cuyos GeomSubsets `materialBind` enlazan materiales distintos se convierte como varias *piezas*: los triángulos de cada subconjunto y los que ningún subconjunto reclama. Los triángulos de una pieza se listan en el dispositivo en el orden de la malla (marca, suma prefija, dispersión) y se empaquetan como si fueran una malla, de modo que el efecto no cambia. Una malla de prueba con dos caras enlazadas en rojo y azul por subconjuntos y la malla enlazada en verde se convierte en 272 gaussianas rojas y 272 azules y ninguna verde; antes del cambio era toda verde.

Los materiales en sus propios términos. La superficie enlazada — MaterialX `standard_surface`, OpenPBR `open_pbr_surface`, `gltf_pbr` o `UsdPreviewSurface` — se lee con sus propios nombres de entrada y sus propios valores por defecto (standard_surface gris 0.8 a rugosidad 0.2, OpenPBR rugosidad 0.3, glTF un metal totalmente rugoso, UsdPreviewSurface 0.18 a 0.5), siguiendo las conexiones a través de grafos de nodos y entradas de interfaz hasta el nodo de imagen que produce un valor. Un valor calculado por un grafo (una mezcla, un ruido) no tiene respuesta sin sombrear un punto, así que se mantiene el valor por defecto y se imprime un aviso; un horneado TX lee después esos campos desde el material compilado en el dispositivo. El espacio de color de una textura se toma del metadato `colorSpace` de la entrada de fichero (o de `sourceColorSpace`), y los mapas se limitan a `--texture-size` (1024 por defecto): un mapa es una imagen `float4` en el dispositivo, así que un mapa de 4k ocuparía 268 MB.

3.3 El tamaño de una celda

Todo lo que viene después escala con la celda h, la longitud en el mundo que representa una gaussiana. La conversión ofrece cuatro maneras de elegirla, en orden creciente de preferencia.

Por modelo (`--density per-model`, el valor por defecto cuando no se da una cámara). $h = R/N$, donde N es `--resolution` (512 por defecto) y R el mayor de los dos rangos del plano de proyección de la caja del modelo — los `orthogonalUvs` del original. De ello se seguían dos defectos: un suelo de 16 unidades diluía el coche apoyado en él, y un panel orientado hacia delante recorría una celda 2.5 veces más fina que el capó, porque su plano de proyección es el lado corto del coche.

Por malla (`--density per-mesh`). $h = L_m/N$ con L_m el lado más largo de la caja propia de la malla, el mismo para todos sus triángulos (`cellByLongest`), acotado a $[h_{\min}, h_{\max}]$ dados por `--cell-min`/`--cell-max`; si se dejan a 0 se derivan de la celda del modelo h_M: $h_{\max} = h_M$, $h_{\min} = h_M/8$, una razón de densidad de 64 entre la parte más fina y la más gruesa. Medido en `MustangHero.usda` (el coche sobre un suelo de 16 unidades) a `--resolution 512`:

modo	total	parachoques `Object_199`	capó `Kapoot`	suelo
por modelo	375 199	762 (celda 0.0313)	2 326	262 144
por malla, por defecto	934 338	3 082 (celda 0.0039)	20 279	262 144
por malla, `--cell-min` = `--cell-max` = 0.03125	375 157	764	2 339	262 144

La fila de control reproduce el modo por modelo al 0.01 %; la diferencia restante es solo el efecto de la orientación.

Desde una cámara (`--cell-from-camera CAM --camera-pixels P`, a partir de Mesh2GS, arXiv 2606.21898). La celda de cada pieza es lo que cubre un píxel de la cámara `CAM`, de `P` píxeles de ancho, donde la caja de la malla de la pieza le queda más cerca:

$$h = \frac{z\,(A/P)}{f}, \qquad z = \max\!\left(\,\left\|\operatorname{clamp}(\boldsymbol{e}, \boldsymbol{b}_{\min}, \boldsymbol{b}_{\max}) - \boldsymbol{e}\right\|,\; z_{\text{near}}\right),$$

con A la apertura horizontal de la cámara, f su distancia focal, $\boldsymbol{e}$ el ojo, y el resultado acotado solo por `--cell-min`/`--cell-max` del usuario. Mesh2GS toma $z_{\min}$ sobre la caja dentro del frustum y sobre las muestras temporales de la cámara; la distancia euclídea en `--time` es la misma para una caja a la vista y más fina, nunca más gruesa, para una al lado del frustum. Con dos tarjetas iguales a tres y siete unidades de una cámara de 64 píxeles, la tarjeta cercana recibió 930 gaussianas y la lejana 169: una razón de 5.5 frente a los $(7/3)^2 = 5.4$ esperados. Este es el modo usado en todos los horneados de producción del Corvette (`--cell-from-camera /World/Camera --camera-pixels` 640, 1280 o hasta 5120).

El presupuesto. `--max-splats` (2 M por defecto) se reparte sobre todo el stage. Cada pieza se cuenta primero (una ejecución del efecto con sitio para una gaussiana, cuyo recuento y exploración cubren todo lo que escribiría una ejecución); donde el total W supera el presupuesto B, la pieza i que quiere w_i recibe $s_i = \max(1, Bw_i/W)$ y recorre una celda $\sqrt{w_i/s_i}$ veces más gruesa. Una celda k veces más gruesa cubre la superficie con k^2 gaussianas menos, así que las cuotas proporcionales y el engrosamiento uniforme son lo mismo. Con dos tarjetas que quieren unas 1 300 bajo un presupuesto de 800, las cuotas salieron 356 y 361; gastado en el orden de las mallas, la segunda tarjeta tenía unas 145.

Límites por triángulo y por ejecución. Un triángulo recorre como mucho `--max-cells` celdas de su caja proyectada (2^{18} por defecto); las que sobran se cuentan y se avisan. A 4736 px, `Infinite_Black` del Corvette perdió así 359 628 celdas (agujeros visibles) y se volvió a hornear con `--max-cells 1048576`. La imagen de salida de una ejecución está limitada a 2 M gaussianas (`kRunCeiling`, 192 MB); una pieza que quiere más se convierte por tramos — el efecto informa del primer triángulo que el techo cortó y el host vuelve a ejecutar desde ahí con `firstTriangle` —, lo que da el mismo array que habría escrito una sola ejecución.

3.4 Una gaussiana por celda

La proyección. Para un triángulo $(\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c})$ de normal geométrica $\widehat{\boldsymbol{n}}_g$, los dos ejes (i, j) hacia los que menos apunta $\widehat{\boldsymbol{n}}_g$ definen la proyección; un punto se lleva a $\boldsymbol{o}(\boldsymbol{x}) = (x_i - b_{\min,i},\, x_j - b_{\min,j})/R$, con R el rango en uso (el del modelo, el lado más largo de la malla o la caja derivada de la cámara). Una celda es $1/N$ de ese cuadrado unidad, es decir, $h = R/N$ en el mundo, acotada como en la sección 3.3. Se emite una gaussiana en cada celda cuyo centro cae dentro del triángulo proyectado, localizada por sus coordenadas baricéntricas $\boldsymbol{\lambda}$, en el punto del mundo $\boldsymbol{\mu} = \lambda_0 \boldsymbol{a} + \lambda_1 \boldsymbol{b} + \lambda_2 \boldsymbol{c}$.

Los tamaños. Con $E = [\boldsymbol{b} - \boldsymbol{a},\ \boldsymbol{c} - \boldsymbol{a}]$ y $O = [\boldsymbol{o}_b - \boldsymbol{o}_a,\ \boldsymbol{o}_c - \boldsymbol{o}_a]$, el jacobiano de la inversa de la proyección es $J = E\,O^{-1} = [\boldsymbol{j}_u, \boldsymbol{j}_v]$ (protegido con $|\det O| > 10^{-24}$). Las dos desviaciones típicas en el plano y la tercera son

$$s_u = \min\!\left(\left\|\boldsymbol{j}_u\right\| \sigma \tfrac{1}{N},\ \ell_{\max}\right), \qquad s_v = \min\!\left(\left\|\boldsymbol{j}_v\right\| \sigma \tfrac{1}{N},\ \ell_{\max}\right), \qquad s_n = \varphi \min(s_u, s_v),$$

donde σ es `--sigma`, $\ell_{\max}$ la arista más larga del triángulo y φ `--flatness`. Tres detalles se leyeron del código original en lugar de suponerse: el jacobiano es sobre las coordenadas triplanares, no sobre las coordenadas de textura; el factor $1/N$ lo aplica el exportador del original, no ningún shader (sin él cada gaussiana tiene el tamaño del modelo); y el marco no son las columnas del jacobiano sino la arista más larga del triángulo $\widehat{\boldsymbol{t}}$, la normal de sombreado interpolada $\widehat{\boldsymbol{n}} = \text{normalize}(\sum_k \lambda_k \boldsymbol{n}_k)$ y su producto vectorial. La covarianza es, por tanto,

$$\Sigma = R_{\text{frame}}\, \operatorname{diag}(s_u^2, s_v^2, s_n^2)\, R_{\text{frame}}^{\top}, \qquad R_{\text{frame}} = [\widehat{\boldsymbol{t}}_\perp,\ \widehat{\boldsymbol{n}} \times \widehat{\boldsymbol{t}}_\perp,\ \widehat{\boldsymbol{n}}],$$

con $\widehat{\boldsymbol{t}}_\perp$ la arista hecha ortogonal a $\widehat{\boldsymbol{n}}$. La cota $\ell_{\max}$ se añadió después de que 1 859 astillas de un BMW, vistas casi de canto por su propia proyección, produjeran columnas del jacobiano veinte órdenes de magnitud demasiado largas y dibujaran púas blancas de metros de largo.

Anchura. El $\sigma = 0.65$ del original deja una superficie convertida transparente en torno a un 30 % para un trazador de rayos. Medido en un cuadrado unidad iluminado de frente (píxel de la malla 0.5032):

σ	cobertura	píxel
0.65	0.702	0.351
0.85	0.848	0.425
1.0	0.911	0.456
1.2	0.957	0.479
1.5	0.986	0.493

El valor por defecto es, por tanto, $\sigma = 1.0$; `--sigma 0.65` recupera el original.

Planitud y surfels. El original escribía 10^{-7} unidades del modelo como tercer tamaño, lo que hacía depender la delgadez de una gaussiana de la escala del modelo: un peón de 66 mm se trazaba bien mientras que un zorro de 100 unidades se convertía en un fantasma bajo el trazador de rayos. El tercer tamaño es ahora la fracción φ (0.1 por defecto) del menor de los tamaños en el plano. `--flatness 0` escribe surfels —gaussianas con una escala exactamente cero, el disco del splatting gaussiano 2D —, que el rasterizador dibuja mediante una intersección rayo–plano por píxel y que el trazador de caminos acota a 10^{-4} del eje más ancho.

La opacidad como cobertura. Un punto de una superficie convertida queda bajo varias gaussianas a la vez, así que una opacidad escrita directamente en cada una cubre de más: una máscara de 0.5 cubría el 96 % de lo que había detrás. Toda opacidad que lee la conversión (`--opacity`, la constante del material, el

valor de un mapa, lo que conserva un vidrio) se trata como la *cobertura* F de la superficie y se convierte en el α de cada gaussiana desarrollando la transmitancia de la pila hasta segundo orden: sobre la retícula de celdas las huellas suman $2\pi\sigma^2$ y sus cuadrados $\pi\sigma^2$, de modo que

$$\frac{\pi\sigma^2}{2}\,\alpha^2 + 2\pi\sigma^2\,\alpha = -\ln(1-F) \;\Rightarrow\; \alpha = \frac{\sqrt{4S^2 + 2SL}\; - 2S}{S} + \frac{1}{255}, \quad S = \pi\sigma^2,\; L = -\ln(1-F),$$

donde el 1/255 devuelve el área que pierde una gaussiana al cortarse en 1/255; $F \geq 0.999$ se lleva a 1 y 0 a 0. Medido en planos de cobertura constante 0.25 / 0.5 / 0.75, el rasterizador da 0.251 / 0.501 / 0.752 y el trazador de rayos lo mismo a tamaño completo, frente a 0.253 / 0.510 / 0.770 con solo el término de primer orden. Un mapa de recorte por debajo de `--opacity-cut` (0.5) no escribe ninguna gaussiana, así que el presupuesto va donde está la superficie.

Un triángulo que no atrapa ninguna celda. Un triángulo más pequeño que una celda puede no contener ningún centro de celda; rasterizado, desaparece. Convertido, eso eliminaba la rejilla y las gomas de las ventanas del Buick (80 137 triángulos se convertían en 34 011 gaussianas). Ahora tal triángulo recibe una gaussiana en su baricentro, dimensionada por sus propios segundos momentos (`m2sOwnSize`): la covarianza del triángulo uniforme $\Sigma_\triangle = \frac{1}{12}\sum_k \boldsymbol{d}_k \boldsymbol{d}_k^\top$ (vértices respecto al baricentro, en el plano tangente), ensanchada como se ensancha una celda, $\Sigma = 12\sigma^2\Sigma_\triangle$, con el eje ancho acotado a $[0.05\,s_{\max}, s_{\max}]$ del tamaño de la celda y el estrecho a al menos 0.02 del ancho. Antes de esto, el labio enrollado de un panel — una tira de triángulos menores que la celda — dibujaba una banda de cuentas del ancho de una celda, de tres píxeles de grosor, a lo largo de cada arista del Corvette donde el trazador de caminos dibuja una línea finísima. En los horneados del Corvette, el vidrio y los neumáticos salen a aproximadamente una gaussiana por triángulo tanto a 640 como a 1280 px.

Orden determinista. El original añadía con un contador atómico, así que dos conversiones del peón producían las mismas 729 073 gaussianas en órdenes distintos. La emisión son ahora tres pasadas — contar las celdas cubiertas por cada triángulo, una exploración exclusiva en un grupo de trabajo de 256 hilos, escribir las gaussianas de cada triángulo en su desplazamiento — que comparten una función (`m2sTriangleOf`), de modo que contar y escribir no pueden discrepar. Dos conversiones son ahora idénticas byte a byte, un presupuesto conserva las primeras gaussianas en el orden de la malla y la gaussiana i es la misma gaussiana de una pose a la siguiente. Un registro que se decodifica en nada se escribe como una ranura vacía en lugar de descartarse, de modo que los índices de los arrays siguen alineados.

3.5 Donde una superficie termina o se pliega

Una gaussiana mide una celda de ancho, y en el Corvette una celda es un píxel de la cámara de conversión. Una parte más estrecha que eso — la pestaña que un capó dobla hacia abajo dentro de su junta — seguía recibiendo gaussianas del ancho de una celda en su propio plano; asomaban fuera de la junta sobre la pintura de al lado, llevando la oscuridad horneada de la junta, y cada borde de panel se convertía en una línea negra gruesa y ondulada donde el trazador de caminos dibuja una fina.

Aristas duras (`mesh2splat_edges.slang`). Por malla, sobre todas sus piezas, cada arista y cada vértice se indexan por la posición de sus puntos y se ordenan por radix en el dispositivo; cada tramo de claves de arista iguales es una arista. Una arista es *dura* cuando está abierta (la usa un solo triángulo: la parte termina o empieza otra malla), cuando los dos triángulos que la comparten se pliegan más de 40° (`kCreaseDegrees`) o cuando la comparten más de dos triángulos (no variedad). Como las claves son posiciones sobre toda la malla, la frontera entre dos materiales sobre una misma superficie suave no es una arista. Cada arista dura se convierte en una recta con un interior, $(\widehat{\boldsymbol{m}}, c)$ con interior $\widehat{\boldsymbol{m}} \cdot \boldsymbol{x} \geq c$ — el plano que pasa por la arista, ortogonal a su triángulo, orientado hacia el tercer vértice —, que su propio triángulo conoce como

un bit y que se comunica a cada triángulo que toca cualquiera de sus extremos (hasta cinco rectas cada uno), puesto que una gaussiana cerca de un vértice puede quedar junto a la arista dura del triángulo vecino.

Contener el alcance (`m2sHoldEdges`, `--edge-reach` ρ, 1.5 por defecto). Para cada recta contenida en el plano de la gaussiana, sea $\boldsymbol{m}$ su normal en la base tangente, C la covarianza en el plano, $r^2 = \boldsymbol{m}^\top C\,\boldsymbol{m}$ la varianza a través de la recta y d la distancia con signo del centro a ella. El tamaño permitido a través de la recta es $a = \max(d/\rho,\ 0.05\,r)$, y donde $r > a$

$$C \leftarrow C - \kappa\,(C\boldsymbol{m})(C\boldsymbol{m})^\top, \qquad \kappa = \frac{1 - a^2/r^2}{r^2},$$

lo que hace $\boldsymbol{m}^\top C\,\boldsymbol{m} = a^2$ exactamente y conserva la varianza a lo largo de la dirección conjugada de la recta, de modo que una tira de gaussianas sigue cubriendo la tira. Se ignoran las rectas que no están en el plano de la gaussiana (la pestaña vista desde la pintura de la que se dobla) y aquellas tras las que el centro queda a más de un cuarto de su tamaño; las rectas se aplican en un orden canónico, de modo que el resultado no depende del orden en que se encontraron. La nueva covarianza se rediagonaliza en una tangente y dos tamaños. El número de gaussianas no cambia; solo su forma.

Cáscaras cerradas. La misma pasada cuenta las aristas no compartidas exactamente por dos triángulos, un contador leído por malla. Una malla se marca como cerrada cuando

$$n_{\text{open}} < 2\sqrt{1.5\,T}\ ,$$

con T sus triángulos ($1.5T$ aproxima sus aristas soldadas), y sus gaussianas llevan `primvars:athenea:splat:closed = 1`, lo que permite a un reproductor o a un corte descartar nodos que miran hacia atrás. Primero se probó una proporción fija de aristas (2 %) y falló: el borde de una lámina abierta crece como la raíz cuadrada de su área, así que un tubo lo bastante fino pasaba por cerrado — los cilindros de las llantas (1 %) se descartaban y se veía luz entre los radios en el teléfono. Frente a la raíz cuadrada, las puertas (ninguna arista abierta), los logotipos (1.4) y las pinzas (1.5) siguen cerrados y las llantas (4.0), los neumáticos (3.1), el capó (3.1) y el parachoques (4.4) no: 102 de las 192 mallas del Corvette. La puerta de un coche son dos pieles; dibujada sin descarte, la piel interior se ordenaba delante de la exterior a parches, una red de grietas oscuras al tamaño del teléfono.

Medido. La base TX del Corvette a 640 px de `/World/Camera`, horneada de nuevo con las aristas contenidas (1.5 por defecto) y, como control, con `--edge-reach 0`, ambas de 5 380 804 gaussianas; fotograma diurno a 1920 × 1080 frente al path trace de la malla:

horneado	relMSE (HDR)	PSNR (pantalla)	SSIM
base anterior (build antigua)	0.0900	21.83 dB	0.8555
control, aristas sin contener	0.0902	21.86 dB	0.8554
aristas contenidas, $\rho = 1.5$	0.0887	21.93 dB	0.8567

La ganancia global es pequeña porque las aristas son una fracción pequeña del fotograma; el efecto visible está en las juntas de los paneles.

3.6 Captura del material

Cada registro de gaussiana lleva la posición, la opacidad derivada de la cobertura, tres tamaños, una rotación y un color, seguidos — según lo que necesite la nube — de la normal de sombreado y el metálico, la rugosidad, la transmisión y las coordenadas de textura; cuatro articulaciones y sus gradientes; tres entradas

de relieve para el desplazamiento; una entrada de emisión; cuatro entradas de capas; y el operador de forma.

Base. El color es el color base del material por su peso base, por el mapa de albedo donde lo haya (decodificado a lineal). El metálico y la rugosidad son las constantes del material, multiplicadas por los canales del mapa empaquetado donde lo haya (azul metálico, verde rugosidad), como glTF multiplica sus factores; un canal ausente es un factor uno. El original tomaba por defecto $(0.1, 0.5)$ solo del mapa, así que todo material sin mapa se convertía en el mismo plástico; pasando los valores propios del material, un cuadrado convertido bajo una luz de frente da 0.5009 frente a 0.5032 de la malla.

Normal de sombreado. Un mapa de normales se lee en el marco que dan sus UV — $\partial \boldsymbol{p}/\partial u$ a partir de las posiciones y las UV del triángulo, ortonormalizado frente a la normal, como hace el shader de la malla — y el resultado se guarda como normal de sombreado de la gaussiana (`primvars:athenea:splat:normal`) junto a su marco geométrico. Antes el mapa se leía respecto a la arista más larga, y cada faceta del mármol del peón giraba sus vetas según su propio ángulo. El disco en sí no se gira salvo que se pase `--normal-map-turns`, lo que abriría la superficie donde el relieve es pronunciado.

Capas (propuesta 026). Una gaussiana lleva el peso, el color y el índice especulares; el peso, la rugosidad y el índice de la laca; el color del brillo aterciopelado (sheen) por su peso y su rugosidad; y el interruptor de oscurecimiento de la laca — en las unidades de OpenPBR, tres palabras por splat en el dispositivo. Hasta tres mapas sobre estas entradas se muestrean por gaussiana (`m2sLayersAt`). Hacía falta porque la pintura del Corvette (`Car_Paint_Main`) es OpenPBR de metalicidad 1 sobre una base de aproximadamente 0.05 bajo una laca transparente de peso 1: todo lo que el ojo ve de ella es la laca, y sin la laca un coche convertido salía casi negro.

Metales de Schlick. Los metales de OpenPBR y glTF son el `generalized_schlick_bsdf` de MaterialX (el color de frente, el color especular a 82°), no un conductor del índice artístico; para un metal oscuro ambos difieren en un factor dos a ángulos rasantes (a 60°: 0.172 el conductor, 0.077 el Schlick desde $F_0 = 0.047$). El tipo de material se escribe por gaussiana (`primvars:athenea:splat:schlickMetal`) y el renderizador lo refleja con el ajuste DFG de sus dos extremos. Una variante anterior de la misma confusión hacía que el horneado descartara todo metal OpenPBR como pulido dieléctrico: el parachoques delantero de un Mustang guardaba en sus 27 016 gaussianas la constante que se decodifica en negro; conservando como cuerpo los lóbulos de Schlick con $F_0 > 0.2$, el color medio del parachoques pasó a ser (0.45, 0.51, 0.61).

Emisión y transmisión. La emisión es el color del material por su mapa, lineal y sin acotar, en una entrada propia. La transmisión T y el color de transmisión $\boldsymbol{c}_T$ son un canal que el original no tenía (el vidrio salía blanco opaco). El color se lleva hacia el color de transmisión,

$$\boldsymbol{c} \leftarrow \operatorname{lerp}\big(\boldsymbol{c},\ \operatorname{lerp}(\mathbf{1}, \boldsymbol{c}, w_T)\,\boldsymbol{c}_T,\ T\big),$$

con $w_T = 1$ para glTF (transmisión teñida por el color base) y 0 para OpenPBR y standard_surface. La opacidad es la correspondencia de cobertura de la sección 3.4 aplicada a $o \cdot o_{\text{mat}} \cdot \text{máscara} \cdot \operatorname{lerp}(1, g, T)$, siendo g lo que aún cubre una superficie totalmente transmisora (sección 3.8).

La marca de rugosidad del material. La rugosidad guardada puede ensancharse por el filtro especular (sección 3.7). La rugosidad propia del material antes del ensanchamiento se escribe aparte, `primvars:athenea:splat:materialRoughness` (-1 bajo un mapa de rugosidad, que se lee como nítida). El reproductor la usa para decidir si el lóbulo de pulido responde su parte ocluida desde el campo reflejado horneado (rugosidad del material de al menos 0.3: pintura, 0.34) o la deja oscura (cromo 0, metal rugoso 0.22); en el árbol diezmado se empaqueta en la palabra `pbr` (bit 26 presente, bits 27–31 el valor en cinco bits).

3.7 El filtro especular (ensanchamiento de Toksvig)

Una gaussiana tiene una sola normal. Donde una tira de cromo se curva bajo ella, el píxel trazado promedia un reflejo que se desplaza por la huella y muestra una línea fina; la gaussiana muestra la luz que encuentra su única normal, a pleno brillo, sobre toda su huella. En el Corvette, el embellecedor de la rejilla — de cuatro píxeles de alto — volvía como una fila de cuentas blancas, y el cromo daba 1.6 veces el path trace mientras que una bola de cromo se validaba dentro del 1 %.

La conversión ensancha por ello la rugosidad según la dispersión de normales bajo la gaussiana (`m2sSpread`, `m2sWidened`). La tasa de giro es el mayor ángulo entre normales de vértice por unidad de distancia entre vértices, sobre las tres aristas del triángulo, y la dispersión es esa tasa por el mayor tamaño de la gaussiana:

$$s = k \max_{e} \frac{\angle(\boldsymbol{n}_{e_0}, \boldsymbol{n}_{e_1})}{\left\| \boldsymbol{p}_{e_1} - \boldsymbol{p}_{e_0} \right\|} \max(s_u, s_v),$$

con k = `--specular-filter` (1 con `--transfer`, 0 en otro caso). Con la rugosidad perceptual r y el $\alpha = r^2$ de GGX, la dispersión se suma a la varianza de las pendientes (Toksvig):

$$\alpha'^2 = \alpha^2 + 2s^2, \qquad r' = \operatorname{saturate}\Big((r^4 + 2s^2)^{1/4} \Big).$$

Un triángulo plano, o uno cuyos vértices comparten normal, queda igual. La rugosidad de la laca se ensancha del mismo modo salvo que la nube lleve el operador de forma.

El operador de forma. Para una transferencia, la conversión escribe también, en los ejes propios de la gaussiana $(\boldsymbol{u}, \boldsymbol{v})$, el operador de forma simétrico S con $\mathrm{d}\boldsymbol{n} = S\,\mathrm{d}\boldsymbol{x}$, resuelto a partir de las diferencias de normales de los vértices frente a sus diferencias de posición (`m2sShape`, guardado como (S_{uu}, S_{uv}, S_{vv})). El renderizador lo usa para girar el reflejo a través de la huella píxel a píxel (el «lóbulo nítido» por píxel). Ensanchar también la laca difuminaría dos veces, así que donde se escribe S la laca no se ensancha, y el reproductor lee los lóbulos nítidos con la rugosidad sin ensanchar $r_0 = \max(r'^4 - 2s^2, 0)^{1/4}$ (`unwidened`), con s guardada por gaussiana.

Medido. A 640 px la laca salía con rugosidad 0.17–0.24 y un 5–14 % de paso alto en los pliegues; a 1280 px la dispersión bajo cada gaussiana es menor y la rugosidad de la laca sale exactamente 0, así que el ensanchamiento importa menos cuanto más fino es el horneado. Aun así, quitarlo es peor (sección 3.11): con `--specular-filter 0` aparece una línea oscura discontinua en el pliegue del faldón y la tira de cromo centellea.

3.8 Vidrio

Una gaussiana no puede refractar, pero puede dejar pasar lo que tiene detrás mediante su opacidad, reflejar con su lóbulo especular y llevar un índice de refracción según el cual el renderizador desvía (`primvars:athenea:splat:ior`, uno por nube; con capas, cada gaussiana desvía según su propio índice especular). La tarea de la conversión es decidir, para cada pieza transmisora, si el vidrio es un *sólido* (un cuerpo: una bola, una lente) o una *pared fina* (una lámina o placa cuyas dos caras no desvían nada), porque ambos se dibujan de forma distinta.

Clasificación (`classifySheets`, `mesh2splat_sheet.slang`), para cada pieza transmisora cuyo material no se declara ya de pared fina. En el dispositivo, las aristas de la pieza se indexan por posición, se ordenan y se cuentan sus tramos; se suman el volumen con signo V (tetraedros al origen) y el área A. Entonces

- *abierta*: más de 1/200 de las aristas tienen un solo triángulo, o alguna arista tiene más de dos;

- *placa*: cerrada, y el grosor $2V/A$ está por debajo de cuatro celdas del modelo o de $0.02\sqrt{A}$ ($2V/A$ es la separación de una placa y dos tercios del radio de una bola).

Una pieza abierta o placa se lee como pared fina — salvo que esté *tintada* (luminancia del color de transmisión por debajo de 0.9), que de momento sigue siendo sólida, ya que una pared fina deja pasar el fondo en gris. El vidrio del Corvette resultó ser placas cerradas de 3–4 mm de grosor (propuesta de investigación 055), que la regla de aristas por sí sola llamaba sólidas; la regla del grosor las reconoce. Cada decisión se imprime, p. ej. para la lente del piloto trasero: «tinted, kept solid glass: 200 of its 17284 edges open, 0 shared by more than two; 1.199 thick (2V/A) against a cell of 0.007921». `--thin-glass MAT` y `--solid-glass MAT` la anulan por material.

Cobertura. Un vidrio sólido cubre `--glass-opacity` de lo que tiene detrás (0.6 por defecto; una lente TX responde su fondo solo a partir de las cúpulas, así que un vidrio que cubriera por completo ocultaría cualquier otra luz que lo atraviese). Una pared fina cubre exactamente lo que la lámina refleja de frente, ambas caras con interreflexión:

$$g_{\text{thin}} = \min\left(\frac{2R}{1+R},\ 0.998\right), \qquad R = \left(\frac{n-1}{n+1}\right)^2,$$

y el fotograma escala su reflejo de vuelta por $1/R$. Con el índice del parabrisas, 1.16, eso es una opacidad de 0.0055, y el rasterizador dibuja estas láminas con una opacidad de al menos 0.1 y el color reducido en el mismo factor, lo que mantiene intacto el producto $\alpha\,\boldsymbol{c}$ — el reflejo — (relMSE del parabrisas 2.26 → 0.89 bajo el cielo del taller).

3.9 Captadores de sombra y de luz

El suelo no se convierte como superficie; lo que se convierte es su interacción con el objeto, de modo que la reproducción es solo de gaussianas.

Captador de sombra (`--shadow-catcher --prim OBJETO`). El suelo se encuentra automáticamente (la mayor malla plana fuera del objeto cuya parte superior está en la base del objeto; `--catcher-ground` nombra una). Un parche de cuadriláteros — la huella del objeto ensanchada `--catcher-margin` alturas (1.5), dividida en una retícula para que ningún triángulo supere `--max-cells` — se añade a una copia de sesión del stage y se convierte solo, con una celda de una centésima de la altura del objeto, como una transferencia TX cuyos rayos encuentran el objeto y el suelo. Al dibujar, la opacidad de la gaussiana es $-\ln(\text{razón})/2\pi$, donde la razón es la luz que le llega con el objeto frente a sin él, ya que una retícula de gaussianas de $\sigma = 1$ celda suma 2π y una capa de opacidad a deja pasar $e^{-2\pi a}$. El del Corvette: un parche de 5.9 × 8.3 m, 11 × 11 cuadriláteros, 309 k gaussianas con una celda de 1.26 cm, horneado en 79 s. Frente al path trace:

	bajo el coche (GT 0.182)	**contacto del parachoques (GT 0.043)**	**suelo abierto (GT 0.217)**
mapa de sombras propio del suelo de malla	0.182	0.041	0.215
captador, profundidad 2.4, dos triángulos	0.144	0.002	0.206
captador, profundidad 2π, retícula	0.192	0.030	0.220

Captador de luz (`--light-catcher`). El mismo parche, alcanzando `--light-reach` longitudes del objeto (3) por delante y por detrás, con una celda de una treintava parte de la altura, convertido como un horneado de radiancia por grupo de luces con la cúpula en negro; dibujado de forma aditiva, lleva la mancha de la luz de cruce y el resplandor de los pilotos traseros.

3.10 Validación frente al path trace

`--validate DIR` mide una conversión material a material frente al stage trazado por caminos, para cualquier stage. Los grupos son los bindings de material del stage (una malla de dos materiales está en ambos); la conversión es la del propio comando, ejecutada una vez por grupo con todas las mallas ajenas a él ocultas, de modo que cada opción es lo que se mide. La referencia se traza una vez (`--validate-paths` 512, `--validate-bounces` 6) y se conserva; el stage se rasteriza una vez como mallas con su Cryptomatte, y la máscara de cada grupo es la cobertura de cada rango cuyo id es el hash de una de sus mallas. Dentro de la caja de la máscara, el efecto Measure da relMSE, p99 y las medias de la nube y del rasterizado de la malla a su lado, escritos en `validate.json`. `--validate-sky white` (o una imagen) dibuja todo bajo otro cielo con las demás luces del stage apagadas, de modo que el error de una transferencia es solo el de su oclusión y su reflexión. Es lo que reveló, entre otras cosas, que una bola de vidrio sólido se trazaba a la mitad del brillo del path trace (0.096 frente a 0.195) antes de que los rayos se desviaran hacia su interior, y que la bola de pintura bajo una cúpula blanca daba un 13 % por debajo a causa del Fresnel de la laca.

3.11 Resultados medidos sobre el Corvette

Todos los horneados siguientes son transferencias TX, `--cell-from-camera /World/Camera`, 64 caminos por gaussiana, 3 rebotes, sin caminos adaptativos extra, con el filtro especular y el alcance de aristas en sus valores por defecto (1 y 1.5). «Recorte del coche» es el PSNR de pantalla sobre el recorte de 900 × 420 del coche, y «fotograma» sobre el fotograma completo de 1920 × 1080, ambos frente al path trace de la malla, tras cortar el árbol de fusión a 0.25 px con descarte de caras traseras.

640 → 1280 px. Las hojas del horneado de 640 px medían 3–4 mm en el capó, y las rayas de la puerta y los pelos del faldón — ausentes en el path trace — estaban en la nube. El coche se volvió a hornear a 1280 px en 8 grupos de partes de primer nivel equilibrados por recuento, cada uno convertido con todas las demás partes ocultas (nombres de parte tomados del manifiesto de Cryptomatte). Los ocho horneados escribieron 891 387, 896 496, 809 073, 790 403, 1 631 765, 1 073 954, 1 302 878 y 1 307 473 gaussianas (8 703 429 en total) en 212–304 s de transferencia cada uno, unos 34 min en conjunto.

	640 px	**1280 px**
hojas	5.38 M	8.70 M
capó / puerta	50 k / 79 k	151 k / 266 k
vidrio, neumáticos	aprox. una por triángulo	aprox. una por triángulo
recorte del coche / fotograma (PSNR)	16.55 / 22.18 dB	17.88 / 23.90 dB

Análisis de la nube de 1280 px frente a la de 640 px: los reflejos del capó pasan de 3.2 a 2.0 px de su posición en el path trace, el contraste de los reflejos abiertos de la puerta de 0.35/0.53 a 0.56/0.74 del path trace, los agujeros de la rejilla en el corte grueso de 2.24 a 1.04, y los pelos del faldón de 987 a 459 px solo por la resolución.

La prueba A/B/C. Antes de fijar los ajustes de 1280 px se hornearon ocho partes (ambas puertas, ambos paneles del faldón, la aleta delantera, la rejilla cromada, el marco de la rejilla y el capó) con todo el coche proyectando sombra, 931 017 gaussianas en cada variante:

variante	**ajustes**	**resultado**
A	como el horneado de 640 px ($k = 1$, $\rho = 1.5$)	elegida: sin pelos, la puerta lisa

variante	ajustes	resultado
B	`--specular-filter 0`	línea oscura discontinua en el pliegue del faldón; la tira de cromo centellea
C	B + `--edge-reach 1.0`	alargamiento y pelos idénticos a B

La hipótesis a prueba era que las rayas de la puerta eran el ensanchamiento de Toksvig de la laca saturándose en los pliegues y que los pelos del faldón eran las gaussianas alargadas de la pasada de aristas. Ninguna se sostuvo a 1280 px: el ensanchamiento es necesario en los pliegues y un alcance de aristas más estricto no cambió nada, así que se mantuvo A.

El nivel de primer plano a 5120 px. De cerca en el teléfono, la cámara está mucho más próxima que `/World/Camera`, unos 0.5 mm por píxel del teléfono frente a hojas de 3–4 mm. Una prueba del logotipo y la rejilla desde una cámara cercana pasó de 108 527 a 404 594 gaussianas entre 1280 y 5120 px y mostró nítidos la bandera a cuadros, la pajarita y la malla de la rejilla. Las 66 partes a las que el horneado de 1280 px ya había dado como mucho 1.5 gaussianas por triángulo (insignias, rejillas, faros, pinzas, retrovisores, ruedas, limpiaparabrisas, molduras) se volvieron a hornear a 5120 px en siete grupos; los paneles, a 5120 px o tan finos como mantuviera cada parte bajo un tope de hojas (5 M, rebajado a 4.5 M después de que un árbol de 5.01 M no cupiera en la memoria del dispositivo), con `--max-cells 1048576`:

grupos	hojas	resolución (px)
fine0–6 (66 partes detalladas)	18 610 627	5120
puerta (piloto)	4 084 170	5120
Bottom	4 897 428	3392
Part_3	4 370 459	4480
Infinite_Black	4 481 738	4736
seis paneles grandes y el parabrisas	15 994 566	5120
pan0–4 (paneles pequeños)	17 088 152	5120
Front_Bumper_001	5 008 986	4480
24 árboles	**74 536 126**	

La transferencia de cada panel grande tardó 2–25 min (p. ej. el capó, 2 264 198 gaussianas en 120 s; Front_Bumper_001, 5 009 059 en 1 487 s) en una máquina compartida con otros trabajos de GPU, así que no son medidas de rendimiento. Se escribieron los 24 árboles de fusión (74 328 031 fusiones, 208 095 raíces, 148 864 157 nodos en 49 634 páginas, 87.4 GB), que comparten un codebook de oclusión de 8 182 entradas; la sección 6.9 da los detalles. (La cuenta horneada de Front_Bumper_001, 5 009 059, es la del registro de la conversión; el árbol tiene 5 008 986 hojas, la cifra que usa la tabla.)

3.12 Limitaciones

- Una gaussiana tiene una sola normal y la huella de una celda: un tubo de cromo de pocos píxeles de ancho sigue siendo un 38 % más brillante que el path trace en el parachoques delantero, y el labio inferior del parachoques sale irregular.
- Los faros no llevan emisión en el horneado de transferencia; los faros delanteros salen oscuros frente al trazado de caminos.

- Un vidrio fino tintado se convierte como sólido, y una nube conserva un único índice salvo que lleve capas.
- La correspondencia de cobertura supone el solapamiento $2\pi\sigma^2$ de la retícula de celdas; los triángulos menores que una celda y las celdas desplazadas y divididas se solapan de otra forma y no se corrigen.
- Los instanciadores de puntos anidados solo se expanden en el nivel más interno; el movimiento escrito como velocidades se lee en la muestra.
- La memoria del dispositivo limita una parte a unos 5 M de hojas para el árbol de fusión que sigue; las partes mayores se convierten a una resolución de cámara más gruesa.

4. Transporte de luz horneado en gaussianas (transfer) y reiluminación

Una nube convertida desde una malla puede llevar su luz de dos maneras. Un *horneado de radiancia* guarda, por gaussiana, la luz que llegó bajo la iluminación del propio stage; es exacto para esa iluminación y erróneo bajo cualquier otra. Un *transfer* guarda en cambio cómo el entorno de cada gaussiana le llevaría un entorno arbitrario, y un fotograma combina esa geometría con el cielo y las luces que tenga. Esta sección describe el transfer tal como athenea lo hornea y lo evalúa en su forma TX («tarea TX»: un transfer correcto bajo cualquier cielo y cualquier luz), el modelo de reflexión construido encima, el tratamiento por píxel de los lóbulos nítidos, y lo que se midió que aporta cada pieza frente a un path trace de la malla de origen. Todo lo descrito corre en la GPU: el horneado es el path tracer de la propia escena en un modo transfer, y la evaluación son los kernels de proyección y mezcla de splats (`shaders/athenea/splat/splat_relight.slang`, `splat_project.slang`, `splat_blend.slang`).

4.1 Notación

Una gaussiana tiene una posición $\boldsymbol{p}$, una normal unitaria $\boldsymbol{n}$ (su eje más corto, o una normal de sombreado guardada, girada hacia el ojo), un albedo ρ, una metalicidad m, una rugosidad perceptual r con parámetro GGX $\alpha = r^2$ y, opcionalmente, un clear coat (peso w_c, rugosidad r_c, índice η_c), un sheen y una transmisión. $\boldsymbol{\omega}_o$ es la dirección hacia el ojo y $\boldsymbol{m} = \text{reflect}(-\boldsymbol{\omega}_o, \boldsymbol{n})$ la dirección especular. Y_k son los armónicos esféricos reales, con índice $k = l^2 + l + m$; el grado 2 tiene 9 coeficientes y el grado 3 tiene 16. Un cielo tiene coeficientes de radiancia L_k (rgb), un sol de dirección $\boldsymbol{s}$ e irradiancia $E_\odot$ extraído de él, y un mapa prefiltrado $\tilde{L}(\boldsymbol{\omega}, r)$.

4.2 Qué guarda una gaussiana

El transfer es una tira de valores por gaussiana, f16 en el dispositivo. No hay token de modo: la cuenta es el formato (`common/transfer_layout.slang`).

valores	formato	origen
9	grado 2, directa	primer transfer
36	grado 2, directa + indirecta (27)	primer transfer
84	grado 2, ambas + campo reflejado (48)	TX con `--transfer-degree 2`
16	grado 3, directa	
64	grado 3, directa + indirecta (48)	
112	grado 3, ambas + campo reflejado (48)	**TX por defecto**
10	dos lóbulos zonales en el marco de la gaussiana	nubes con esqueleto

Junto a ella:

- **Direcciones abiertas** (`primvars:athenea:splat:shadowBits`): 2 palabras (64 bits, una rejilla octaédrica de 8 x 8; primer transfer), 8 palabras (16 x 16 celdas sobre la esfera entera; TX por defecto) o 32 palabras (32 x 32). Un bit se pone donde el rayo del horneado en esa celda salió de la escena.
- **Curvatura** (`primvars:athenea:splat:curvature`): el operador de forma en los dos ejes del plano de la gaussiana, (S_{uu}, S_{uv}, S_{vv}), tres floats, escrita con `--transfer`.

- **Rugosidad del material** (`primvars:athenea:splat:materialRoughness`): la rugosidad propia del material antes del ensanche del horneado (sección 4.9), -1 bajo un mapa de textura. En el dispositivo va en cinco bits (27-31) de la palabra pbr, marcada por el bit 26.

La mitad directa es un escalar por coeficiente (solo geometría); la indirecta es rgb, coloreada por aquello en lo que rebotó la luz; el campo reflejado es rgb. El formato TX cuesta, por gaussiana:

	primer transfer	**TX**
directa	9	16
indirecta	27	48
campo reflejado	–	48
visibilidad	2 ints (64 bits)	8 ints (256 bits)
fichero, bytes	152	480
dispositivo, bytes	80	256

Para el Corvette entero (unos 10 M de gaussianas entonces) son 4.8 GB de fichero y 2.6 GB de dispositivo solo para el transfer; los códecs que lo llevan a un teléfono (un PCA por clusters del transfer, un codebook k-means de Hamming de los bits de sombra) se describen en la sección 7.

4.3 El entorno en el momento de dibujar

`technique::Environment` prepara cada domo en tres formas, todas buffers leídos a través de un único módulo (`light/environment.slang`) por el rasterizador y por el trazador.

Armónicos esféricos de radiancia. `env_project` suma los téxeles de un nivel de la cadena de mips del domo, cada uno pesado por el ángulo sólido lat-long $2\pi^2 \sin\theta \, \Delta u \, \Delta v$ (la medida que usa el muestreo por importancia del domo):

$$L_k = \sum_{\text{téxeles}} L(\omega_t)\, Y_k(\omega_t)\, 2\pi^2 \sin\theta_t \, \Delta u \, \Delta v.$$

Se proyectan dieciséis coeficientes (grado 3); la irradiancia de un cuerpo sin transfer lee los nueve primeros, convolucionados al leer con los pesos zonales del coseno truncado $\{\pi, 2\pi/3, \pi/4\}$. Un cielo constante vuelve a dar πL de irradiancia en toda normal: 6.2841 frente a 6.2832 (0.089 %) sobre 64 direcciones.

El sol, sacado. Un disco mil veces más brillante que el cielo no es una función de grado 2; proyectado, se emborrona en un lóbulo de sesenta grados. `env_sun` encuentra la región más brillante cuya luz está tan concentrada como un disco de su ángulo sólido (ángulo cuadrático medio pesado por la luz alrededor de su centro como mucho el doble que el de un disco, $\theta^2_{\text{cone}}/2$), saca sus téxeles tanto de los armónicos como de la fuente del prefiltrado, y devuelve una dirección $\boldsymbol{s}$, una irradiancia $E_\odot$ y un semiángulo. La irradiancia se suma exactamente sobre los téxeles que los armónicos omiten, al mismo nivel y con la misma medida, de modo que lo que sale de los coeficientes llega como luz (forma cerrada 0.1119 frente a 0.1113).

El mapa prefiltrado. El cielo sin su sol, convolucionado con un lóbulo GGX en una cadena de mips octaédrica de ocho niveles, el nivel k para la rugosidad $(k/7)^2$ (0, 0.020, 0.082, 0.184, 0.327, 0.510, 0.735, 1). El lado de la base iguala el téxel del propio domo, $S = \sqrt{2WH/\pi}$, redondeado hacia abajo a una potencia de dos en $[256, 2048]$. El prefiltrado usa muestreo por importancia filtrado (Krivanek y Colbert): cada muestra GGX lee la fuente un nivel más basto de lo que pide su propio ángulo sólido, nunca más estrecho que el téxel que escribe, con la secuencia girada por un hash por téxel, 64 muestras en el nivel 1 que se doblan has-

ta 1024. Los mips lat-long de la fuente pesan cada fila por $\cos\theta_0 - \cos\theta_1$, de modo que cada nivel conserva la media por ángulo sólido del cielo. Sin esto, un sol de 1.5 grados en un nivel rugoso era una dispersión de manchas del tamaño de un téxel (la banda de 4-16 px de una bola de pintura a 2.13 veces el path trace).

4.4 El horneado

4.4.1 La integral de transfer y su estimador

El transfer es la proyección de la visibilidad por el coseno truncado, con el albedo tomado como uno:

$$T_k = \frac{1}{\pi}\int_{S^2} V(\boldsymbol{\omega})\max(\boldsymbol{n}\cdot\boldsymbol{\omega},0)\,Y_k(\boldsymbol{\omega})\,d\boldsymbol{\omega},$$

y un fotograma lo recombina con el cielo como $\langle T, L\rangle = \sum_k T_k L_k$. Lo recoge una tercera variante del kernel de path tracing, `kTransfer`, junto al horneado de radiancia y al camino de cámara. En el primer vértice de una gaussiana el material se sustituye por la propia dirección muestreada: las direcciones están estratificadas (una rejilla $\sqrt{N}\times\sqrt{N}$ con jitter) y son uniformes sobre el hemisferio, densidad $1/2\pi$, frente a un estimador que quiere $\cos\theta/\pi$, así que una muestra que escapa vale $2\cos\theta\,Y_k(\boldsymbol{\omega})$ y una bloqueada nada. Uniforme en lugar de coseno, y sobre el hemisferio medido como una esfera con la mitad lejana vacía, porque los armónicos son ortonormales solo sobre la esfera; ajustados coeficiente a coeficiente sobre un hemisferio se pasan (el peón salía diez veces demasiado brillante). La acumulación *es* la proyección; no se resuelve ningún ajuste.

Los mismos caminos dan dos mitades sin ningún rayo más: un camino que escapa justo después del primer vértice contribuye a la mitad directa (escalar); uno que escapa tras rebotar lleva el color de lo que encontró y contribuye a la mitad indirecta T_k^{ind} (rgb). Comprobado frente a formas cerradas: un punto sin oclusión transfiere el coseno truncado, $(0.2820, 0.3258, 0.1581)$ frente al exacto $(0.2821, 0.3257, 0.1577)$ para las bandas 0-2, y un cielo constante de 0.7 lee 0.6998. Una esquina de suelo y pared, transfer recombinado frente a un horneado de radiancia de los mismos puntos:

	transfer	horneado de radiancia	peor punto
sin pared, 1 rebote	0.6000	0.6000	0.0 %
pared negra, 1 rebote	0.3091	0.3039	2.5 %
pared blanca, 3 rebotes	0.5095	0.5325	4.3 %

4.4.2 Direcciones abiertas y la mitad directa por cuadratura

En la primera muestra de cada punto TX traza un rayo any-hit por celda de una rejilla octaédrica de $S_c \times S_c$ ($S_c = 16$ o 32) sobre la esfera *entera*: también la mitad lejana, empezando desde la cara de atrás de la superficie, de modo que una dirección detrás de un sólido encuentra el sólido y una detrás de una lámina sale. El bit de una celda se pone donde pasa al menos la mitad de la luz: el rayo de oclusión atraviesa hasta ocho superficies cuyo material transmite, pesadas por la transmisión por la luminancia del color menos el Fresnel a ese ángulo (una novena cierra el paso). Las celdas se trazan desde cuatro orígenes sobre la huella de la gaussiana (la celda c desde el origen $(c + \lfloor c/S_c \rfloor) \bmod 4$: el centro y tres puntos llevados a la superficie dentro del disco de la gaussiana), de modo que una ranura más estrecha que una gaussiana ya no se lee como una línea discontinua de sombras enteras.

Los mismos rayos dan entonces la mitad directa por cuadratura (sin grano), cada celda pesada por su ángulo sólido octaédrico:

$$T_k^{\text{dir}} = \sum_c \tau_c \, \frac{\max(\boldsymbol{n}\cdot\boldsymbol{\omega}_c, 0)}{\pi} \, \Delta\Omega_c \, Y_k(\boldsymbol{\omega}_c), \qquad \Delta\Omega_c = \frac{4}{S_c^2} \left\| \boldsymbol{\omega}_c \right\|_1^3,$$

con τ_c la fracción transmitida a lo largo del rayo de la celda. Las palabras se escriben de cuatro en cuatro a medida que se trazan cada 128 celdas, de modo que el kernel no guarda ningún array del tamaño de la rejilla.

4.4.3 El campo reflejado

Una reflexión necesita la luz rebotada *por dirección*, no integrada. Para cada camino que sale de la gaussiana, encuentra la escena y luego escapa, el throughput pasado el primer vértice es la radiancia que llega por su primera dirección $\boldsymbol{\omega}_1$ bajo un cielo blanco de radiancia uno. Se proyecta sobre el grado 3 de $\boldsymbol{\omega}_1$, uniformemente sobre el dominio del que se sacó:

$$F_j \approx \frac{1}{N} \sum_{i:\ \text{escapó tras un rebote}} \Omega_{\text{meas}} \, \frac{\beta_i}{w_i} \, Y_j(\boldsymbol{\omega}_{1,i}),$$

con β_i el throughput, w_i el peso del primer vértice y $\Omega_{\text{meas}} = 2\pi$ (o 4π sobre la esfera). Son 16 coeficientes rgb, `primvars:athenea:splat:transferReflected`. No cuesta ningún rayo. Como F es cero en las direcciones abiertas, dieciséis armónicos leen una dirección cerrada junto a una abierta a una fracción de lo que hay allí (la parte cerrada de la pintura era 0.0014 de 0.115). Así que el horneado proyecta también el indicador de las direcciones cerradas C_j sobre las mismas muestras, y la conversión divide ambos, una convolución normalizada que da la radiancia media de las direcciones cerradas alrededor de cada dirección, cae a la media de las direcciones cerradas donde su proporción está por debajo de 0.1 (mezclada hasta 0.4), y vuelve a proyectar el resultado sobre una rejilla de 16 x 16 por el ángulo sólido de cada celda (`m2sFieldOverClosed`).

Para una gaussiana que transmite, la primera dirección se saca sobre la esfera entera (`bakeSphereDirection`: la altura del estrato estirada a $[-1, 1]$ y la mitad inferior reflejada en el plano tangente). Las muestras de delante conservan el estimador del transfer ($4\cos\theta$ con densidad $1/4\pi$); las de detrás alimentan solo el campo, tras doblarse hacia dentro de un dieléctrico sólido por su índice (`refractInto`), de modo que el campo a lo largo de $-\boldsymbol{\omega}_o$ contiene lo que el rayo del ojo encuentra a través de ambas caras. Sin doblar, una bola de vidrio validaba a la mitad del path trace (0.265 frente a 0.541 bajo un domo blanco).

4.4.4 Rebanadas, filtrado y el horneado partido

El Corvette TX entero pedía la respuesta de su horneado de una vez: 35 float4 por gaussiana, 7.8 GB, con 5 GB libres. `Converter::transfer` hornea por tanto en rebanadas cuya respuesta es como mucho 1.5 GB (nunca por debajo del lote propio del horneado, 2^{19}), y como mucho 2^{20} gaussianas donde corre el filtro. Los rayos de cada rebanada se copian en el dispositivo, se hornean (`StageRenderer::bakePointsOnDevice` en modo transfer), se filtran sus mitades rebotadas, se escriben con `m2sTransferInto` y se leen de vuelta a los arrays del fichero antes de la siguiente. El horneado de radiancia usa el emparentado `StageRenderer::bakeSplitOnDevice`, que guarda las mitades directa e indirecta como sumas separadas para que la indirecta pueda filtrarse antes de que `combineBake` ajuste ambas. El transfer sigue el mismo principio: su mitad indirecta y su campo (los valores más ruidosos, a 64 caminos por gaussiana en los horneados del Corvette) pasan por el filtro de horneado de splats, un filtro à-trous de `--bake-filter` iteraciones (3), cuyos pesos son la posición en el plano tangente, la orientación y el id de prim; la mitad directa, que contiene las sombras, no se filtra nunca.

4.4.5 Lóbulos zonales para nubes con esqueleto

Nueve armónicos están en el mundo; un esqueleto gira la gaussiana debajo de ellos. Rotarlos cuesta una 9 x 9 por gaussiana. Un lóbulo zonal rota rotando su eje, así que una nube con esqueleto guarda dos lóbulos, cada uno con el (u, v) octaédrico de su eje en el marco propio de la gaussiana y los coeficientes zonales z_0, z_1, z_2: diez valores (`primvars:athenea:splat:transferZonal`). Como armónicos, un lóbulo alrededor de $\boldsymbol{a}$ es $g_{lm} = z_l\, k_l\, Y_{lm}(\boldsymbol{a})$ con $k_l = \sqrt{4\pi/(2l+1)}$. Para un eje fijo los mejores coeficientes son la proyección $z_l = k_l \sum_m f_{lm} Y_{lm}(\boldsymbol{a})$, que deja un error $\|f\|^2 - \sum_l z_l^2$, así que el ajuste (un efecto AOFX, `splattransferzonal`) busca el eje en el dispositivo (la dirección de la banda 1, la normal, 64 direcciones de Fibonacci y luego un paseo que divide el paso a la mitad), ajusta el segundo lóbulo al residuo y reajusta el par dos veces (Sloan 2008). Al dibujar, `splatTransferFrame` gira cada eje con el marco deformado y reconstruye nueve armónicos del mundo, que todo lo de después lee sin cambios. El horneado posa la nube en `--time` con el propio skinner del motor, de modo que el marco contra el que se ajustan los lóbulos es el cuaternión empaquetado que lee un renderer. Los bits de sombra siguen siendo dos palabras, reagrupados en el marco de la gaussiana. La mitad indirecta no se guarda zonal.

4.5 El cuerpo

Para una gaussiana con transfer, la parte que no depende del ojo es

$$\boldsymbol{B} = \sum_{k<16} L_k \left(T_k^{\text{dir}} + T_k^{\text{ind}} \right) \;+\; E_\odot \frac{\max(\boldsymbol{n}\cdot\boldsymbol{s}, 0)}{\pi} O(\boldsymbol{s}) \;+\; E_\odot\, T^{\text{ind}}(\boldsymbol{s}), \qquad T^{\text{ind}}(\boldsymbol{s}) = \max\Big(\sum_k T_k^{\text{ind}} Y_k(\boldsymbol{s}), 0 \Big),$$

siendo el último término el rebote del sol, que el primer transfer perdió con el sol que sacó de los armónicos. $O(\boldsymbol{s})$ es la parte del sol que llega: para TX la lectura bilineal de las cuatro celdas alrededor de $\boldsymbol{s}$, dejando fuera las celdas detrás de la normal; para los 64 bits lo mismo en una rejilla de 8 x 8; sin bits, el cociente de grado 2

$$O_{\text{SH}}(\boldsymbol{s}) = \frac{\sum_k T_k Y_k(\boldsymbol{s})}{\sum_k \widehat{A}_l\, Y_k(\boldsymbol{n})\, Y_k(\boldsymbol{s})}, \qquad \widehat{A}_l = \{1, 2/3, 1/4\},$$

con numerador y denominador compartiendo el truncamiento, de modo que lee uno donde nada ocluye. El cuerpo difuso es entonces $U\rho\,(1-m)\left[(1-t)\,\boldsymbol{B} + t\,(\ldots)\right]$ con U el throughput de las capas (sección 4.6.4) y t la transmisión. $\boldsymbol{B}$, la parte del sol y el acoplamiento del campo (abajo) se guardan por gaussiana mientras el cielo y las luces no cambian (`transferViewless`, el `txCache`); un fotograma calcula solo lo que cambia el ojo.

El término constante del transfer es además una oclusión ambiental: sobre un hemisferio abierto $T_0 = A_0 Y_0/\pi = 0.28209$, así que $o = \text{sat}(T_0/0.28209)$ es la fracción del cielo que llega a la gaussiana.

Luces que no son el cielo. Una luz distante, esférica, de disco o rectangular se evalúa con una muestra en su centro a través de los mismos lóbulos. Con las celdas TX, su sombra son las celdas leídas sobre el cono que la luz subtiende ($\tan^{-1}$ de su radio sobre la distancia, o la mitad del diámetro angular de una luz distante), la más oscura entre esa y cualquier sombra medida con rayos; su rebote es T^{ind} leído a lo largo de ella, por el L/p de la muestra, sobre el cuerpo y el sheen, y acopla el campo reflejado igual que el sol (`splatLightBounce`).

4.6 Reflexión

4.6.1 Qué parte de un lóbulo ve el cielo

El primer transfer estrechaba toda reflexión con el ajuste de Lagarde y de Rousiers del término constante,

$$O_{\mathrm{L}}(\boldsymbol{n}\cdot\boldsymbol{\omega}_o, o, r) = \mathrm{sat}\Big((\boldsymbol{n}\cdot\boldsymbol{\omega}_o + o)^{2^{-16r-1}} - 1 + o\Big),$$

un número para todo el hemisferio: una puerta reflejaba el cielo abierto donde el suelo está en su espejo, y las llantas del Corvette leían 2.4 veces la malla con path trace, `Metal_rough` 6.5 veces, los discos de freno cien. Con las celdas TX se pregunta al propio lóbulo (`splatLobeOpen`, `splatConeOpen`): la lectura de celdas $o_c(\cdot)$ (bilineal sobre cuatro celdas, excluidas las que quedan detrás de $\boldsymbol{n}$) en el espejo y en un anillo de seis direcciones a su alrededor con ángulo

$$\theta_r = \max\Big(\min(1.4\tan^{-1}\alpha,\ 1.2),\ 2\cdot\frac{\pi}{2S_c}\Big),$$

más o menos el ángulo dentro del cual GGX refleja la mitad de su energía, nunca más estrecho que una celda; y

$$O(\boldsymbol{m},\alpha) = \frac{\frac{1}{3}o_c(\boldsymbol{m}) + \frac{1}{9}\sum_{k=1}^{6} o_c(\boldsymbol{d}_k)}{\frac{1}{3}+\frac{6}{9}}$$

sobre las lecturas que pesaron algo. Un cono dentro de una celda leía el único bit de esa celda y salpicaba la pintura del Corvette (relMSE 0.393 frente al path trace; 0.298 con el cono limitado a una celda).

4.6.2 Qué muestra la parte cerrada: el campo acoplado al cielo

El campo se horneó bajo un cielo blanco. Bajo uno real su patrón se mantiene y su brillo se escala por canal según cuánto más rebota el entorno bajo ese cielo que bajo el blanco (`splatFieldCoupling`):

$$\kappa = \frac{\sum_k T_k^{\mathrm{ind}} L_k + E_{\odot}\, T^{\mathrm{ind}}(\boldsymbol{s})}{\sqrt{4\pi}\ \ T_0^{\mathrm{ind}}},$$

cayendo a la radiancia media del cielo donde la gaussiana no rebota nada. El acoplamiento es de rango uno: exacto para una gaussiana rodeada por una superficie iluminada por un cielo, y pone el color de un suelo iluminado, no su forma, bajo un cielo iluminado por un lado. Leído a lo largo de una dirección a la rugosidad de un lóbulo, las bandas del campo se estrechan como un GGX reflejado tomado como una von Mises-Fisher de concentración $1/2\alpha^2$:

$$F(\boldsymbol{d};\alpha) = \max\Big(\sum_{j<16} F_j\, e^{-l_j(l_j+1)\alpha^2}\, Y_j(\boldsymbol{d}),\ 0\Big).$$

Las dos rugosidades que necesita una gaussiana (polish y coat) se sacan de una sola lectura de los 48 valores (`splatFieldAlongPair`). Las alternativas que se sopesaron fueron una matriz de transfer de 9 x 9 por gaussiana (243 números, 5 GB para el coche, y aún grado 2) y un índice por dirección dentro de la nube (necesita una caché de radiancia y no ve nada de un suelo de malla).

4.6.3 Polish y coat

La reflexión de la base («polish») y el coat son

$$\boldsymbol{R}_{\mathrm{polish}} = U\,g\,P(\boldsymbol{n}\cdot\boldsymbol{\omega}_o)\left[O\,\tilde{L}(\boldsymbol{m},r) + (1-O)\,\chi\,\kappa\,F(\boldsymbol{m};\alpha)\right] + U\,g\,P\,E_{\odot}\,O(\boldsymbol{s})\,p_{\odot}(\boldsymbol{m},\boldsymbol{s}),$$

$$\boldsymbol{R}_{\mathrm{coat}} = w_c E_c\left[O_c\,\tilde{L}(\boldsymbol{m},r_c) + (1-O_c)\,\kappa\,F(\boldsymbol{m};\alpha_c)\right] + w_c E_c\,E_{\odot}\,O(\boldsymbol{s})\,p_{\odot}(\boldsymbol{m},\boldsymbol{s}),$$

donde P es el albedo de entorno del polish, g la ganancia de pared fina (sección 4.7), E_c el albedo del coat y χ la puerta del campo del polish:

- **La apertura del polish y su puerta.** $\chi = 1$ y $O = O(\boldsymbol{m}, \alpha)$ solo donde la rugosidad propia del material es al menos `kPolishFieldRoughness` $= 0.3$; en el resto $\chi = 0$ y $O = O_{\mathrm{L}}$. En el cromo (0) y el metal rugoso (0.22) del Corvette, la apertura por celdas leía un relMSE de 1.64 y 0.221 frente al path trace donde la de los armónicos leía 1.05 y 0.128 para la misma media, y el campo leía los metales un tercio más claros (cromo 0.261 donde el path trace lee 0.197). Un lóbulo de 0.3 o más ancho es más ancho de lo que resuelve el grado del campo, así que lo que lo cierra (el suelo de una bola) es lo que dice el campo: bolas verdes y blancas a 0.3421 sobre un suelo gris reflejaban el cielo en su mitad inferior sin esto. La puerta lee la rugosidad *del material*, no la guardada, porque el ensanche del horneado lleva la rugosidad guardada de un tubo fino de cromo por encima de 0.3. El coat usa siempre las celdas y el campo.
- **El sol analítico.** El mapa no contiene sol, así que el sol se añade a través de la distribución de direcciones reflejadas de cada lóbulo con el ojo a lo largo de la normal, $D(\boldsymbol{h})/4$ para el vector medio entre espejo y sol, con el semiángulo del disco $\theta_\odot$ incorporado a la anchura:

  $$p_\odot(\boldsymbol{m}, \boldsymbol{s}) = \frac{\tilde{\alpha}^2}{4\pi\, d^2}, \qquad d = \cos^2\!\frac{\theta}{2}(\tilde{\alpha}^2 - 1) + 1, \qquad \tilde{\alpha}^2 = \alpha^2 + \theta_\odot^2, \qquad \cos\theta = \boldsymbol{m}\cdot\boldsymbol{s}.$$

  Integra uno, así que peso por $E_\odot$ por $p_\odot$ es la reflexión entera del sol. El split sum leía un sol compacto una cuarta parte corto en todo nivel del mapa (0.57-0.74 del path trace en una bola de metal flotante; parte de esa diferencia se atribuyó después a la densidad del domo del propio path tracer, ya corregida).

4.6.4 Capas, Fresnel, metales, sheen

Albedo de entorno. P es el `ggxDirectionalAlbedo` de MaterialX con compensación de dispersión múltiple, no el ajuste de Lazarov, que difería hasta un 76 % en rasante; un dieléctrico usa el Fresnel exacto frente a F_0, un conductor su color en ambos extremos, mezclados por m. Donde el metal del material es un Schlick generalizado (OpenPBR, glTF), va de su color de frente al color especular en rasante (`ggxEnvSchlickMetal`); para un metal oscuro un conductor de índice artístico y un Schlick difieren al doble a sesenta grados, y la pintura del coche se leía mate y clara como conductor.

El Fresnel del coat. Schlick se queda bajo en el medio de la curva de un dieléctrico (0.070 a sesenta grados donde una laca a 1.45 refleja 0.081); un coat sobre negro leía 0.084 frente a 0.099 del path trace. El albedo del coat es el ajuste escalado por exacto sobre Schlick en el ojo, desvaneciéndose con la rugosidad:

$$E_c = E_{\mathrm{fit}}(r_c, \boldsymbol{n}\cdot\boldsymbol{\omega}_o, \eta_c)\cdot \mathrm{lerp}\!\left(\frac{F(\boldsymbol{n}\cdot\boldsymbol{\omega}_o, \eta_c)}{F_{\mathrm{Schlick}}},\ 1,\ \mathrm{sat}(\alpha_c)\right),$$

y exactamente F por debajo de $\alpha_c = 10^{-3}$. Frente a una cuadratura del lóbulo queda dentro del 3 % con α 0.3 y 0.6, donde el ajuste se quedaba un 8-16 % por debajo.

Capas. Como el nodo `layer` de MaterialX: el coat sobre el sheen sobre la base, cada capa quitando de lo que hay debajo la parte que refleja en el ojo. Con E_{fit} el albedo ajustado del coat, E_s el albedo direccional del sheen y w_s el mayor canal del color del sheen,

$$U = \left(1 - w_c E_{\mathrm{fit}}\right)\left(1 - w_s E_s\right)\cdot \mathrm{lerp}\!\left(1,\ \frac{1-K}{1-E_b K},\ w_c\right), \qquad K = 1 - \frac{1 - F_0(\eta_c)}{\eta_c^2},$$

siendo el último factor el oscurecimiento del coat de OpenPBR (la luz que vuelve de la base se refleja de nuevo hacia abajo en la cara interior del coat), con E_b el albedo de la base; la bola de pintura leía 1.5 veces la malla en su centro sin él. El sheen (Imageworks, Conty y Kulla) es un lóbulo tan ancho como un cuerpo,

así que se responde como uno: su albedo direccional por $\boldsymbol{B}$, bajo el coat.

4.7 Vidrio, transmisión y paredes finas

Una gaussiana no puede refractar. Lo que transmite es la luz que le llega por detrás: $(1-t)$ del cuerpo mira hacia el lado del ojo, t mira hacia el otro, y lo que hay detrás se ve a través de la opacidad de la gaussiana.

- **Pasa lo que no se refleja**: la mitad transmitida se pesa por $1 - w_{\text{spec}} E_{\text{fit}}(r,\eta)$, con w_{spec} el peso especular; un Fresnel por cara, como lo paga el lóbulo de transmisión del path tracer; antes una bola salía la mitad más brillante, con un borde 2.5 veces demasiado caliente.
- **El vidrio TX rasterizado** ve a través a lo largo de $-\boldsymbol{\omega}_o$: el cielo donde las celdas de detrás están abiertas (`splatConeOpen` alrededor de $-\boldsymbol{n}$) y κF donde están cerradas, de modo que un parabrisas muestra el habitáculo, emborronado a grado 3. **Un vidrio sólido es una lente**: su propia cara lejana cierra todas las celdas de detrás, así que el valor del campo bajo cielo blanco a lo largo de $-\boldsymbol{\omega}_o$ decide la apertura (smoothstep$(0.5, 0.85)$ de su luminancia) y la parte abierta lee el cielo nítido a lo largo de la salida de una esfera estimada a partir de la curvatura media guardada (`lensExit`: doblado al entrar, a través de la cuerda, doblado al salir), con el Fresnel y el segundo tinte de la cara lejana (`lensExitThrough`). Llenar la lente con el campo dibujaba la cabeza transparente del peón lechosa y plana (0.58 frente a la malla con path trace).
- **Rugosidad a través del vidrio**: una inclinación de microfaceta θ gira un rayo especular 2θ pero uno transmitido mucho menos, así que el cielo a través de ambas caras se lee a $r_t = r\sqrt{\text{sat}((\eta-1)/\sqrt{2}\,)}$; la cabeza del peón pasó de un relMSE de 0.081 a 0.048.
- **La reflexión de la cara lejana** devuelve $(1-R)\,R\,(1-R)$ por el tinte de la luz a lo largo del espejo (el color de la gaussiana lleva el primer tinte); una bola transparente bajo un cielo uniforme leía 0.952 sin ella, 0.9998 con ella (0.92 a través de ambas caras + 0.04 de polish cercano + 0.037 esto).
- **Paredes finas.** Una placa clasificada como fina (aristas abiertas, o un grosor $2V/A$ por debajo de cuatro celdas o de un cincuentavo de su propio tamaño) se convierte con opacidad $2R_0/(1+R_0)$, $R_0 = ((\eta-1)/(\eta+1))^2$, y su reflexión se escala por la ganancia $g = 1/R_0$ a su propio índice (R_0 con un mínimo de 10^{-3}; 1.5 solo donde no se guardó índice), de modo que la mezcla deja la reflexión entera. La tapa del faro del Corvette a 1.15 recibía 0.0097 de opacidad y, con el antiguo suelo de 1.5, una ganancia de 25 donde 205 la deja entera. Las gaussianas de una lámina se dibujan con una opacidad de al menos 0.1 con su color reducido en la misma proporción (y su rampa con él): relMSE del parabrisas de 2.26 a 0.89 bajo el taller, de 0.31 a 0.067 bajo goegap.

4.8 La reflexión a lo largo de la huella

La investigación midió el error de la pintura como nitidez, no energía: frente al path trace emborronado un píxel su relMSE bajaba de 0.80 a 0.28. Una gaussiana mostraba la reflexión de su centro plana sobre toda su huella.

4.8.1 El giro de la normal por píxel

La conversión escribe el operador de forma del triángulo del centro en los ejes $(\boldsymbol{u},\boldsymbol{v})$ de la gaussiana, $S = \text{sym}\big([\Delta\boldsymbol{n}_1\ \Delta\boldsymbol{n}_2]\,[\Delta\boldsymbol{p}_1\ \Delta\boldsymbol{p}_2]^{-1}\big)$ proyectado sobre $(\boldsymbol{u},\boldsymbol{v})$ (`m2sShape`). La proyección invierte su propia linealización en el plano del splat: con J la aplicación 2×2 de coordenadas del plano a píxeles, un píxel a la derecha y uno abajo son $\boldsymbol{e}_x = J^{-1}(1,0)$, $\boldsymbol{e}_y = J^{-1}(0,1)$, y la normal a un píxel de distancia es

$$\boldsymbol{n}_x = \text{normalize}\big(\boldsymbol{n} + \boldsymbol{h}_x\big), \quad \boldsymbol{h}_x = \text{clamp}_{0.5}\big((S\,\boldsymbol{e}_x)_u\,\boldsymbol{u} + (S\,\boldsymbol{e}_x)_v\,\boldsymbol{v}\big),$$

con el giro limitado a media unidad para que un splat visto de canto no barra el cielo.

4.8.2 La pendiente, y el lóbulo nítido

Para una reflexión rugosa la proyección vuelve a leer el cielo a lo largo de reflect($-\boldsymbol{\omega}_o, \boldsymbol{n}_x$) y $\boldsymbol{n}_y$ y guarda el cambio del color por píxel, $\boldsymbol{g}_x, \boldsymbol{g}_y$ (`kSlopeMark`). La mezcla dibuja $\max(\boldsymbol{c} + \boldsymbol{g}_x d_x + \boldsymbol{g}_y d_y, 0)$ en el desplazamiento $\boldsymbol{d}$. La mayor caída de la rampa sobre la elipse de tres sigma se limita al color por canal, $3\sqrt{\boldsymbol{g}^\top \Sigma\, \boldsymbol{g}} \;\le \boldsymbol{c}$, de modo que el sombreado nunca llega a cero dentro de la huella (antes, una línea negra discontinua a lo largo del anillo dorado de un peón). La pendiente de un lado ganó a la diferencia central y a minmod en el Corvette (pintura 0.393 frente a 0.584 y 0.564 de relMSE).

Una rampa lineal no puede dibujar un punto. Un coat más liso que `kSharpRoughness` (0.2) y un polish de base más liso que `kSharpPolishRoughness` (0.5) se pasan en cambio a la mezcla (`kSharpMark`): la proyección resta del color la lectura del centro (cielo y sol analítico), sin recortar, y escribe la coordenada octaédrica del espejo $\boldsymbol{a}_0$ y sus pasos $\Delta\boldsymbol{a}_x, \Delta\boldsymbol{a}_y$ (puestos a cero a través del pliegue del octaedro), los pesos de los lóbulos (peso del coat por O_c; peso rgb del polish por O), su rugosidad y sus pesos de sol. Por píxel la mezcla decodifica $\boldsymbol{m}(\boldsymbol{d}) = \mathrm{octDecode}(\boldsymbol{a}_0 + \Delta\boldsymbol{a}_x d_x + \Delta\boldsymbol{a}_y d_y)$ y suma

$$w_c\, \tilde{L}(\boldsymbol{m}(\boldsymbol{d}), r_c') + \boldsymbol{w}_p\, \tilde{L}(\boldsymbol{m}(\boldsymbol{d}), r_p') + E_{\odot}\big(\boldsymbol{w}_{p\odot}\; p_{\odot}(\boldsymbol{m}(\boldsymbol{d}), \boldsymbol{s}; r_p') + w_{c\odot}\; p_{\odot}(\boldsymbol{m}(\boldsymbol{d}), \boldsymbol{s}; r_c')\big),$$

recortando después. Un sol de 1.5 grados reflejado en un coat curvo se dibuja donde cae dentro de la gaussiana y no como una mancha del tamaño de la gaussiana. Coste: una lectura del cielo por píxel por registro nítido, y la memoria de grupo de la pendiente en 17 412 de los 17 416 bytes de WebGPU.

4.8.3 La dispersión propia del píxel

La lectura nítida es una muestra puntual en el centro del píxel, mientras que la normal gira $\boldsymbol{h}_x, \boldsymbol{h}_y$ a través del píxel: una caja de pendientes cuya varianza, doblada para el espejo, ensancha el lóbulo:

$$\alpha'^2 = \alpha^2 + 2 \cdot \frac{\left\|\boldsymbol{h}_x\right\|^2 + \left\|\boldsymbol{h}_y\right\|^2}{12},$$

con $\alpha' \le 1$, aplicada después del des-ensanche y del ensanche de pliegue de la sección 4.9. El labio enrollado de un panel ya no centellea. El port del lóbulo nítido al teléfono no tenía este término y dibujaba motas brillantes en los pliegues de la puerta y del umbral: con él, 27/25 motas pasaron a 2/1 (en el coche de 1280 px el teléfono leyó entonces la puerta a 35.53 dB y el umbral a 32.36 frente al path trace, desde 33.77 y 31.87 de la versión anterior, que además tenía el horneado de 640 px).

4.9 Deshacer el ensanche del horneado

4.9.1 Toksvig en la conversión

Una gaussiana tiene una normal. Donde una tira fina y curva de cromo gira la reflexión a lo largo de una gaussiana, el píxel con path trace promedia un brillo que se mueve en una línea fina, y una gaussiana dibujaba una cuenta de su propio tamaño a pleno brillo (el cromo del Corvette leía 1.6 veces el path trace). La conversión ensancha la rugosidad, y la del coat, por la dispersión de normales bajo la gaussiana (Toksvig):

$$s = \lambda \max_{\text{aristas } kj} \frac{\angle(\boldsymbol{n}_k, \boldsymbol{n}_j)}{\left\|\boldsymbol{p}_j - \boldsymbol{p}_k\right\|} \max(\sigma_u, \sigma_v), \qquad \alpha'^2 = \alpha^2 + 2s^2,$$

con $\lambda =$ `--specular-filter` (1 con `--transfer`) y $r' = \mathrm{sat}(\alpha'^{1/2})$.

4.9.2 Lóbulos nítidos sin ensanchar

Los lóbulos nítidos ya giran el espejo por píxel; leídos a la rugosidad ensanchada, el giro se cuenta dos veces. La laca de una puerta, un espejo ($r_c = 0$), salía mate: guardada a $p_{50} = 0.112$ en la puerta y 0.080 en el capó (investigación t63b), saturando a 1 en los pliegues. La proyección recupera la dispersión a partir de la curvatura guardada,

$$s \approx \kappa_{\max} \max(\sigma_u, \sigma_v), \qquad \kappa_{\max} = \left|\frac{S_{uu} + S_{vv}}{2}\right| + \sqrt{\left(\frac{S_{uu} - S_{vv}}{2}\right)^2 + S_{uv}^2},$$

y lee el coat y el polish nítidos a

$r = \left(r'^4 - 2s^2\right)^{1/4}$ (limitado a 0).

4.9.3 El coat del pliegue

En un pliegue la laca ensanchada llegaba casi a 1 y una junta dibujaba una raya oscura (juntas de la puerta a 0.69 del path trace). El coat se limita:

$$r_c \leftarrow \max\left(\text{unwidened}(r_c, s),\ \min(r_c,\ 0.3)\right) \quad \text{si } r_{\text{mat}} \geq 0.3,$$

`kCoatFoldRoughness` = 0.3. Aplicado en todas partes, iluminaba el cromo, el vidrio y los radios de una llanta que el path trace muestra oscuros (cromo -1.4 dB, radios -0.5 dB por región en el visor web), de ahí la puerta solo para pintura. A 1280 px el horneado ya no ensancha el coat (sale exactamente a 0), así que en el coche re-horneado este término no cambia nada.

4.9.4 Un tubo gira, un pliegue se ensancha

El giro por píxel extrapola el operador de forma del triángulo del centro; pasada una arista deja de valer. La normal y la curvatura guardadas son exactas en el centro (normal a p_{90} 0.009 grados de la normal interpolada de la malla), pero el 64 % de las huellas $\pm 2\sigma$ de un capó cruzan una arista, y el error RMS de la normal extrapolada llegaba a p_{90} 46 y p_{99} 198 grados. Sea $t = 2s$ el giro a lo largo de $\pm 2\sigma$. En la nube de 1280 px el cuerpo de un tubo de cromo está en $t \in [0.3, 1]$, coherente con sus vecinas (2 % de signo opuesto); las normales suavizadas a través de un pliegue dentro de triángulos más estrechos que la gaussiana están por encima de 1 (de 5 a 9 rad en un umbral). Con

$$w = \text{smoothstep}(0.7,\ 1.5,\ t), \qquad S \leftarrow (1 - w)\, S, \qquad \alpha' = \min\left(\sqrt{\alpha^2 + w\,(t/2)^2},\ 0.5^2\right)$$

(`kFoldTurnLow` 0.7, `kFoldTurnHigh` 1.5, `foldWidened` limitado a `kFoldRoughness` 0.5 en r, nunca más estrecho que antes), un tubo gira sus píxeles del todo y dibuja su línea fina de luz, mientras que un pliegue se integra como lo integra el path tracer dentro de un píxel. El smoothstep evita que una cámara en movimiento vea un salto.

4.10 Medidas frente al path trace

Todas las cifras del coche son frente al path trace de la malla `simplify3/pt/day_pt_Camera.exr` (`/World/Camera`, 1920 x 1080, `autoshop_01_4k`). «Recorte del coche» es el PSNR sobre el recorte de 900 x 420 del coche, «fotograma» sobre el fotograma entero; los cocientes por clase son la media de una región sobre la del path trace (investigación `proto/door/*.py`).

4.10.1 La puerta TX, por material (relMSE)

estado	cromo	metal rugoso	pintura
primer transfer	0.536	0.0896	0.705
TX, celdas y campo en todo	1.64	0.221	0.393
polish abierto por armónicos, cono ≥ celda, campo solo en el coat	0.591	0.0835	0.341
después: + coat nítido y polish nítido por píxel	0.545	0.083	0.281

4.10.2 Coste de reproducción

El Corvette TX entero (14.7 M de gaussianas, sin suelo, 1920 x 1080) se dibujaba en 158 ms frente a unos 83 de una nube reiluminada de su tamaño; de los 84.5 ms de la proyección, el campo y su acoplamiento se llevaban 24, las celdas leídas por los lóbulos 16.5, la parte del sol y su rebote 12, el transfer del cuerpo 7.5. Leer el campo una sola vez para ambos lóbulos y guardar en caché los términos que no dependen del ojo atacan el primero y el último.

4.10.3 El coche entero, T63-T75

prueba	qué	recorte del coche (dB)	fotograma (dB)
T63b	hojas a 640 px (5.38 M), antes de los pasos de esta sección	16.37	21.92
T64	hojas, lóbulos nítidos sin ensanchar (b26d276)	16.58	22.15
T63c	corte a 0.25 px con culling de caras traseras	16.29	–
T65 cut_a	+ sin ensanchar + coat limitado en pliegues	16.51	22.15
T65 cut_b	+ campo del polish en materiales rugosos (35ae3cb)	16.55	22.18
T66	hojas (corte a 0.01 px), sin cull / con cull	16.73 / 16.75	22.40 / 22.43
T69 / T70	+ giro limitado a 0.2 rad / + coat de pliegue solo en pintura	16.57 / 16.57	22.20 / 22.20
T71	**horneado a 1280 px**, corte a 0.25 px, culling de caras traseras (5.0 M)	**17.88**	**23.90**
T72	T71 sin el límite	17.88	23.91
T73	T71 + polish nítido con puerta en la rugosidad sin ensanchar	17.88	23.90
T74	T73 + separación tubo/pliegue	17.90	23.93
T75	T71 + separación tubo/pliegue (la que queda)	17.88	23.91

El PSNR sobre un recorte se mueve poco; los cocientes por clase de abajo muestran dónde actúan los cambios.

4.10.4 La puerta y el capó, por clase (investigación t63b-t66)

La pintura es una base de metal oscuro (rugosidad 0.3421) bajo un coat de rugosidad 0 a 1.45. Cada píxel se clasifica por dónde cae su espejo: el suelo (cerrado; responde el campo) o el HDRI (abierto). En t63b, el 86 % del déficit de la puerta estaba en píxeles que reflejan suelo. Media sobre el path trace:

clase de la puerta	t63b hojas	t63c corte+cull	t65 cut_a	t65 cut_b	t66 sin cull	t66 con cull
espejo hacia el suelo < 3 m	0.78	0.89	0.92	**1.02**	0.90	1.02
espejo hacia el suelo 3-30 m	0.69	0.77	0.76	0.88	0.79	0.88

clase de la puerta	t63b hojas	t63c corte+cull	t65 cut_a	t65 cut_b	t66 sin cull	t66 con cull
espejo hacia el HDRI (abierto)	0.92	1.04	1.03	1.13	1.01	1.13
juntas	0.69	0.83	0.95	**1.04**	0.90	1.02

Deshacer el ensanche (t64) dejaba las clases de suelo en 0.78/0.67 y las juntas en 0.69; el campo del polish (cut_b) subió el suelo cercano a 1.02 y el coat de pliegue las juntas a 1.04, como predecía una estimación analítica del término que faltaba (bajo-coat 0.94 x albedo del polish 0.078 x suelo reflejado 0.11 = 0.008 frente a un déficit medido de 0.0056-0.0085). El culling de caras traseras sube toda la puerta un 12 % aproximadamente (t66). El 1.13 que queda en la clase abierta se reparte según si el espejo encuentra el propio coche: el 68 % del exceso está en píxeles «coche» (2.04 del path trace), donde las celdas de 16 x 16, de unos 11 grados, dejan un 25 % de cielo abierto detrás de un retrovisor o un pilar.

Contraste local (desviación típica del paso alto sobre la media, σ = 1.5 / 4 px), relativo al path trace:

	t63b	t64	t65 cut_b	T71 (1280 px)
puerta, reflejos abiertos	0.12 / 0.26	0.40 / 0.61	0.35 / 0.53	0.56 / 0.74
capó, reflejos abiertos	0.27 / 0.38	0.41 / 0.52	0.41 / 0.51	–

Una simulación de la reflexión del coat con el espejo del path trace sobre el HDRI prefiltrado mostró que la rugosidad guardada del coat explica por sí sola el emborronado: con r_c = 0.11 la puerta conserva 0.24 / 0.31 del contraste de r_c = 0, con 0.08 el capó 0.21 / 0.28.

4.10.5 Los brillos del capó

Teselas de 24 px con brillo (168), alineadas por correlación del paso alto dentro de $$6 px:

render	desplazamiento p_{50}	σ que iguala	pico (PT 5.8)
t63b (ensanchado)	4.1 px	2.5	1.9
t68 sin giro	4.1 px	1.5	2.1
t68 sin pendiente	4.2 px	2.5	1.8
t64 / t66 (hojas)	3.2 px	1.5	3.0
t65 cut_b	3.2 px	1.5	2.6
T71 (1280 px)	2.0 px	0.75	3.3

La media del capó pasó de 0.89 a 0.98 del path trace a 1280 px. El error de extrapolación de la normal escala con el cuadrado de la celda (mediana de 0.146 grados a 640 px, 0.060 a 960, 0.032 a 1280).

4.10.6 La normal extrapolada, y la separación tubo/pliegue

Error RMS de la normal extrapolada en el capó (grados):

variante	p_{50}	p_{90}	p_{99}
sin giro	0.575	22.7	35.8
operador de forma del centro	0.146	46	198
+ límite $\|S\| 2\sigma \leq 0.2$ rad	0.147	20.2	32.0
mínimos cuadrados con muestras de la malla	0.088	9.8	14.6

El límite (T69) movió el recorte del coche de 16.55 a 16.57 dB, pero empeoró las juntas de la llanta delantera (1.48 a 1.54); T72 mostró que era irrelevante a 1280 px. La separación tubo/pliegue lo sustituyó:

	T71	T73 (puerta)	T74 (puerta + separación)	T75 (separación)
motas brillantes en el pliegue del umbral	10	11	1	1
tira cromada, RMS	0.0358	0.0535	0.0415	0.0359
tira cromada, paso alto / PT	0.41	1.12	0.58	0.41

T75 llevó además las motas de la puerta de 405 a 389 y las de la llanta de 204 a 190, sin cambiar fotograma ni recorte. A 1280 px el horneado ya no ensanchaba el coat, y las gaussianas del pliegue del umbral reflejaban como espejos con normales que giraban, rayas brillantes sobre la ranura oscura del path trace; la separación las quita sin tocar la tira. Poner en cambio una puerta al polish nítido según la rugosidad sin ensanchar (T73) triplicó el error de paso alto de la tira y se descartó, igual que `--specular-filter 0` (una línea oscura discontinua en el pliegue del umbral, cromo que centellea): el polish necesita ahí su ensanche.

4.11 Limitaciones

- **Campo de rango uno.** El acoplamiento pone el color de un suelo iluminado, no su forma, en una reflexión bajo un cielo iluminado por un lado; a grado 3 el campo resuelve un horizonte a unos veinte grados y mezcla la sombra del coche en el suelo con el suelo lejano (en simulación: correlación 0.56 a grado 3, 0.63 a grado 5, 0.85 con un mapa octaédrico de 32 x 32).
- **Resolución de las celdas.** Celdas de 16 x 16, de unos 11 grados, dejan pasar cielo donde un coat nítido refleja el propio coche (la clase abierta de la puerta a 1.13); celdas de 32 x 32 (+96 B por gaussiana) lo reducirían a la mitad.
- **Un sol por domo**; las lámparas dentro de una escena se sombrean con bits que dicen qué direcciones salen de la escena, no cuáles llegan a la lámpara. El horneado no tiene emisión de lámparas: los faros salen oscuros frente al path trace.
- **Defectos que quedan** en el coche de 1280 px: la tira cromada del parachoques delantero un 38 % más clara que el path trace y con el pico más bajo; la reflexión del suelo cercano en la llanta a 0.72; los brillos del capó desplazados 2.0 px, que una curvatura ajustada sobre la huella a partir de la malla reduciría más o menos a la mitad.
- **Los transfers zonales** guardan solo la mitad directa, y conocen solo la pose que se trazó; la oclusión entre miembros en otras poses no se multiplica.

5. Assets animados: esqueletos, skinning y visibilidad por partes (el peón y el gorrión)

Esta sección describe cómo athenea convierte un asset USD con rig (UsdSkel) en una nube de gaussianas 3D a la que un esqueleto sigue moviendo. Trata cómo se convierte la nube, cómo se posa en la GPU en cada frame, cómo su luz y su autosombra siguen siendo válidas bajo la deformación, cómo la tratan los niveles de detalle y cómo se reproduce en un navegador y en un teléfono. Dos assets recorren la sección. El gorrión molinero (`SparrowBird.usda`, 609 articulaciones, 71 clips de animación) es el caso de prueba animado. El peón de The Open Chess Set es el control rígido. Todas las cifras proceden de `docs/decisions.md`, de los ficheros fuente que se nombran junto a ellas o de los ficheros de medida del reproductor web (`sparkwebGPU`). Las cifras que no se midieron se indican como no medidas.

5.1 Principio de diseño: llevar el rig, no los frames

Hay dos maneras de animar una nube de gaussianas. La primera guarda la posición, la orientación y la escala de cada gaussiana en cada frame. La segunda guarda las gaussianas una vez, en la pose de bind, junto con las articulaciones que mueven a cada una, más una transformación por articulación y por frame. athenea usa la segunda, por tres razones, todas medidas:

1. **Tamaño.** Para un personaje verosímil de 150 k gaussianas en 120 frames, los arrays por frame cuestan 960 MB en media precisión (1.2 GB en float). El rig (cuatro articulaciones por gaussiana, 60 articulaciones por frame) cuesta 2.9 MB. Se midió, tal como se escribe, un tubo sintético (200 anillos × 64 segmentos, 40 articulaciones, 184 320 gaussianas). Su fichero ocupa 9.27 MB con 25 instantes y 9.50 MB con 121, frente a unos 713 MB para el mismo rango como arrays por frame. Cinco veces los frames cuestan 230 kB.
2. **Exactitud entre muestras.** Un rig es exacto en cualquier instante del rango. Una nube muestreada solo es tan buena como la interpolación del lector.
3. **Identidad estable.** La nube posada se escribe en buffers que la sobreviven, así que un renderer ve la misma nube de un frame al siguiente. Eso es lo que permite al trazador de rayos *reajustar* su estructura de aceleración en lugar de reconstruirla (§5.6).

El coste de esta elección es que todo lo horneado en espacio de mundo (radiancia, transfer en armónicos esféricos, visibilidad) deja de ser válido en cuanto se mueve un miembro. De esto se ocupan §5.7 y §5.8.

5.2 Lectura de un stage con rig

El conversor (`athenea mesh2splat`) no pasa por Hydra. Lee las mallas con su propio `MeshStage`, lo que deja dos maneras de tratar una malla con skinning:

- **Posada.** Sin `--skinned`, el stage se posa antes de leerlo. `UsdSkelBakeSkinning` escribe los puntos posados como muestras de tiempo en la *capa de sesión*, sobre `GfInterval(t, t)`. Esto cuesta una pose, deja intacto el fichero en disco y deja sin tocar un stage sin `SkelRoot`. Prueba: un cuadrado movido por una articulación que se desliza (1.2, 0.4, 0) entre t = 0 y t = 1 se convierte en unos límites, plegados en el dispositivo, que difieren exactamente en ese deslizamiento.
- **Llevada.** Con `--skinned` se lee la pose de **bind** y se omite `UsdSkelBakeSkinning`. Posar primero aplicaría el skinning dos veces a la geometría. Para cada malla, `MeshStage` resuelve su skel binding con un `UsdSkelCache` sobre cada `SkelRoot` y un `UsdSkelSkinningQuery` por prim con skinning. Devuelve las

influencias de las articulaciones, `geomBindTransform`, la ruta del esqueleto y el orden de articulaciones del esqueleto.

Orden de las articulaciones. Una malla puede enumerar su propio subconjunto de articulaciones en `skel:joints`. El mapper de UsdSkel solo traduce del orden del esqueleto al de la malla, así que se aplica sobre la identidad para saber el índice de esqueleto de cada índice de malla. Las influencias se escriben siempre en el orden del esqueleto, de modo que una nube tiene un solo orden de articulaciones, sea cual sea el número de mallas de las que se convirtió. Una malla de un stage con skinning a la que no mueve ninguna articulación (el stage del gorrión lleva un cilindro y un plano junto al pájaro) recibe igualmente un registro de influencias por gaussiana: cuatro articulaciones de peso cero, que el skinner lee como «quédate en la pose de bind». Así el rig y la nube se mantienen alineados uno a uno.

`--skinned` fuerza `--no-bake`. Un horneado de radiancia guarda el entorno y los rebotes de la escena (el suelo bajo una pata), y llevarlo hacia arriba con un miembro levantado equivale a girar un lightmap. Una nube con skinning lleva su material y se reilumina en cada frame. Su transfer de luz, cuando se pide, se guarda de una forma que gira con la gaussiana (§5.7).

5.3 Influencias por gaussiana, y sus gradientes

`mesh_pack.slang` escribe las cuatro influencias más pesadas de cada esquina de triángulo en una segunda imagen con exactamente la misma disposición que la de la malla, para que el efecto AOFX direccione las dos igual. El efecto sabe dónde está cada gaussiana dentro del triángulo, así que mezcla allí las esquinas (`blendInfluences`, `mesh2splat.slang`). Con coordenadas baricéntricas $\lambda_a, \lambda_b, \lambda_c$ y pesos de esquina w_{cj}, el peso bruto de la articulación j es

$$\tilde{w}_j = \sum_{c \in \{a,b,c\}} w_{cj}\,\lambda_c .$$

Entran doce pares (articulación, peso) y salen cuatro. Una articulación ya presente suma el peso, un hueco vacío recibe una articulación nueva y, si no, la nueva sustituye a la más ligera. Elegidas las cuatro articulaciones, cada peso conservado se recalcula como su mezcla completa y los cuatro se renormalizan, $w_k = \tilde{w}_k / W$ con $W = \sum_{i \in K} \tilde{w}_i$. Por tanto, una gaussiana siempre queda llevada por completo.

Gradientes. $\tilde{w}_j$ es lineal sobre el triángulo, con $\nabla\tilde{w}_j = \sum_c w_{cj} \nabla\lambda_c$, donde $\nabla\lambda_b = (e_2 \times n)/|n|^2$, $\nabla\lambda_c = (n \times e_1)/|n|^2$ y $\nabla\lambda_a = -\nabla\lambda_b - \nabla\lambda_c$. El gradiente del peso renormalizado sale de la regla del cociente:

$$\nabla w_k = \frac{\nabla\tilde{w}_k - w_k \sum_{i \in K} \nabla\tilde{w}_i}{W}.$$

Es exacto en el centro de la gaussiana. Sobre un disco solo actúan las dos componentes a lo largo de los ejes de reposo de la gaussiana $(\widehat{u}, \widehat{v})$, y los gradientes suman cero, así que solo se guardan los de las tres primeras articulaciones. Son tres palabras de dos halves, **12 bytes por gaussiana**, en `primvars:athenea:splat:jointWeightGradients` (`half[]`, `elementSize` 6). El cuarto gradiente es menos la suma de los otros tres. Con los gradientes, el registro del efecto para una gaussiana llevada crece a nueve entradas.

Medido. Un cuadrado ligado por entero a una articulación se convierte en 1056 gaussianas, todas con esa articulación y todo el peso. A través del efecto, 272 gaussianas de un quad cuyas esquinas nombran todas la articulación 3 salen en la articulación 3, con los demás huecos vacíos y los pesos sumando uno. Con una sola articulación en todas partes, el gradiente guardado es menor que 10^{-3}.

Truncamiento. Cuatro influencias es un truncamiento. En el gorrión, el 27 % de los vértices del ala tiene más de cuatro (6686 vértices, `elementSize` 25), y una gaussiana toma la unión de las articulaciones de tres esquinas, así que la fracción de gaussianas truncadas es al menos igual de grande. Esa fracción no se ha medido.

5.4 Skinning de una gaussiana en el dispositivo

`scene::SplatSkinner` lanza `shaders/athenea/scene/splat_skin.slang`, un hilo por gaussiana. Sus entradas son la nube de reposo, los pares (articulación, peso), las transformaciones de skinning de las articulaciones en el instante actual y tres matrices: la transformación de bind G, `skelToWorld` y `worldToPrim`. Las matrices de USD son row-major y actúan sobre vectores fila, así que el host las traspone como hace `geom::Skinner` con las mallas.

5.4.1 La media

Sea $q = G\,p$ el punto de reposo en espacio de bind, $X_j q = A_j q + t_j$ la transformación de skinning de la articulación j, y $M = \texttt{worldToPrim} \cdot \texttt{skelToWorld}$. La media posada es linear blend skinning, con exactamente la aritmética que usa el skinner de mallas:

$$\mu' = M \sum_j w_j X_j (G\,\mu).$$

5.4.2 El marco: el Jacobiano entero

La covarianza posada debería ser la covarianza de reposo empujada por la derivada del mapa de posiciones. Cuando los pesos varían en el espacio, esa derivada es

$$J = L(M)\Big[\sum_j w_j A_j\, L(G) \;+\; \sum_j (X_j q)\,\nabla_p w_j^{\mathsf{T}}\Big],$$

donde $L(\cdot)$ es la parte lineal. El primer término es el habitual «mezclar las partes lineales de las articulaciones». El segundo, el término *elástico*, es el que estira una gaussiana a través de un pliegue donde dos articulaciones no coinciden en adónde va su punto. Con dos articulaciones separadas por un ángulo relativo θ y una transición de pesos de longitud L, su tamaño relativo al primer término es de unos $2\tan(\theta/2)\,d/L$, que es del orden de uno en ángulo recto.

El kernel construye el término elástico en el mismo bucle que forma $X_k q$. Acumula $\sum_k (X_k q)\,g_k$ sobre cada eje de reposo, *relativo al punto mezclado*. Los gradientes suman cero, así que es la misma suma, pero se calcula como diferencia de puntos cercanos, lo que se mantiene exacto en float lejos del origen. Una articulación de peso cero pero con gradiente no nulo también contribuye.

La covarianza posada en el plano de la gaussiana es

$$\Sigma' = J\,E\,S^2 E^{\mathsf{T}} J^{\mathsf{T}}, \qquad E = [\widehat{u}\;\widehat{v}],\; S = \operatorname{diag}(s_u, s_v).$$

El kernel toma una base ortonormal (e_1, e_2) del plano generado por $a = J\widehat{u}\,s_u$ y $b = J\widehat{v}\,s_v$, escribe $[a\;b] = [e_1\;e_2]K$ con K triangular superior, y diagonaliza $C = KK^{\mathsf{T}}$ en forma cerrada. El autovalor mayor es $m + r$, con $m = \frac{1}{2}(c_{00} + c_{11})$ y $r = \sqrt{\frac{1}{4}(c_{00} - c_{11})^2 + c_{01}^2}$. El menor se calcula como $(k_{00}k_{11})^2/(m + r)$ en lugar de $m - r$, que en float se cancelaría a nada para una gaussiana fina. Los nuevos ejes son los autovectores y los nuevos tamaños las raíces cuadradas de los autovalores. El primer eje es el autovector más cercano al primer eje llevado, y donde c_{01} es puro redondeo la base se conserva exactamente, para que una gaussiana redonda no gire. El tercer eje es la normal del plano, y el tercer tamaño (el grosor de un disco) queda como se convirtió. La normal de sombreado gira con el cofactor del J entero, porque $(Ja) \times (Jb) = \operatorname{cof}(J)(a \times b)$.

El kernel solo escribe la posición, la rotación y las dos escalas del plano. La opacidad, el color, los armónicos, los canales PBR y la palabra que guarda el tercer tamaño no se tocan. Un frame cuesta, por tanto, un kernel sobre la nube, y no se vuelve a decodificar nada.

5.4.3 Medido

La prueba de la tira, en `tests/scene/test_loading.cpp`, usa 4096 gaussianas sobre una rampa lineal de pesos, dobladas un cuarto de vuelta. Compara la covarianza posada decodificada de cada gaussiana con la analítica $J = (1 - w)I + wR + (Rp - p)\nabla w^{\mathsf{T}}$, y cuenta los errores relativos de Frobenius por encima del 3 %. La tolerancia viene del formato: un cuaternión smallest-three de diez bits no puede fijar un eje con más precisión que unos $\sqrt{2}$ /1023.

caso	solo la mezcla, Gram–Schmidt (antes)	Jacobiano entero, marco propio (después)
doblada, ejes a 0.1 rad de la rampa	2048 de 4096	0
doblada, ejes girados 30°	2048	0
giro rígido y desplazamiento (control)	0	0
una articulación que cizalla, pesos constantes	4096	0
doblada, sin gradientes guardados	2048	1984

Una prueba anterior colocó 4096 gaussianas sobre una hélice, cada una con su propio marco y tamaño. En reposo, cada gaussiana se queda donde estaba. Bajo un giro rígido de 1.2 rad alrededor de un eje inclinado más un desplazamiento, cada gaussiana se lleva rígidamente. Solo se leen cuatro contadores.

En el clip `Flight` del gorrión (61 frames, 1500 triángulos del ala, la malla posada como referencia), el término de mezcla solo se aparta del mapa propio de la malla una mediana del 4–6 %, un percentil 90 del 14–19 % y un 99 del 37–50 %, en la mano del ala y los abanicos de plumas. Con el término elástico, los percentiles 90 y 99 bajan al 5–6.5 % y al 11–13 %. En imágenes con path tracing (1 125 022 gaussianas, 1024×768, frames 5, 31, 36) los dos kernels difieren en unos pocos niveles de 8 bits en cerca de una décima parte de los píxeles (p99 14–16), y las alas se ven iguales. El relMSE de la imagen entera frente a la malla empeora un 2–3 %, pero una ablación muestra que es cosa de la métrica: los gradientes negados, intercambiados, a la mitad y al doble (0.0900, 0.0907, 0.0887, 0.0983 frente a 0.0904) se ordenan por el tamaño del cambio, no por su corrección, porque domina el error de conversión. La corrección se apoya en la prueba de la tira y en un diagnóstico sobre 174 305 gaussianas del ala: entre vecinas con las mismas cuatro articulaciones, los gradientes guardados predicen el cambio de los pesos guardados con un residuo RMS del 12.5 %, frente al 158 % con los ejes intercambiados y el 100 % sin gradiente.

5.5 El fichero: el rig como USD

Una nube con skinning es un `ParticleField3DGaussianSplat`. El rig se escribió primero bajo `AtheneaSplatSkinningAPI`, y ahora se escribe con los nombres propios de UsdSkel. `mesh2splat --skinned` aplica `SkelBindingAPI` y escribe `skel:jointIndices` y `skel:jointWeights` (`elementSize` 4), `skel:geomBindTransform`, `skel:joints` y `skel:skeleton`. También escribe `primvars:athenea:splat:skinningXforms`: una matriz por articulación, en el orden del esqueleto, **muestreada en el tiempo**. Es la única propiedad de una nube animada que cambia de un frame a otro. Las transformaciones son `UsdSkelSkeletonQuery::ComputeSkinningTransforms` en cada instante de `--range START:END[:STEP]`.

Ligado al Skeleton. usdSkelImaging construye ext computations solo para mallas, curvas y puntos. Sí resuelve, en cambio, los `skinningTransforms` de cualquier Skeleton en cualquier instante. `HdAtheneaSkelSplatSceneIndex` (`modules/usd/src/SkelSplat.cpp`) pone esas transformaciones resueltas en la nube como `athenea:splat:skinningXforms`, remapeadas por `UsdSkelAnimMapper` cuando la nube nombra un subconjunto de articulaciones, y reenvía a sus nubes los avisos de cambio de un Skeleton. El primvar que escribe la conversión pasa a ser una **caché**: un stage que compone la nube sin su Skeleton sigue moviéndose, y donde están los dos gana la caché. El skinning por cuaterniones duales (`skel:skinningMethod`) no está implementado para nubes; un prim así se anima por linear blend, con un aviso que se imprime una vez.

Medido. 512 gaussianas ligadas a un Skeleton de una articulación cuya animación pone la articulación en t = 1 donde la caché la tiene en t = 2 dibujan, con la caché bloqueada, el frame t = 2 de la caché en t = 1 píxel a píxel (p99 0, máx. 0). En el gorrión, las 609 articulaciones que recibe el motor a través del Skeleton coinciden con las de la capa de rig; las transformaciones propias de USD frente a la caché escrita por Blender difieren como mucho 5×10^{-4}, y los frames difieren en una media de 8×10^{-6}.

Escala de tiempo. Sin `timeCodesPerSecond`, la nube se leía a 24 bajo un stage de 30 fps y se retrasaba respecto a la malla (error medio 0.031 en el frame 16, 0.049 en el 61); con la tasa escrita, 1.4×10^{-5} y 1.1×10^{-5}.

Clips. El gorrión llega como cinco ficheros FBX con 71 clips. Cada clip solo cambia `skinningXforms` (609 matrices por frame), así que un clip es una capa sobre una nube compartida: 377 MB para los 71, frente a 22 GB para 71 nubes convertidas. `SparrowClips.usda` los ofrece como un **variant set** de USD. La nube entra como payload, porque la opinión local de una subcapa ganaría a la variante, y cada clip entra como referencia dentro de su variante. El motor sigue siendo genérico: lista los variant sets, selecciona uno en la capa de sesión e informa de `animationRange()`, el intervalo de las muestras escritas (por ejemplo 1..25 para `land_hop_1`, 1..33 para `air_fly_A0`). Elegir un clip lee unos 3 MB, no 213 MB.

asset (`mesh2splat --skinned`)	gaussianas	articulaciones	instantes	fichero
tubo con rig, resolución 512	184 320	40	25 / 121	9.27 / 9.50 MB
zorro de Khronos, resolución 512	204 517	24	28	13.35 MB
gorrión, resolución 1400 (primera conversión)	4 991 908	609	33	—

5.6 El motor: pose, límites, reajuste y desenfoque de movimiento

`Engine::carryCloud` empareja los dos arrays del fichero en la disposición (articulación, peso), conserva la nube subida como pose de bind y ejecuta el skinner sobre un `GpuSplats` aparte que comparte los buffers de armónicos y PBR, ya que el skinner no escribe ninguno de los dos. Al medirlo aparecieron dos errores. Primero, la extensión de una nube posada no es la de la pose de bind, así que ahora se vuelve a plegar en el dispositivo después del skinning. Segundo, `Engine::bounds()` plegaba la nube subida en lugar de la dibujada.

Un paso de tiempo no es una nube nueva. Al principio, cada cambio de time code volvía a decodificar todos los arrays y reconstruía en la CPU los pares de influencias (137 MB en el gorrión de 4.27 M gaussianas). Ahora los arrays se comparan por identidad (excepto `skinningXforms`), los pares solo se reconstruyen al subir, y los arrays sin muestras de tiempo se retienen (`_held`), porque los arrays enteros comprimidos del fichero crate llegan en buffers nuevos en cada lectura. Visor, 400 frames a 1280×720:

	antes	después
subidas de la nube en 400 frames	400	**1**
raster, mediana / en caliente	134.96 / 31.56 ms	**56.87** / **13.40 ms**
`rt`, mediana / en caliente	268.66 / 55.31 ms	**178.99** / 55.05 ms

Reajustar, no reconstruir. Una pose no cambia ni el número de partículas ni su orden, así que `GaussianRayTracer::sync` reajusta. En la ruta hardware recalcula los proxies y actualiza en sitio el BLAS de cada trozo. En la ruta de BVH por compute reajusta el árbol una altura cada vez, de abajo arriba, sin lectura de vuelta. Cada `refitsPerRebuild` = 240 reajustes se hace una reconstrucción completa. Frente a reconstruir en cada pose, las imágenes coinciden con p99 0, máx. 1. Un árbol sin reajustar da p99 255. Gorrión a resolución 256 (300 862 gaussianas, BVH por compute): 140.9 / 143.1 / 135.6 ms por frame antes, 84.7 / 83.9 / 83.7 ms después (−40 %).

Desenfoque de movimiento desde el esqueleto. `ParticleField` muestrea `skinningXforms` a lo largo del obturador. El skinner mezcla la segunda pose en el mismo bucle (solo los puntos, cerca de un tercio de kernel) y escribe el desplazamiento de cada gaussiana como dos palabras de halves. El rasterizador convierte ese desplazamiento en un término de covarianza de rango uno. Hydra toma las muestras que abarcan el obturador en lugar de recortarlas, así que el desplazamiento se escala por $(t_c - t_o)/(t_1 - t_0)$; sin ese factor las alas se desenfocan el doble. En el gorrión de 5.9 M a 1280×800, un frame tarda 90.0 ms con obturador 0 y 100.5 ms con obturador 0.5 (+11 %).

5.7 Luz que gira con un miembro: lóbulos zonales de transfer

Una nube estática puede llevar un transfer precalculado de nueve armónicos esféricos en espacio de mundo. Bajo un esqueleto esos armónicos conservarían el cielo de la pose de conversión, y girarlos cuesta una matriz 9×9 por gaussiana. athenea guarda en su lugar uno o dos **lóbulos zonales** por gaussiana, con el eje de cada lóbulo expresado en el marco propio de la gaussiana. Una función zonal gira al girar su eje, una sola 3×3, así que cualquier marco que produzca el skinner arrastra los lóbulos consigo, y no hace falta asignar ninguna gaussiana a una parte.

Un lóbulo de eje a y coeficientes z_l es, en armónicos, $g_{lm} = z_l \, k_l \, Y_{lm}(a)$ con $k_l = \sqrt{4\pi/(2l+1)}$. Para un eje fijo, los mejores coeficientes son la proyección $z_l = k_l \sum_m f_{lm} Y_{lm}(a)$. Por el teorema de adición, el residuo es

$$\| f - g \|^2 \;=\; \| f \|^2 \;-\; \sum_{l=0}^{2} z_l^2,$$

así que el eje se elige para maximizar $\sum_l z_l^2$. En el dispositivo, la búsqueda prueba la dirección de la banda 1, la normal y 64 direcciones de Fibonacci sobre un hemisferio, y después refina con un paseo que divide su paso por dos. El segundo lóbulo se ajusta al residuo del primero, y los dos se reajustan uno contra otro dos veces (el ajuste ZH de Sloan). Los datos son diez floats por gaussiana, `primvars:athenea:splat:transferZonal` (`elementSize` 10): por lóbulo, el (u, v) octaédrico del eje en el marco y z_0, z_1, z_2. En el dispositivo es el buffer `transfer` de siempre con `transferCount` = 10. Al leer, `splatTransferFrame` lleva cada eje al mundo con la rotación actual de la gaussiana y reconstruye los nueve armónicos.

La pose del horneado. Al principio, el horneado posaba la nube en `--time` con cuatro influencias por gaussiana, mientras Hydra animaba la malla con todas las suyas (diez por vértice en las plumas, 25 en las alas). Donde no coincidían, el rayo del horneado no encontraba su superficie, y 566 603 de 1 504 160 gaussianas salieron transparentes. Ahora el horneado mantiene el esqueleto en su pose de bind (`StageRen-`

`derer::holdBindPose`) y posa la nube con transformaciones identidad: 1 504 156 de 1 504 160 encuentran su superficie. El transfer guarda por tanto la autosombra de la pose de bind (alas extendidas). La oclusión que proyecta otro miembro en otra pose es cosa de §5.8.

Comprobado. En `[zonal]`, un transfer de dos lóbulos sobrevive al ajuste y al almacenamiento en f16 dentro del 0.5 %. Girar el marco de una gaussiana por R y leer a lo largo de Rw da lo que la gaussiana sin girar da a lo largo de w. En una prueba USD, 512 gaussianas llevadas por una articulación que gira 60° coinciden con la misma nube bajo un Xform de esa rotación, con relMSE por debajo de 10^{-4} en las rutas raster y trazada. El error del ajuste en el gorrión no se ha medido. La mitad indirecta del transfer no se guarda zonal.

5.8 Campos de visibilidad por partes: horneados una vez, un producto de lecturas de tabla por frame

Una nube reiluminada también tiene que hacerse sombra a sí misma, y las medidas descartaron las rutas obvias. En la ruta trazada `--splat-shadows` no cambiaba nada, porque un splat trazado no tenía pasada de sombra. En la ruta raster costaba 36× el frame (49.58 → 1810.24 ms) y aplastaba el pájaro a negro. Barrer el desplazamiento propio de 1 a 800 sigmas propias movía la imagen media de forma monótona del 0 % al 99 % del valor sin sombra, sin meseta. La literatura coincide en que la visibilidad horneada no sobrevive a la articulación salvo si se factoriza por parte del cuerpo: Lin et al. usan MLP direccionales por parte, y DNF-Avatar una rejilla escalar de AO por hueso. athenea combina las dos ideas: una **tabla direccional por parte**.

Partición. `partitionJoints` toma el subárbol más grande que aún no es una parte, con al menos `minJoints` (6) articulaciones, y lo convierte en parte, hasta que hay `--parts` (12). En el gorrión salen raíz, centro, pecho, pelvis, cabeza, y cada ala en tres segmentos. Cada gaussiana pertenece a la parte de su articulación dominante.

Horneado (`athenea visibility`, `splat_visibility_bake.slang`). Se hace una vez, en la pose de bind. Para la parte p, una rejilla de N^3 sondas cubre la caja de la nube más un margen del 5 %. Cada sonda guarda un mapa octaédrico de O^2 direcciones. Un texel guarda la transmitancia de un rayo que sale de la sonda a través solo de las gaussianas de la parte p. Las gaussianas de la parte se reúnen en una nube propia y se trazan con rayos inline, con el mismo producto de $(1-\alpha)$ que usan los kernels de sombra. Cada sonda guarda además un término ambiente, su media sobre las direcciones, que se usa para las cúpulas.

Lectura (`splat_visibility_read.slang`). En cada frame, `visPartFrames` calcula un mapa afín por parte desde el espacio posado de vuelta al espacio de bind donde vive el campo:

$$x_{\text{rest}} = UA_{k_p}^{-1}\,(x' - t_{k_p}) - Ug, \qquad U = L(G)^{-1},$$

donde k_p es la articulación cabeza de la parte, $X_{k_p} = (A_{k_p}, t_{k_p})$ su transformación de skinning y g la traslación de G; a este mapa se le llama T_p^{-1}. Antes de precalcularlo costaba dos inversas 3×3 por gaussiana, por parte y por luz, unos 260 millones por frame en el gorrión; precalcularlo bajó la lectura de 12.7 a 8.0 ms (5 887 323 gaussianas, 1600×1000). El factor de la gaussiana i para la dirección de luz ω_ℓ es entonces el producto sobre las demás partes:

$$V_{i\ell} = \prod_{p \neq \pi(i)} F_p\Big(T_p^{-1}x_i,\; L(T_p)^{-1}\omega_\ell\Big),$$

donde $\pi(i)$ es la parte propia de la gaussiana, F_p se lee de forma trilineal sobre las sondas y bilineal sobre el mapa octaédrico, y vale 1 fuera de la caja de la parte. Una cúpula usa el término ambiente. Los factores van al mismo `shadowFactors` que leen las dos rutas, así que la ruta trazada, que no tenía ninguna sombra de splats, también los recibe. No se traza ningún rayo por frame.

Hallado al construirlo. Los texels sin escribir de una parte vacía se leían como sombra total y dejaban el pájaro negro; ahora una rejilla de 0 la marca y se salta. El horneado traza rayos inline, que el destino CUDA no tiene, así que es un módulo aparte y los dispositivos CUDA leen campos horneados en otra parte. Una parte no se hace sombra a sí misma: su campo, leído en una gaussiana suya, es la multitud en la que esa gaussiana está. Un mapa de 8×8 (unos 22° por texel) no resuelve el borde de un ala; 16 y 32 son exactos fuera de una banda de una celda de sonda.

Medido en el gorrión: 4 269 858 gaussianas, 609 articulaciones, 12 partes, rejilla 24, octava 16.

horneado (M5 Pro)	22 s, 3 538 944 texels por parte ($24^3 \times 16^2$)
campos en el fichero	81 MB (dos f16 por palabra)
factores por debajo de ½, frame 31, sol + cúpula	21.3 %
visor raster, 400 frames, mediana / en caliente	71.82 / 9.69 ms (56.87 / 13.40 sin ellos)
visor `rt`, mediana / en caliente	193.88 / 51.65 ms (178.99 / 55.05 sin ellos)
L4 (CUDA) raster, 8 frames 1280×720	73.04 ms (67.48 sin ellos)
L4 (CUDA) `rt`	128.77 ms (121.48 sin ellos)

En la L4 los campos cuestan 5–7 ms por frame para 4.27 M gaussianas × 2 luces × 11 partes de consultas. La sombra cae donde cae la de la malla (el ala lejana, el flanco bajo el ala levantada, la raíz de la cola) y sigue a las alas de un frame a otro. Con todas las tarjetas de plumas (9.5 M gaussianas a resolución 1400), el horneado se quedó sin memoria en la parte más grande tanto en el M5 Pro (Metal) como en una L4 a través de Vulkan (un BLAS de 1.17 GB para unos 3.3 M gaussianas). Por eso el pájaro se hornea a resolución 1100.

Límites. Se deja fuera la autosombra local de una parte (pluma sobre pluma). El término ambiente es una media sobre la esfera, no un lóbulo coseno. Se supone que las partes son rígidas y que ocluyen de forma independiente. El suelo bajo el pájaro se sombrea con un mapa aparte en el espacio de la luz, que se estabilizó frente a la «respiración» de una caja posada (dimensionado a partir de la esfera de reposo, encajado en la rejilla de texels del mundo). Una alternativa con rejilla de densidad (propuesta 018) está anotada pero no implementada.

5.9 Niveles de detalle y diezmado de nubes con skinning

El octree de `.athc` fusiona celdas en orden de Morton. Eso no sirve para una nube con skinning, porque una celda que tomara ala y cuerpo juntos no sabría a qué articulación seguir. Dos mecanismos lo sustituyen.

Niveles discretos por reconversión. `mesh2splat --lod-levels N` convierte a la resolución pedida, después a la mitad, y así sucesivamente, cada nivel una nube con su propio rig. `Engine::lodLevelsFor` dibuja el nivel más grueso cuya celda no abarca más de `athenea:lod:threshold` píxeles (1 por defecto) en el punto más cercano de sus límites *posados*. Los niveles no dibujados no se posan (`deferredPose`). Clips del gorrión a 1100 / 550 / 275 celdas (5.9 M / 1.5 M / 0.4 M gaussianas), 1600×1000: 49.6 → 51.5 ms cuan-

do el pájaro llena el frame; **71.3 → 24.7 ms** a cuatro veces la distancia. Todos los niveles quedan residentes con sus propios campos de visibilidad (81 MB cada uno), y el cambio de nivel es un salto, no un fundido.

Diezmado que conserva el rig. `usd::decimateStage` encuentra cada array por gaussiana y fusiona cada uno según su tipo (`lod_attributes.slang`). Cuando `skel:jointIndices` tiene un `skel:jointWeights` a juego, el par se fusiona como rig (modo 3). Sobre el tramo de registros R al que representa una gaussiana conservada, cada uno con peso de fusión ω_r,

$$c_j = \sum_{r \in R} \omega_r \sum_k w_{rk} [\, j_{rk} = j \,],$$

reuniendo hasta 16 articulaciones distintas. Se conservan las `width` más pesadas y se renormalizan, $w'_j = c_j / \sum_{\text{kept}} c$. La articulación dominante de un registro se pliega además en la clave de fusión, así que una fusión nunca cruza el borde de una articulación. Comprobado: un suelo cuyas mitades pertenecen a dos articulaciones pasa de 4096 a 256 gaussianas, cada gaussiana conservada entera en la articulación de su lado. El nivel más fino del gorrión (5 887 323 gaussianas, con ids, material, partes de visibilidad y rig) conserva el 87.7 % en 28 s. Posado en el tiempo 20, se dibuja como la nube sin diezmar con un RMS de imagen de 0.002.

5.10 El gorrión

El gorrión y un BMW fueron los dos primeros assets convertidos que no se crearon para este renderer, y la mayor parte de lo que destapó estaba fuera de las matemáticas del skinning.

- **Exportación de la animación.** El exportador de Blender no escribía muestras de articulaciones para este rig importado de FBX, así que `scripts/sparrow-anim.py` muestrea la pose evaluada. Resolver un cambio de base frente a `restTransforms` destrozaba la cabeza, el pico y los dedos, porque reposo y bind son poses distintas en este pájaro (peor 1.989 en `Toe.L.010`). Ahora escribe $W(t) = P(t)\,L^{-1}B$, de modo que $W(t)B^{-1}$ es la propia matriz de skinning de Blender; lo que queda es cuantización, 4.9×10^{-5}–7.1×10^{-5} en los frames 1, 31, 61.
- **Stage corregido.** `SparrowBird.usda` lleva los arreglos del asset: nada de metálico en plumas ni ojos, un mapa de normales de pluma sin premultiplicar (`UV2_treesparrow_normal_fixed.png`) y el segundo juego de UV (`UVMap_001`) para el alfa de las plumas. Todas las medidas posteriores lo usan.
- **Recortes y presupuesto.** Respetar la `opacity` conectada al alfa de las plumas bajó su presupuesto de 15 378 695 a 5 898 220 gaussianas a resolución 1400. Las mallas por encima del techo de 2 M por pasada se convierten por trozos; con todas las tarjetas, el pájaro tiene 9.5 M gaussianas.
- **Referencia de malla.** Hacer que coincidieran los renders raster y con path tracing de la malla bajó el relMSE del raster frente al trazador de 7.28 a 3.67 (960×540).

Medido nube frente a malla (`--no-bake`, cámara del ala, alfa sin cúpula, relMSE con ella), después de que la cobertura pasara a seguir la opacidad:

	alfa antes → después (malla)	relMSE antes → después
ala del gorrión, raster	0.2395 → 0.2367 (0.2253)	0.1742 → 0.1635
ala del gorrión, trazada	0.2385 → 0.2358 (0.2462)	0.1331 → 0.1247

Para la web, el pájaro se convirtió con `--skinned --transfer --resolution 512`, con lóbulos zonales horneados a 256 caminos y 3 rebotes en la pose de bind y la visibilidad de 12 partes añadida (`sparrow_tx_vis.usdc`: 1 504 160 gaussianas, 1 504 096 con contenido). Frente a la malla con path tracing a 1024

caminos (1024^2, `air_fly_A0`, frame 16), el render raster de la nube da relMSE 0.0025. El cuerpo de la nube sale algo más oscuro y marrón, porque el transfer zonal solo guarda la luz directa.

5.11 El peón: el control rígido

El peón de ajedrez (The Open Chess Set, ASWF, CC BY 4.0) **no tiene esqueleto**. Son dos mallas: un cuerpo de mármol negro `M_Pawn_Body_B` y una cabeza `M_Pawn_Top_B` con `transmission = 1`, una bola de cristal sobre un anillo de oro. Está en esta sección porque es el asset que todo cambio relacionado con la animación tenía que dejar igual:

- Cuando `MeshStageOptions` ganó un tiempo, el peón se convirtió en un fichero idéntico byte a byte al de antes.
- Cuando el eje plano pasó a ser $\min(\text{along}_U, \text{along}_V) \times 0.1$ para corregir el fantasma del zorro bajo el trazador de rayos, el mármol del peón pasó de 0.0644/0.0798/0.0778 a 0.0645/0.0799/0.0780.
- Toda ruta con skinning, con transformaciones identidad, se reduce a la rígida, y el peón es la referencia de la rígida.

Su primera conversión de principio a fin produjo 729 073 gaussianas a resolución 512. Sus resultados de material y transfer pertenecen a las secciones 3 y 4. La validación de alta calidad del peón blanco (`athenea-renders/pawn-hq`, `pawn_mesh.usda`, 2048^2, 2048 caminos de referencia) registra, por material:

material	gaussianas	relMSE	p99	media (nube)	media (malla con path tracing)
`M_Pawn_Body_W`	1 568 499	0.0218	0.273	0.167 / 0.165 / 0.133	0.164 / 0.162 / 0.132
`M_Pawn_Top_W` (cristal)	307 296	0.1418	0.354	0.236 / 0.260 / 0.221	0.365 / 0.397 / 0.279

Tres ejecuciones posteriores a 1024^2 en la misma carpeta (`s49`, `s54`, `s59`) dan un relMSE del cristal de 0.149, 0.081 y 0.043, con el cuerpo sin cambios en 0.025. Lo que cambió entre ellas no está anotado junto a ellas. El cuerpo queda a unos pocos por ciento de la malla con path tracing. El error se concentra en la cabeza de cristal.

5.12 Reproducción en la web y en un teléfono

El reproductor web (`sparkwebGPU`, un fork WebGPU de Spark) reutiliza los kernels de athenea compilados a WGSL. El escritor de `.athc` de athenea rechaza `--skinned`, así que el lado web lee la nube USD (`usd-athc`, sobre el crate `openusd`) y escribe un `.athc` v3. Ese fichero añade una sección `SKIN` por elemento, con las influencias empaquetadas como articulación u16 | peso unorm16 y los gradientes como halves, y un blob de esqueleto `ATSK` con los nombres de las articulaciones, la transformación de bind y el primer clip. Cada uno de los demás clips es un fichero `ATCL` que se descarga al reproducirlo. Los nodos LoD fusionados llevan la mezcla de las influencias de sus splats ponderada por opacidad (las cuatro más pesadas, renormalizadas) y ningún gradiente.

El kernel de skinning (`athenea_adapter/skin.slang`) reescribe `splatSkin` línea a línea, porque el original enlaza once storage buffers y Chrome en un Mac permite diez por etapa. Una prueba de GPU ejecuta el kernel propio de athenea sobre los mismos splats y compara los dos. Frente al raster de athenea, el resultado es un relMSE de 0.0015–0.0053 con una razón de brillo de 1.01. No se escribe movimiento de obturador.

página	contenido	medido
`/pawn` (publicada 2026-10-06)	cuerpo 727 061 + cabeza 673 711 splats, transfer, 6 HDRI CC0	60 fps en Chrome; conjunto de datos v8: descarga 281.3 → 82.7 MB, sin subir; memoria de GPU en escritorio 1127 → 800 MB
`/sparrow` (publicada 2026-10-07)	1 504 096 splats, 125.7 MB, 12 clips (6.4 MB gzip), transfer zonal, HDRI	memoria de GPU en escritorio 1084 → 684 MB
`/sparrow` en teléfonos	`sparrow-mobile`: 488 656 splats, 31.7 MB, cortado del árbol LoD (`--keep-splats 600000`)	60 fps headless en modo móvil, animado
sparrow-v2 (candidato)	1 100 334 splats (−26.8 %), 75.9 MB; móvil 371 138, 26.1 MB	mismo error que el publicado (4.46e-3)

Podar un asset que se mueve. Las gaussianas ocultas en la pose de bind no se pueden quitar sin más, porque otras poses las muestran. Medido sobre 12 clips × 4 muestras = 48 poses más la pose de bind, con 61 vistas cada una: el 61.2 % del gorrión está oculto en la pose de bind, pero solo el **32.5 %** lo está en todas las poses. Quitar lo que oculta la pose de bind cuesta un relMSE de hasta 4.5×10^{-2} en vistas cercanas de poses no muestreadas. Quitar lo que ocultan todas las poses cuesta como mucho 2.9×10^{-4}. sparrow-v2 es lo segundo. Como comparación, el peón estático solo tiene un 2.7 % oculto.

5.13 Lo que no está hecho

- Más de cuatro influencias por gaussiana (P014C). Esto limita la precisión en las puntas de las alas del gorrión, tanto en la animación como en la coincidencia del horneado con la malla.
- El skinning por cuaterniones duales para nubes. El retargeting y la mezcla de clips, que una nube con transformaciones horneadas no puede hacer.
- Varias mallas con skinning con transformaciones de prim distintas (solo se conserva la de la primera), y esqueletos con transformación animada propia.
- La ruta trazada posa una vez por frame, así que no desenfoca una nube con skinning. Las instancias LoD no llevan movimiento.
- El término de autosombra propio de una parte. La oclusión entre partes en poses distintas de la de bind, dentro del transfer zonal. Los horneados de visibilidad por encima de unos 3 M gaussianas por parte.
- El error del ajuste zonal en el gorrión, y una medida geométrica de la cobertura en un pliegue visto de cerca, que es lo que mostraría la ganancia del término elástico del Jacobiano.

6. Niveles de detalle: árboles de fusión, cortes por error, páginas y oclusión

Una nube horneada desde una malla a 1280 px contiene 8.7 millones de gaussianas para un solo coche. El nivel de primer plano, horneado a 5120 px, contiene 74.5 millones. El presupuesto del teléfono, fijado por el usuario, es de 50–60 MB residentes y unas 250 k gaussianas dibujadas a 30 fps. Cada cifra entre esos límites depende de la estructura de niveles de detalle. Esta sección describe primero el nivel de detalle original del motor (§6.1), hecho para escenas capturadas, y explica por qué no cumple ese presupuesto. Después trata lo que lo sustituyó en el trabajo por piezas: árboles de fusión por pares por pieza (§6.2), un único corte por un error de pantalla acotado (§6.3), la opacidad que recibe una fusión (§6.4) y un intento fallido de mejorarla (§6.5), dos descartes dependientes de la vista (§6.6, §6.7), páginas ordenadas por error (§6.8), el nivel de primer plano a 5120 px (§6.9) y las medidas (§6.10). Todo kernel nombrado aquí es un kernel de cómputo en Slang. La CPU solo lee ficheros y lleva la contabilidad (§2).

Notación. Un nodo n es una hoja (una gaussiana horneada) o una fusión. μ_n es su centro y Σ_n su covarianza, con escalas $s_n = (s_x, s_y, s_z)$ y opacidad o_n. par(n) es su padre. τ es el umbral en píxeles, f es la focal en píxeles a distancia unidad (`pixelsPerUnit`) y e es el ojo.

6.1 El nivel de detalle original del motor

Octree por código de Morton. `LodBuilder` (`modules/lod`, `shaders/athenea/lod/lod_common.slang`) ordena los splats por un código de Morton de 30 bits. La celda del octree de un splat en el nivel r son los $3r$ bits altos de su código, así que las celdas de un nivel son tramos de prefijos iguales. Una pasada de fronteras y una suma prefija las encuentran, sin recorrer ningún árbol. Cada celda lleva momentos, siguiendo a Kerbl et al. 2024. Con $w = o \cdot A_2(s)$, donde A_2 es el área de los dos ejes más largos,

$$W = \sum w, \quad S_1 = \sum w\,\mu, \quad S_2 = \sum w\,(\Sigma + \mu\mu^\top), \quad C = \sum w\,c.$$

Los momentos se suman, así que cada nivel más grueso se suma desde sus hijos. Una celda se convierte en una gaussiana con $\mu = S_1/W$, $\Sigma = S_2/W - \mu\mu^\top$ (barridos de Jacobi en el shader), $c = C/W$ y $o = \min(W/A_2, 0.99)$. Los niveles guardados van desde `coarsestLevel` hasta el nivel más profundo cuyo número de celdas es como mucho la mitad del de splats.

Corte. `CutSelector` dibuja un grupo cuando la arista de su celda, proyectada a la distancia más cercana de la celda, es como mucho τ y la proyección de la celda padre supera τ. Una celda hija está dentro de su padre, así que la prueba es monótona a lo largo de cualquier línea de la raíz a una hoja, y cada lugar se dibuja en exactamente un nivel. Lo único que se lee de vuelta es un recuento por nivel y una lectura por instancia y fotograma.

Chunks, `.athc` y streaming. Los splats se cortan en chunks de 65 536 splats consecutivos en orden de Morton. Los niveles fusionados se quedan en el dispositivo, y los chunks se cargan en streaming. El formato `.athc` (`.lrtc` en el árbol de lucabRTrender) tiene una cabecera de 104 B, tablas de niveles y chunks, y bloques alineados a 4 KiB para que un chunk pueda mapearse y traerse por fallo de página por sí solo. `StreamingPool` mapea el fichero. Dos hilos de carga copian los chunks pedidos, los más pedidos primero. Cada ranura se toma de una ranura libre, del chunk pedido hace más tiempo o de un chunk pedido menos de la mitad. Un grupo del nivel más fino al que le faltan chunks dibuja su propia gaussiana fusionada, así que la imagen nunca tiene un agujero. En train_7k (742 k splats, M5 Pro), la construcción tardó 47 ms y

produjo 146 k gaussianas fusionadas. A 8 px el corte dibujó 83 k splats (11 %), con un render de 3.3 ms frente a 12.6 ms y un error absoluto medio de $3 \cdot 10^{-3}$. En train_30k, el `.athc` cargado en streaming en 4 de 16 ranuras escribió EXR idénticos byte a byte y se asentó en 2 cortes.

Por qué se sustituyó para el teléfono. La estructura se mapeó el 10 oct (`PIECES-PLAN.md`), y cuatro propiedades no encajaban con el objetivo.

1. El umbral se fija por instancia y no es un presupuesto. Cada instancia cuesta dos envíos bloqueantes y una lectura por fotograma, y no se comparte ningún presupuesto entre instancias.
2. Las celdas del octree ignoran el objeto. Una celda fusiona lo que caiga dentro, incluido un emblema con el panel bajo él y el vidrio con la pintura.
3. Un splat en streaming cuesta 132 B, frente al objetivo de ∼55 B.
4. Solo los splats van en chunks. Los niveles fusionados, hasta uno con hasta la mitad de celdas que splats, están siempre residentes, y para millones de hojas ellos solos superan el presupuesto.

6.2 Árboles de fusión por pares por pieza

`athenea decimate --tree` construye un árbol binario de fusiones por fusión de pares de vecinos más cercanos mutuos. Es NanoGS (Xiong et al. 2026) en el dispositivo (`lod_pairs.slang`, `lod_pairs_final.slang`, `Lod.cpp`), con guardas adecuadas a nubes horneadas con material.

6.2.1 Los nodos como momentos

Un nodo es un vector de 24 floats: W, $W\mu$, $W(\Sigma + \mu\mu^{\top})$ (6), $W \cdot \mathrm{sRGB}(c)$, Wn, $W\alpha$, $W \cdot$ metálico, $W \cdot$ transmisión, $W \cdot$ especular, la clave, el número de registros, la razón de grosor ponderada y el índice del nodo en el árbol. Aquí $w = o \cdot \max(s_x s_y, s_y s_z, s_x s_z)$ (`discArea`), y α es la rugosidad GGX al cuadrado. Una fusión suma los dos vectores. Como las sumas son asociativas, los momentos de un nodo son los de todas sus hojas sea cual sea el orden de las fusiones. Todos los demás atributos (transfer, bits de sombra, material, articulaciones) se fusionan una sola vez al final, ponderados por masa sobre el tramo de hojas del nodo (`lodMergeAttribute`).

La gaussiana fusionada (`pairFinalize`) toma μ y Σ ajustadas por momentos, descompuestas en autovectores por Jacobi con el eje más delgado en tercer lugar. Fusionar por momentos conserva el segundo momento. Un bloque de 2×2 discos tan anchos como su separación se fusiona por tanto en un disco de 0.58 de esa separación, lo que deja agujeros visibles de cerca. A lo largo de cada eje ancho u, la fusión se ensancha así. Sea v_u la varianza ponderada por área de los centros de los miembros, c_u su anchura media y $d^2 = 12\sqrt{v_u v_v}\ /(n-1)$ la separación de una rejilla de n miembros sobre esa dispersión. Entonces

$$s'_u = \mathrm{clamp}\left(\frac{c_u}{d}\sqrt{12v_u + d^2},\ s_u,\ 2s_u\right).$$

La opacidad, con la regla por defecto, es la media de los miembros ponderada por área, limitada a 0.99. La cobertura de masa $W/A(s')$ se guarda como `coverageMass`. Una fusión de discos cuyo eje más delgado es menor que 0.12 de su eje medio se escribe como surfel (escala más delgada exactamente 0).

6.2.2 El coste

Para los nodos a, b con $\pi = W_a/(W_a + W_b)$ y Σ fusionada, el término de forma es la cota de Runnalls de la divergencia KL por unidad de masa. El área del disco sustituye al volumen, que un surfel no tiene:

$$c_{\mathrm{shape}} = \max\left(0,\ \tfrac{1}{2}\left[\log I_2(\Sigma) - \pi \log I_2(\Sigma_a) - (1-\pi)\log I_2(\Sigma_b)\right]\right),$$

donde I_2 es el segundo invariante (la suma de los productos de autovalores dos a dos, calculada sin autovalores). El coste completo es

$$c = \sigma_{\text{px}}\left(c_{\text{shape}} + w_f\,\phi^2 + w_n(1-\cos\theta_{ab})(1+F_0/\alpha)\right) + w_c\left\|\Delta c_{\text{sRGB}}\right\|^2 + w_m\,\Delta_{\text{mat}},$$

donde ϕ es el grosor que la fusión añade sobre su miembro más grueso. El término $\sigma_{\text{px}} = \text{smoothstep}(0.5, 2, \text{px})$ anula los términos de forma de las fusiones que miden menos de medio píxel en la cámara de la conversión. Valores por defecto: $w_c = 20$, $w_n = 1$, $w_m = 4$, $w_f = 4$.

6.2.3 Guardas

Un par cuyos miembros difieren en cualquiera de estas formas cuesta ∞ (`kNoMerge`):

guarda	regla	por defecto
pieza	la clave difiere (prim, articulación, lámina, pared fina, metal Schlick)	siempre
vidrio	transmisión > 0.5 en un solo lado	siempre
vidrio con vidrio	solo con `--merge-glass`	no
espejo	$\alpha < 0.01$ en un solo lado	siempre
pliegue	$\cos\theta_{ab} < \cos 60°$	60°
brillo	$\alpha < 0.1$ y $1-\cos\theta_{ab} > \frac{1}{2}\kappa^2\alpha$	$\kappa = 1$
borde de color	$\left\|\Delta c_{\text{sRGB}}\right\| > 0.15$	0.15
aguja	aspecto del plano por encima de `maxAspect` y de 1.25× el de los miembros	1000 (desactivada)
trazo	fusión más ancha que `maxPixels` en la cámara	no

6.2.4 Pasadas y la cima relajada

En cada pasada, cada nodo calcula el coste de fusionarse con los 8 vecinos a cada lado a lo largo de un orden de Morton de los centros. Después repite esto a lo largo de un segundo orden desplazado media celda fina, y se queda con la pareja más barata. Los pares que se eligieron mutuamente se reparten en un histograma de $\log_{10}$ del coste, y el host elige el umbral de coste que fusiona tantos pares como permite la pasada, que es como mucho la mitad de los nodos. La fusión se detiene en `treeRatio` (4096×) o cuando ya nada puede fusionarse. Con `relaxTop`, la fusión sigue después con el borde de color (2.0), el ángulo, la guarda de vidrio entre sí, el aspecto y el límite de píxeles levantados. Las guardas de clave, de vidrio con no vidrio y de espejo se mantienen. Cada una de esas fusiones guarda su coste **negado**, lo que la marca como fusión más allá de las guardas. Esto da a una vista lejana niveles gruesos a los que cortar: una tapa de faro pasó de 76 486 raíces a 123 (3d7ee66).

Disposición. El árbol se escribe como un ParticleField. Las hojas van primero, en orden de profundidad, de modo que cada nodo es un tramo contiguo de hojas. Las fusiones siguen en el orden en que se hicieron. Cada nodo lleva `lodParent` (una raíz es su propio padre), `lodStart` (su primera hoja), `lodSize` (su número de hojas) y `lodCost`.

6.2.5 Muchas piezas: `--passes`, `--merge-glass`, `--object-groups`

En el Corvette entero (5.38 M hojas), construir el árbol de una vez funcionó, pero escribir sus atributos agotó la memoria del dispositivo. `--object-groups K --object-group i` conserva las gaussianas cuyo `cryptoObject` módulo K es i. Las fusiones nunca cruzan piezas, así que el árbol de cada grupo es exactamente el subárbol que el asset entero tendría para esas piezas. Cada pasada fusiona como mucho la mitad

de lo que queda, y un grupo de muchas piezas pequeñas fusiona pocos pares a la vez. El tope por defecto de 60 pasadas dejó un grupo con 116 549 raíces de 323 613 hojas. Con `--passes 1000` llegó a 1 298 raíces en 310 pasadas (94fa9f1). `--tree-report` mostró dos causas de las raíces restantes: el vidrio macizo nunca se fusionaba (raíces igual a hojas), y los neumáticos seguían fusionando un goteo por pasada hasta el tope.

grupo (coche a 640 px)	raíces antes	con `--merge-glass --passes 4000`
grupo 2	190 490	4 237
grupo 5	36 508	817
coche entero (8 grupos)	—	17 133 raíces, 5.38 M hojas, 3 582 páginas

6.3 El corte por error, con cotas

El primer corte (`--cut-pixels`, 655c027) conservaba un nodo cuando su anchura en pantalla $2\max(s)f/d$ era como mucho τ y la de ningún antepasado lo era. Las fusiones más allá de las guardas solo pasaban a 0.25τ. Ese corte sigue disponible, pero lo sustituyó un corte por error (2bb1b80, e13b08b; investigación 100, a partir de Nanite).

Error propio. Sea $r_n = \left\|s_n\right\|$ (la raíz cuadrada de $\operatorname{tr}\Sigma_n$) y $\alpha_n = \max(\text{rugosidad}, 0.02)^2$. El error propio de una fusión, en metros, es

$$\varepsilon_0(n) = r_n\Big(\,|\text{cost}_n| + w_c\, s_c(n) + w_n\,\frac{\theta(n)^2}{\alpha_n} + w_s \max(E_n - E_0, 0)\Big), \qquad w_c = 10,\ w_n = 1,$$

y $\varepsilon_0 = 0$ para una hoja. Las fusiones más allá de las guardas ($\text{cost}_n < 0$) no se miden por su coste, que no ve el moiré que causan en el vidrio ni el color que arrastran a través de un borde. Por eso se acotan a

$$\varepsilon_0(n) \leftarrow \max\big(g\,\varepsilon_0(n),\ 2r_n\big), \qquad g = 4\ (\texttt{-cut-guard}).$$

El término de derrame w_s es 0 por defecto (§6.5).

Cotas. El coste de una fusión solo cubre el último par que unió. Una cadena de fusiones, cada una dentro del borde de color, puede derivar a través de ese borde. Por eso una dispersión de color s_c y un cono de normales θ se suben por el árbol con la desigualdad triangular (`treeBoundsUp`), empezando en 0 en las hojas:

$$s_c(p) = \max_{k\,\in\,\text{ch}(p)}\big(\,s_c(k) + \left\|c_k - c_p\right\|\big), \qquad \theta(p) = \max_k\big(\,\theta(k) + \angle(n_k, n_p)\big).$$

Ambas son por tanto cotas sobre cada hoja bajo p.

Error monótono y esferas que cubren. El error se hace monótono, y cada nodo recibe una esfera que contiene las esferas de sus hijos:

$$\varepsilon(n) = \max\big(\varepsilon_0(n),\ \max_k \varepsilon(k)\big), \qquad \rho(n) = \max_k\big(\,\left\|\mu_n - \mu_k\right\| + \rho(k)\big), \quad \rho(\text{leaf}) = 2\max(s).$$

Cada subida es un kernel lanzado sobre todos los nodos, en el que cada nodo eleva a su padre con `InterlockedMax` sobre los bits del float. El kernel se relanza hasta que un contador lee cero, lo que da la profundidad del árbol en rondas. El error se proyecta en el punto más cercano de la esfera:

$$P(n) = \frac{\varepsilon(n)\,f}{\max(\left\|\mu_n - e\right\| - \rho(n),\ 10^{-3})}.$$

Proposición. Si $P(n) \le \tau < P(\mathrm{par}(n))$ selecciona n (a una raíz le basta la primera desigualdad), entonces cada hoja se dibuja exactamente una vez. *Demostración.* ε no decrece al subir. La esfera del padre contiene la del hijo, así que la distancia a la esfera decrece al subir. Por tanto P no decrece a lo largo de ningún camino de una hoja a la raíz. Las hojas tienen $P = 0 \le \tau$, así que el predicado «$P \le \tau$» se cumple en un prefijo del camino que empieza en la hoja, y exactamente un nodo es el último de ese prefijo. □

Así que el corte es una sola prueba paralela por nodo (`treeErrorKeep`), sin subir por los antepasados. `--cut-budget N` biseca $\log\tau$ en $[10^{-4}, 10^{4}]$ en 40 pasos para hallar el umbral más pequeño cuyo corte tiene como mucho N nodos.

prueba	escena	corte por tamaño	corte por error
T15	seis piezas, lejos, 393 px, 0.25 px	83 k, 34.4 dB	34 k, 34.9 dB
T18	seis piezas, cerca, las 426 k de investigación	42.92 / 31.22 dB (fotograma / rejilla)	43.61 / 34.50 dB (prototipo 43.61 / 34.47)

6.4 La opacidad de una fusión: cobertura por mínimos cuadrados

La masa sobre la elipse de momentos cuenta dos veces las colas solapadas de las hojas. Investigación midió el efecto en un 27 % en la mediana y 2.3× en el p10. En la rejilla a 393 px, las fusiones salían más oscuras que sus hojas. `--cut-coverage` (0ab577e) y después `--tree-coverage` (73c0ec2, f980e42) fijan la opacidad de cada fusión a partir de lo que sus hojas cubren de verdad, así:

1. El plano del nodo lo generan sus dos ejes más anchos u, v, con escalas s_u, s_v. Una rejilla de 8×8 cubre el núcleo $[-1.25, 1.25]^2$ en unidades de (s_u, s_v).
2. Cada hoja l bajo el nodo (su tramo, hasta 4096 hojas al escribir el árbol y 2048 en un corte) se proyecta sobre el plano. Su covarianza en el plano es $(u^\top \Sigma_l u,\ u^\top \Sigma_l v,\ v^\top \Sigma_l v)$ y el desplazamiento de su centro es m_l. Un tramo más largo conserva su opacidad basada en la masa. Las hojas se componen como lo haría el rasterizador: $C(x) = 1 - \prod_l \left(1 - \min(o_l, 0.99)\, G_l(x)\right)$.
3. La huella propia del nodo en la rejilla es $G(x) = \exp(-\frac{1}{2}\|x\|^2)$. La opacidad es el ajuste por mínimos cuadrados de aG a C:

$$a^\star = \frac{\sum_x G(x)\, C(x)}{\sum_x G(x)^2}, \qquad o_n = \mathrm{clamp}(a^\star, 0, 0.99).$$

Se midieron dos alternativas antes de quedarse con esta forma. Normalizar por $\sum G$ en vez de $\sum G^2$ daba 0.5 para una fusión tan ancha como una hoja opaca. Un ajuste sobre $\pm 2\sigma$ en vez de $\pm 1.25\sigma$ daba 0.84 a las fusiones de un panel, porque el borde es donde los vecinos llevan la superficie, y un guardabarros salía moteado. Las hojas de canto respecto al plano, como los alambres de una rejilla vistos de frente, no cubren nada, así que la fusión va a 0. Un cargador descarta todo lo que esté por debajo de 1/255, lo que rompería el árbol, así que una fusión se escribe como mínimo a

$o_n \ge 2/255$ (`treeCoverageFloor`).

El árbol se reescribe en su propio orden de fichero. Solo cambian las opacidades de las fusiones (`treeCoverageToFile`), porque `lodParent`/`lodStart` son índices del fichero y una reescritura anterior a través del almacén en orden de Morton los había revuelto. Investigación encontró un error: el ajuste leía los ejes de un Blob como columnas, la rotación inversa, y así giraba mal cada gaussiana. Las fusiones de un guardabarros salían a 0.59 de su masa donde el ajuste de investigación daba 1.00 (4dce865).

prueba	qué	cobertura	masa
T20	rejilla, 393 px, corte al ajustar (48 k)	39.79 dB (desde 33.87)	— ; hojas 41.59
T21	rejilla, 393 px, árbol reajustado (49.4 k)	41.28 dB	39.79 con 48.1 k (`--cut-coverage`)
T24f	seis piezas cerca a 1920, 1 px, cotas	40.96 fotograma / 32.09 rejilla	árbol antiguo 41.11 / 30.09
T47	seis piezas cerca a 1920 (tras corregir los ejes)	**43.86**	42.55
T47	seis piezas lejos a 393	39.95	39.66
T47	rejilla cerca a 393	39.71	**41.16**

La rejilla es la excepción. Las fusiones vecinas se solapan y se componen como $1-\prod$, así que un poco más de opacidad oscurece todo el parche. El reproductor web (Spark) dibuja la masa de cada nodo ($\min(\text{coverageMass}, 0.99)$), y nuestro rasterizador conserva la cobertura.

6.5 La métrica de derrame, y por qué falló en la rejilla

En un corte grueso los agujeros de la rejilla se cerraban: en T66 la región de suelo cercano leía 2.24× el path trace en el corte de 0.25 px, frente a 1.18 en las hojas. Investigación lo atribuyó a fusiones de dos hojas de las tiras subpíxel de `Chrome_Part_Grill`, vistas de frente y de 3–6 px de ancho a $\tau = 0.25$. Su coste es $\sim$0 porque la cobertura de un par, medida en su propio plano, no cambia. Se definió un derrame sobre la propia rejilla del ajuste de cobertura (1bb79f4):

$$E_n = \frac{\sum_x \max\big(aG(x) - C(x),\, 0\big)}{\sum_x aG(x)} \in [0, 1],$$

es decir, la parte de lo que pinta la fusión que sus hojas no pintan. Coincidió con el numpy de investigación con una correlación de 0.994, y se escribe como `lodSpill`. El corte suma $r\, w_s \max(E - E_0, 0)$ al error propio.

T67, grupo 0, 0.25 px	nodos	recorte del coche	fotograma
$w_s = 0$	307 665	16.55 dB	22.18 dB
$w_s = 2$	561 242 (+82 %)	16.79	22.38
$w_s = 5$	599 983	16.79	22.38
$w_s = 10$	612 963 (+99 %)	16.79	22.38

El derrame sí cerró los agujeros: de 1.90 a 1.09 de las hojas en el render, y de 4.66× a 1.0 en el indicador de investigación. El precio fue un 82–99 % más de nodos. Toda fusión derrama alrededor de 0.16, y las fusiones que llenan los agujeros no derraman más que las de un panel, así que ningún suelo E_0 separa los dos grupos. El agujero es una propiedad de la **proyección**: la fusión no cubre en 3D nada que sus vecinas no cubran, y el hueco solo se abre donde las huellas en pantalla de las tiras se separan. Ninguna medida por nodo independiente de la vista lo ve. El valor por defecto es $w_s = 0$. El horneado a 1280 px (§6.10) resolvió el problema en los datos: los agujeros de la rejilla en el corte pasaron de 2.24 a 1.04.

6.6 Descarte de caras traseras en cascos cerrados

Al tamaño del teléfono, un corte mostraba las caras traseras en negro. En la rejilla a 393 px, descartar las caras traseras subió las hojas sin cortar de 38.2 a 41.6 dB (investigación). `--cut-cull-back` descarta un nodo cuyo cono de normales entero mira hacia atrás:

$$\theta(n) < \frac{\pi}{2} \;\wedge\; n_n \cdot \frac{e - \mu_n}{\left\| e - \mu_n \right\|} < -\sin\theta(n).$$

El cono acota cada hoja bajo el nodo (§6.3), así que todo el subárbol del nodo mira también hacia atrás. Dos observaciones fijan dónde se aplica. Aplicado a las seis piezas de prueba, con la tapa de vidrio del faro entre ellas, costó 6 dB a 1920: el vidrio se ve a través, así que sus caras traseras son visibles. En el coche entero a 393 px (T50), las puertas y paneles, que son dos pieles, mostraron una red de grietas oscuras: las fusiones de la piel interior, mirando hacia atrás e iluminadas por el interior oscuro, se ordenaban por parches delante de la piel exterior (T51: lo mismo con o sin cobertura; `--cut-guard 1000` no cambió nada). En un casco cerrado, lo que hay detrás de una cara que mira hacia atrás es el interior de la propia pieza, así que descartarla es seguro y elimina la red (T56). Por eso el descarte se aplica, a hojas y fusiones por igual, solo en **cascos cerrados que no son vidrio** (`primvars:athenea:splat:closed`, transmisión ≤ 0.5).

La regla de aristas abiertas. `mesh2splat` cuenta las aristas no compartidas por exactamente dos triángulos, O. Primero marcó una malla como cerrada cuando $O = 0$ (40a7927) y después por debajo del 2 % de sus aristas (2b05ebb, 111 de 192 mallas). El borde de una lámina abierta crece como la raíz cuadrada de su área, así que un tubo fino pasaba la regla de proporción: el barril de una llanta, al 1 %, se descartaba, y en el teléfono se veía luz entre los radios. La regla que se mantiene es

$$\text{closed} \Leftrightarrow O < 2\sqrt{E}\ , \qquad E = 1.5\,T,$$

para una malla de T triángulos (f52b6b4). Con ella, las puertas (0), los logos (1.4) y las pinzas de freno (1.5) son cerradas, y las llantas (4.0), los neumáticos (3.1), el capó (3.1) y el parachoques (4.4) no, lo que da 102 de las 192 mallas del Corvette. T66 (todas las hojas, 1920 px, frente al path trace): el descarte sube toda la puerta un ∼12 % de forma uniforme (puerta 1.01, suelo cercano 1.02 del path trace), con 4.35 M → 3.70 M gaussianas y el recorte del coche 16.73 → 16.75 dB.

6.7 Máscaras de oclusión

Un transfer TX guarda 256 bits de sombra por gaussiana, uno por celda de una rejilla octaédrica de 16×16 sobre la esfera del mundo, activado donde un rayo desde la gaussiana llegó al cielo. Una dirección cuyas celdas están todas cerradas es una desde la que un ojo fuera del asset no puede ver la gaussiana. Los bits del sombreado no pueden reutilizarse para esto: una fusión guarda los bits de sus hojas por mayoría, y el codebook del teléfono (`shadow_vq`) también es una mayoría, así que ambos cerrarían direcciones desde las que se ve alguna hoja. `--occlusion-mask N` (1000bdf, `occlusion_mask.slang`, `Occlusion.cpp`) construye una máscara conservadora por nodo.

Máscaras. La máscara M_l de una hoja son sus celdas abiertas dilatadas una celda (3×3), reflejadas a través del borde del cuadrado octaédrico en el pliegue. Una celda son ∼11°, y sin el margen el borde de un oclusor cerraba direcciones desde las que el cromo de una rejilla aún se veía. La máscara está toda abierta en el vidrio (transmisión > 0.5, porque lo que hay detrás del vidrio se ve a través de él) y en una hoja sin bits. La máscara de una fusión es el OR de las de sus hijos, subido hasta que ninguna máscara cambia:

$$M_p = \bigvee_{k \,\in\, \mathrm{ch}(p)} M_k.$$

Codebook conservador. Las máscaras se sustituyen por un índice a N entradas de 256 bits cada una. La entrada 0 está toda abierta. El **valor** de un nodo es el número de celdas cerradas entre las *relevantes*, las direcciones con $d \cdot \widehat{up} > -0.25$ (un ojo hasta ~15° bajo el horizonte). Los nodos de los intervalos de menor $\log_2$-valor, que juntos suman menos del 2 % del valor total, toman la entrada 0 y quedan fuera del ajuste. Las demás entradas se siembran con nodos ajustados repartidos por el fichero. La ronda 1 asigna cada nodo a la entrada más cercana en distancia de Hamming sobre las celdas relevantes. Las rondas 2–8 asignan un nodo solo a una entrada que **contiene** su conjunto abierto, eligiendo la que tiene menos celdas relevantes abiertas. Tras cada ronda una entrada pasa a ser el OR de sus miembros. Cada entrada es por tanto un superconjunto del conjunto abierto de cada miembro, y el codebook nunca puede cerrar una dirección que un nodo tiene abierta. El coste del codebook son las celdas que reabre ($M_{\text{entry}} \wedge \neg M_n$) como fracción de todas las celdas cerradas.

Repartido por necesidad. Los árboles de un asset comparten un codebook (`--occlusion-group`, 5be8710). Cada unidad se ajusta sola al tamaño completo y se cuentan las entradas que usa. Después el codebook se reparte como $\text{share}_u = 16 + \lfloor \text{spare} \cdot \text{used}_u / \sum \text{used} \rfloor$, y cada unidad se reajusta dentro de su parte (1103f9f). Repartir por calidad de ajuste (T41) y por pieza (`--occlusion-by-part`, 0fde773) fue peor en ambos casos:

prueba	árboles	codebook	reabierto	descarte frente al ideal
T27	6 piezas, un codebook cada una	256	3.2–10 %	—
T30	6 piezas, un ajuste sobre todo	4096 / 1024	3.2 % / 6.5 %	—
T32	+ valor, solo celdas relevantes	4096 / 1024	21.9 % / 24.7 %	—
T39	valor = celdas cerradas, no hojas	4096	14.7 %	—
T40	repartido por necesidad	4096	7.2 % (rejilla 2.8 %, 1 249 entradas)	86 % cerca / 82 % lejos
coche 640	8 grupos por necesidad	4096	16.2 %	—
coche 640	8 grupos por necesidad	8192	11.9 %	84 %
coche 640	por pieza (192)	4096 / 8192	—	66 % / 74 %
coche 1280	8 grupos por necesidad	8187	12.3 %	—
coche 5120	24 árboles por necesidad	8182	0–23.6 % por árbol	—

(Desde T32, el ajuste cuenta solo las celdas relevantes, así que el total reabierto sobre *todas* las celdas sube. Esos porcentajes no son comparables con T27–T30.) En el nivel de primer plano, el total informó «132.4 %» porque la suma de celdas cerradas sobre ~150 M nodos desbordó un contador de 32 bits. La cifra de cada árbol y el propio codebook eran correctos, y los totales se suman ahora en 64 bits (4a8f8c2). Los resultados por árbol fueron: Windshield 0.0 % (vidrio, 1 entrada usada), Part_2 5.3 %, Hood 7.0 %, la puerta piloto 7.7 %, los grupos finos 8.9–15.9 % y el grupo de paneles pequeños pan2 23.6 %.

La regla del reproductor (y `occludedFrom` en el corte). Esta regla se aplica solo cuando el ojo está fuera de la caja envolvente del asset. Sea $d = \text{normalize}(e - \mu_n)$ y $q = 16\,\text{oct}(d) - 0.5$. Las cuatro celdas $\lfloor q \rfloor + \{0,1\}^2$, limitadas a $[0,15]^2$, son la huella bilineal de 2×2 hacia el ojo. El nodo se descarta cuando las

cuatro están cerradas en su entrada. Investigación midió -12 % cerca y -8 % lejos en el corte por 0.06 dB, con el coche como oclusor (nunca en una escena parcial). En el coche entero a 393 px, con descarte de caras traseras, la oclusión llevó el corte cercano de 522 931 a 360 333 nodos.

6.8 Páginas por error

Un reproductor no puede tener un árbol entero. Necesita unidades que traiga enteras y que pruebe sin leer sus nodos.

Por nivel falla. Las páginas tomadas como cimas en anchura de subárboles hermanos seguían necesitando 114 de las 263 páginas del capó para un corte lejano. Las cotas conservan nodos pequeños dondequiera que gira un brillo o un color, así que el corte abarca nodos desde hojas sueltas hasta 2^{11} hojas (e562899).

Por error funciona. Un nodo se necesita exactamente cuando es una raíz o el error proyectado de su padre supera τ. Así que la disposición sigue el error del padre (`lod_pages.slang`). La **banda** de un nodo es

$$b(n) = \operatorname{clamp}\big(\lfloor \log_2 \varepsilon(\mathrm{par}(n)) \rfloor + 40,\ 0,\ 63\big), \qquad b(\mathrm{root}) = 63.$$

La clave de ordenación, en el orden propio de las páginas, es (banda más gruesa primero, después primera hoja, después el nodo más grande primero):

$$\mathrm{hi} = (63 - b) \ll 22 \mid \mathrm{bucket} \ll 19 \mid \mathrm{start} \gg 6, \qquad \mathrm{lo} = (\mathrm{start}\ \&\ 63) \ll 26 \mid (2^{26} - 1 - \mathrm{leaves}).$$

Una ordenación radix en la GPU ordena los nodos, y la página p contiene las posiciones ordenadas $[pP, (p+1)P)$. Las páginas cruzan los límites de banda, para que una banda gruesa de pocos nodos no sea una página por sí sola. La banda de un padre nunca es más fina que la de su hijo, su primera hoja nunca es posterior, y cuando ambas empatan el padre es el nodo más grande. Por tanto un padre nunca se coloca después de su hijo. Una disposición anterior recorría al revés las hojas de cada banda y ponía 92 padres después de sus hijos en el capó (e2c2046). `--tree-pages P` escribe, para cada nodo, `lodPage`, y para cada página:

- `lodPageInfo` = (banda más gruesa, nodos, primer nodo en orden de página), y
- `lodPageBounds` = la caja que contiene las esferas de los **padres** (la necesidad de un nodo se mide desde su padre) y el error de padre mayor y menor.

El reproductor proyecta el error mayor desde la caja y carga la página cuando esa proyección supera τ. Los bits de cubeta (`--pages-by-facing`: eje dominante de la normal y su signo) empeoraron la residencia, 4.45× frente a 3.98× con $\tau = 2.5$, y están desactivados.

Residencia. Un recorrido necesita el corte y todos sus antepasados, alrededor de 2× el corte. Las páginas contienen alrededor de 4× el corte sea cual sea su tamaño u orden, porque los nodos necesarios y los descartados comparten cada página mientras que el descarte depende de la vista.

prueba	escena	τ	corte	nodos necesarios	páginas contienen (× corte)
T38/T42	seis piezas, cerca 1920, P 3000	1	244 114	—	2.0–2.6×
T38/T42	seis piezas, lejos 393	—	—	—	2.2–4.9× (capó: 2 páginas)
T60	coche 640, cerca 393, P 3000	2	293 629	647 707 (2.20×)	1 170 000 (3.98×)
T60		2.5	263 918	580 997 (2.20×)	1 050 000 (3.97×)
T60		3	198 702	445 195 (2.24×)	957 000 (4.81×)
T60		6.75	92 987	214 827 (2.31×)	423 000 (4.54×)
T59	igual, $\tau = 2$, P 1000 / 500	2	293 629	—	3.91× / 3.87×

prueba	escena	τ	corte	nodos necesarios	páginas contienen (× corte)
T61	igual, $\tau = 2.5$, P 1000 / 500	2.5	263 918	—	3.77× / 3.60×

`treeCulledMark` marca los nodos que los descartes dejan fuera. El subárbol de un nodo descartado también queda descartado, así que un recorrido se detiene ahí. Eso deja los nodos necesarios en 2.2× (un recorrido dentro de subárboles descartados necesitaba ∼4×), pero las páginas siguen conteniendo 3.6–4.5× (a16da43). Por eso el reproductor del teléfono (v5, sparkwebGPU) guarda el recorrido a 12 B por nodo para lo que se necesita y la carga útil en tramos deflate de 64 para lo que se dibuja. De cerca, con $\tau = 3$, eso supuso ∼60 MB y 226 k dibujadas a 60 fps (emulado).

6.9 El nivel de primer plano

En el iPhone de cerca, el logo y la rejilla seguían blandos. La cámara llegaba a unos 0.5 mm por píxel del teléfono, mientras que las hojas a 1280 px medían 3–4 mm. Una prueba de logo y rejilla horneada desde una cámara cercana a 5120 px mostró nítidos la bandera a cuadros, la pajarita y la malla de la rejilla (108 527 → 404 594 gaussianas). El nivel de primer plano (car5120) vuelve a hornear el coche por grupos:

- **Piezas detalladas.** Las 66 piezas a las que el horneado a 1280 px dio como mucho 1.5 gaussianas por triángulo (emblemas, rejillas, faros, pinzas, retrovisores, ruedas, limpiaparabrisas, molduras), a 5120 px, en 7 grupos (`fine0–6`).
- **Paneles.** Cada pieza a 5120 px, o tan fina como la mantenga bajo un tope de hojas (5 M, luego 4.5 M después de que un árbol de 5.01 M no cupiera), con las pequeñas agrupadas hasta 4 M. Hizo falta `--max-cells 1048576`: a 4736 px, Infinite_Black perdió 359 628 celdas más allá de las 262 144 por triángulo por defecto (agujeros) y se volvió a hornear.

Cada grupo se convierte con todas las demás piezas ocultas, pero el horneado traza el escenario entero, así que una pieza conserva las sombras de las otras. Los árboles son por pieza, así que el nivel de primer plano tiene el mismo formato que car1280: las raíces gruesas se quedan, y los niveles más profundos solo se paginan de cerca.

grupos	hojas	resolución
fine0–6 (66 piezas detalladas)	18 610 627	5120
puerta (piloto)	4 084 170	5120
Bottom	4 897 428	3392
Part_3	4 370 459	4480
Infinite_Black	4 481 738	4736
Panel_under_Door_001, Windshield, Front_Fender, Hood, Front_Bumper, Part_2, Part_23	15 994 566	5120
pan0–4 (paneles pequeños)	17 088 152	5120
Front_Bumper_001	5 008 986	4480
24 árboles	**74 536 126 hojas, 74 328 031 fusiones, 208 095 raíces**	

Los 24 árboles contienen 148 864 157 nodos en 49 634 páginas de 3000, y 87.4 GB en disco. Algunos árboles conservan muchas raíces sueltas (fine5: 29 213; Infinite_Black: 92 773), porque las fusiones nunca cruzan piezas ni guardas, a propósito. El empaquetador del teléfono añade encima una raíz sintética de va-

rios niveles, ya que un fichero contiene unas 3 000 raíces.

Memoria para árboles de 4–5 M hojas (5ed0b6a). El primer grupo, una rueda delantera con 4.45 M hojas y 8.88 M nodos (solo el neumático 2.56 M), construyó su árbol y después agotó la memoria del dispositivo al escribirlo. Se encontraron dos causas.

1. `mergeAttribute` reservaba un búfer de pesos tan grande como cada atributo, aunque solo las articulaciones de un rig lo escriben. Eso eran 1.7 GB junto a cada uno de los arrays de 48 floats de un transfer sobre 8.9 M nodos. Ahora es una palabra en los demás casos.
2. La escritura tenía cada array tres veces en memoria unificada: lo leído, una copia, y los arrays fusionados empezaban como otra copia. Los arrays fusionados empiezan ahora solo desde los metadatos, y la escritura del árbol, el último paso, libera cada array en cuanto está en el dispositivo.

Tras la corrección, una puerta de 4.08 M hojas tuvo un pico de 10.5 GB residentes. Pasaron 4.90 M hojas. El primer intento con Front_Bumper_001 (5.01 M) agotó la memoria, lo que llevó al tope de 4.5 M para los paneles posteriores. Una ejecución posterior lo escribió, con 2.23 GB como la memoria libre más baja bajo `memguard`.

Oclusión escrita en el sitio. Los 24 árboles ocupan 87.4 GB (unos 86 GB era la estimación antes del último árbol), y una copia de cada uno no cabría junto a ellos en disco. `occlusionMasksShared` escribe cada árbol en `<tree>.occlusion.usdc` a su lado y lo renombra sobre el original. El codebook compartido (8 182 entradas) tardó 645 s, con un pico de conjunto residente de 10.2 GB; por árbol reabre del 0 al 23.6 % de las celdas cerradas (§6.7).

6.10 Resultados medidos

Todos los renders son de nuestro rasterizador. «Hojas a 4×» significa todas las hojas dibujadas a 4× de resolución y reducidas por caja, la referencia para cortes lejanos (un horneado a 1920 px dibujado a 393 px se transparenta). «PT» es el path trace de la malla a 1920×1080 desde `/World/Camera`, comparado en el fotograma y en el recorte de 900×420 del coche.

El coche a 640 px al tamaño del teléfono (393 px), frente a las hojas a 4×.

prueba	corte	nodos cerca	dB cerca	nodos lejos	dB lejos
T50	error 1 px, cotas	666 585	33.29	343 541	39.15
T51	igual, árboles antes de la cobertura (masa)	666 585	33.27	—	—
T52	$\tau = 0.5$	930 119	33.17	—	—
T56	+ descarte de caras traseras en cascos cerrados (regla del 2 %)	522 931	33.63	—	—
T57	+ oclusión (8192 repartidas por grupo)	360 333	33.58	195 049	39.53
T58	ambos descartes, regla $O < 2\sqrt{E}$ (102 piezas)	378 028	33.53	—	—

El corte lejano (195 k) cabe en las ∼250 k del teléfono a 30 fps. El corte cercano es ∼1.4× eso.

Frente al path trace (1920 px).

prueba	qué	nodos	recorte del coche	fotograma
T63b	640 px, todas las hojas	5.38 M	16.37	21.92
T63b	error 1 px, ambos descartes	675 760	16.28 (31.91 frente a hojas)	21.80

prueba	qué	nodos	recorte del coche	fotograma
T63c	error 0.25 px, descarte de caras traseras	2 313 541	16.29 (34.01 frente a hojas)	21.81
T64	hojas, lóbulos nítidos sin ensanchar	5.38 M	16.58	22.15
T65 cut_b	0.25 px + correcciones de sombreado	—	16.55	22.18
T66	hojas (0.01 px), sin descarte / con descarte	4.35 M / 3.70 M	16.73 / 16.75	22.40 / 22.43
T67	grupo 0 con derrame $w_s = 2$	561 242 (g0)	16.79	22.38
T71	**horneado a 1280 px**, 0.25 px, descarte de caras traseras	4 998 976	**17.88**	**23.90**
T75	T71 + la separación tubo/pliegue (§4.9.4)	—	17.88	23.91

El nivel de detalle cuesta alrededor de 0.1 dB frente al path trace (T63b: 16.37 → 16.28 con 1/8 de las hojas). El horneado explica el resto: su resolución, sus faros y la nitidez de sus reflejos. El rehorneado a 1280 px (8.69 M hojas, 8.67 M fusiones, 20 723 raíces, 5 789 páginas, la oclusión reabriendo un 12.3 %) ganó 1.3 dB en el coche. Investigación, comparando T71 con cut_b: los agujeros de la rejilla en el corte pasaron de 2.24 a 1.04, los brillos del capó de 3.2 a 2.0 px de desvío, y las juntas de 0.90 a 1.02 del path trace.

6.11 Límites y cuestiones abiertas

- **Agujeros solo de proyección.** Un corte grueso de tiras subpíxel (la rejilla) llena agujeros que ninguna medida por nodo e independiente de la vista puede ver (§6.5). En la práctica se resuelve con hojas más finas, no con el corte.
- **Ponderación de la oclusión.** El codebook se reparte por entradas usadas, no por la frecuencia con que una vista descarta un nodo. Ponderar por una órbita de vistas en el horneado debería subir el descarte por encima del 84 % del ideal.
- **Residencia en ~4×.** La limitan páginas que mezclan nodos necesarios y nodos descartados por la vista. El teléfono separa el recorrido de la carga útil en vez de encoger las páginas.
- **Fusiones de vidrio.** Cada gaussiana de lámina fina se eleva al suelo de lámina de 0.1, así que fusionar en menos láminas más grandes hace variar la transmisión de celda a celda. Mientras tanto Spark declara las fusiones de lámina a 4× su tamaño, y queda pendiente un suelo por píxel en lugar de por gaussiana.
- **Las molduras finas y lejanas** siguen convirtiéndose en trazos gruesos en todos los cortes. Las cotas no ven ese error.
- **Sin portar.** La regla de caras traseras en cascos cerrados y el corte por error viven en `athenea-pieces`. El `CutSelector` del motor principal no ha cambiado.

7. Compresión y el reproductor WebGPU en el iPhone

El objetivo fijado el 10 de octubre de 2026 era el Corvette entero, con su transporte de luz completo, dibujado en un iPhone dentro de una página web y juzgado frente al path trace de la malla. El primer presupuesto era de 50–60 MB de datos residentes, y las piezas pequeñas tenían que verse bien. El reproductor es un fork de Spark con backend WebGPU (`~/luc/sparkwebGPU`, rama `webgpu`, sesión `sparkwebgpu-ce`); athenea escribe las nubes, los árboles de fusión, las páginas y la oclusión. Esta sección describe los formatos, el streaming, el dibujado y el reiluminado en el móvil, y lo que se midió. Todas las cifras proceden del registro de ese repositorio (`CLAUDE.md`), de sus notas de investigación (`research/iphone-cost.md`, `research/iphone-compact.md`, `research/errcut.md`), de su documento de formato (`spark/docs/docs/athc-v3.md`), de sus mensajes de commit, de `athenea-pieces/docs/pieces.md` §4 y de `athenea-renders/pieces/iphone/README.md`.

Aparecen tres tipos de medida, y no son intercambiables:

- **Dawn / M5 Pro**: marcas de tiempo de GPU de Dawn sin interfaz (Metal) en un Apple M5 Pro, a la resolución CSS del móvil (393 × 852) o a 1920 × 1080. Son un sustituto de la GPU del móvil.
- **iPhone emulado**: el sitio publicado en Chrome de escritorio en el mismo M5 Pro, con la página en su modo móvil a 393 × 852. Los tamaños de pool, el corte y la memoria son los del móvil; la velocidad no. Según `pieces.md` §4, las cifras emuladas van más o menos al doble que las del iPhone real, así que los dibujados se juzgan en el dispositivo.
- **iPhone real**: iOS 18.7, Safari 26.6.1, pantalla de 393 × 852 @3 (la clase iPhone 14 Pro–16), canvas de 393 × 695 con pixel ratio 1. Son las pasadas de la página de estrés del 10–11 de octubre y las dos pruebas del usuario del 12 de octubre.

La calidad de imagen se da como PSNR en dB, o como relMSE ($\sum (x-y)^2 / \sum y^2$; cuando el PSNR se escribe como $-10\log_{10}$relMSE se indica). La referencia es el path trace en la `/World/Camera` de athenea (1920 × 1080) o el propio camino sin comprimir del reproductor. En cada caso se dice cuál.

7.1 El contenedor: `.athc` versión 3

El `.athc` v2 de athenea es un array plano de palabras por splat. El reproductor web lo reescribe como **versión 3**, organizada por secciones y tiers. Dentro de cada bloque, los datos que necesita un tier son un prefijo, así que cada par (bloque, tier) es una sola petición HTTP Range:

Tier	Secciones	Contenido
1	`CORE`, `SHRS` y, opcionalmente, `LODS`, `LODE`, `OCCL`, `LODM`, `TREE`, `SKIN`	geometría y color del splat; metadatos de LoD
2	`MATL`	normales, emisión, pbr, lóbulos
3	`SHAD`, `CURV`, `TXDI`, `TXIN`, `TXFD`	bits de direcciones abiertas, curvatura, transfer (directo, indirecto, campo reflejado)
4	`CMPC`	el registro compacto para el móvil (§7.4)

Un navegador ofrece gzip y nada mejor (`DecompressionStream`), así que la palanca sin pérdida es lo que se le da a gzip. La **codificación 1** transpone cada array en planos de bytes: el byte 0 de todos los elementos, luego el byte 1, y así sucesivamente. El byte alto de un f16 o un f32 cambia poco de un splat al siguiente en orden de Morton, así que forma tramos que gzip encuentra. La **codificación 2** hace lo mismo tras sus-

tituir cada carril de 16 bits por su diferencia (con desbordamiento) respecto al elemento anterior. `athc-convert --gzip --planes` elige la codificación más pequeña por sección. Medido en las nubes publicadas: Corvette ligero 68.1 → 60.9 MB (−10.6 %), Corvette HD 288.7 → 248.6 MB (−13.9 %), su pintura −23 %, piezas del peón −17 a −33 %. Los bits de sombra son aleatorios y nunca ganan.

7.2 Compresión del transfer por PCA agrupada

El transfer es el 30–70 % de los bytes de una nube reiluminable, y los transfers vecinos se parecen. Se usan dos codificaciones por PCA agrupada: una codificación con pérdida del fichero (codificación 3) y una base conjunta que el móvil mantiene residente.

7.2.1 Codificación 3 (en el fichero)

Siguiendo a Sloan et al. (2003), una sección de transfer de un bloque se guarda como hasta K clusters (k-means sobre una muestra; $K \le n/128$). Cada cluster c tiene una media μ_c y $m_c \le M$ direcciones principales $\boldsymbol{d}_{c,j}$ en f16. Cada elemento i tiene un id de cluster y m_c coeficientes cuantizados:

$$\hat{\boldsymbol{t}}_i = \mu_{c(i)} + \sum_{j < m_{c(i)}} \left(q_{ij}\, \Delta_{c(i)} \right) \boldsymbol{d}_{c(i),j}, \qquad q_{ij} \in \mathbb{Z},$$

donde cada producto y cada suma se redondean a f32 en ese orden y el resultado se redondea al half más cercano. Eso hace que los decodificadores de Rust/WASM y de TypeScript sean idénticos al bit. El paso Δ_c es proporcional al RMS del cluster, lo que da el mismo error relativo en clusters oscuros que en claros. Los coeficientes son enteros zig-zag de 1–3 bytes, guardados como planos de bytes. m_c es el menor número cuyo truncado consume como mucho el 40 % del presupuesto de error. El presupuesto es relativo por bloque:

$$E_{\text{block}} = \frac{\sum_i \left\| \hat{\boldsymbol{t}}_i - \boldsymbol{t}_i \right\|^2}{\sum_i \left\| \boldsymbol{t}_i \right\|^2} \le E, \qquad E = 10^{-5} \text{ por defecto},$$

y el paso se reduce hasta que se cumple. Como los coeficientes SH del transfer son ortonormales, E es el error L^2 sobre la esfera. Un bloque que no cumple E, o que los planos de bytes guardan más pequeño, conserva sus valores exactos. Medido: las secciones de transfer ocupan entre 4 y 7 veces menos (el TX 112 de la tapa del peón pasó de 157 a 28 MB); un splat reiluminado se mueve un relMSE de 4×10^{-8} a 3×10^{-5}; el set ligero del Corvette pasó de 67.9 a 49.5 MB y la página del peón de 281 a 84 MB.

7.2.2 La base conjunta residente en la GPU

En el móvil el transfer nunca se expande. La CPCA conjunta de athenea (propuestas 094/097) divide los 112 valores TX en tres bloques $b \in \{\text{directo (16)}, \text{indirecto (48)}, \text{reflejado (48)}\}$. Usa C clusters compartidos por los tres bloques y $K = 8$ direcciones por bloque. Una gaussiana guarda un id de cluster c y 3×8 pesos:

$$\boldsymbol{t}_i^{(b)} \;\approx\; \mu_c^{(b)} + \sum_{k=1}^{8} q_{ik}^{(b)}\, \frac{s_{c,k}^{(b)}}{127}\, \boldsymbol{B}_{c,k}^{(b)},$$

donde $\boldsymbol{B}$ son los ejes de cada cluster y s sus escalas. En la GPU, cada dirección se guarda premultiplicada por $s/127$ en f16, 504 palabras por cluster. Los **pesos** son int8 o, en el móvil, int4 (modo 1 de la 097): se codifican a 7 niveles y se decodifican como $q_8 = \text{round}(q_4 \cdot 127/7)$. En el móvil, int4 cuesta $+0.7 \times 10^{-4}$ de relMSE del color por splat y $0.1\text{–}4.6 \times 10^{-5}$ de relMSE de imagen; domina el propio truncado de la CPCA.

La ruta de la propuesta 094. Bajo una sola cúpula, solo el bloque reflejado depende del ojo. En cada reiluminado, un hilo por cluster proyecta la media y los 8 ejes de los bloques directo e indirecto sobre la luz (los 16 armónicos de la cúpula, la dirección del sol, el acoplamiento de cielo blanco): 144 palabras por cluster. Los términos independientes de la vista ("conservados") de una gaussiana son entonces productos escalares de sus pesos con esas proyecciones. En cada frame solo se reconstruyen, en registros, los 48 valores reflejados. Con datos reales esta ruta reproduce la reconstrucción completa con un relMSE de 3×10^{-11}.

Evolución de la base.

base	clusters	ajustada sobre	relMSE del transfer	notas
q97 de athenea	256	su muestra de `base_tx`	1.69e-2 (coche entero)	fichero f32 de 1.17 MB; la asignación de clusters coincide con la suya al 100 %
nuestra, 6 piezas (hilo CR)	256	piezas t48	directo 2.7e-4, indirecto 2.5e-3, reflejado 1.1e-3	dentro de la muestra; piezas excluidas entre 10 y 400 veces peor
nuestra, coche entero (CS)	256	uno de cada dos nodos del coche	1.03e-2 (directo 1.6e-3, indirecto 8.0e-3, reflejado 1.06e-2)	imagen a relMSE 1e-3 del transfer sin comprimir
nuestra, prueba de v8 (CX)	512	car1280 + car5120	fine3_0: 0.0181 → 0.0136	todos los grupos de paneles de 1280, −20 a −30 %

El ajuste (`fit-compact-basis`) normaliza la energía por bloque, aplica k-means (semillas k-means++, Lloyd sobre todos los nodos) a los vectores conjuntos, luego una PCA por cluster de cada bloque, y elige la escala de cada dirección con el menor error a través del código int4. La **base de 512 clusters** mantiene el tamaño del registro: el 9.º bit del id de cluster va en el bit 15 libre del índice del diccionario de sombras, lo que limita el diccionario a 32 768 entradas (el del coche tiene 4096). Los ficheros de 256 clusters se leen igual que antes. Coste: base 516 KB → 1.03 MB (f16), proyecciones 147 → 295 KB, tier 4 +1.6–1.8 %, y el mismo tiempo de shade en el M5 Pro (2.82–2.88 ms con 400k dibujados). Todavía no se ha medido con ella en el móvil real.

7.3 Los bits de direcciones abiertas por cuantización vectorial

Las 16 × 16 celdas de direcciones abiertas de cada gaussiana (256 bits) se sustituyen por un índice a un diccionario (`athc-convert --shadow-vq`). El diccionario es k-means en espacio de Hamming, cada centroide la mayoría bit a bit de sus miembros con los empates abiertos, 12 rondas, con semilla en el miembro $\lfloor c\,n/C \rfloor$. Con 1024 centroides por pieza, el error de imagen es de 62.9–65.4 dB (cifra de athenea: 62 dB), y la distancia de Hamming media es de 2.0–2.5 sobre 256 (p99 14–16). Las máscaras en bruto como diccionario ocuparían 15.3 MB a 1M splats, frente a 1.51 MB con la VQ. Sobre seis piezas, un diccionario de asset de 4096 entradas (131 KB, distancia media 1.07) gana en bytes a seis diccionarios de 1024 (197 KB, distancia media 1.00) con casi el mismo error, así que el diccionario compartido es el de por defecto.

7.4 El registro compacto

El camino compacto (hilo CB) guarda una gaussiana como planos de bytes en un único storage buffer. Cada plano está alineado a 64 bytes y se lee con `fieldAt(plane, width, i)`:

plano	bytes	codificación
posición	6	$3 \times$ u16 en la caja de la página: $x = \ell + q\,(h - \ell)/65535$
albedo	3	u8 sRGB

plano	bytes	codificación
opacidad	1	$q \le 191$: $\alpha = (q/191)^2$; $q \ge 192$: $\alpha = 1 + (q-192)/63$ (opacidad de LoD de Spark, > 1)
escala	3	$s = \exp(\ell + (q-1)\,\delta)$, $\delta = (h-\ell)/254$; $q = 0$ significa exactamente 0 (surfels)
rotación	4	palabra de cuaternión: eje octaédrico de 10+10 bits, ángulo de 12
normal	3	octaédrica, 12 bits por eje
curvatura	3	$s8$: $\kappa = \pm f\,(2^{\lvert q\rvert R/127} - 1)$, log con signo en el rango de la página
material	2	u16 a una tabla exacta (pbr + lóbulos, 4 palabras)
cluster	1	u8 (el 9.º bit en el índice de sombra)
pesos	12	3×8 int4
sombra	2	u16 al diccionario
total en GPU	**40**	

Los planos son los que impusieron los datos. La normal de 8+8 bits de la propuesta 097 movía los reflejos del barniz un relMSE de 5.2×10^{-2}; con 12 bits el error es de 2.7×10^{-4}. Un código lineal de opacidad destruía los alfas de 0.01 de las láminas de cristal. El coche tiene 17–22k materiales distintos, así que un índice u8 no los cabía. En una ablación, posición, opacidad, albedo, escala y curvatura aportan cada una menos de 10^{-6} de error de imagen. Frente al reiluminado completo sobre la misma nube, el color por splat del camino compacto queda a un relMSE de $6.3\text{–}7.3 \times 10^{-4}$. La imagen queda a 40.2–41.9 dB con 0.4–1.0M splats.

Frente a los 308 B por gaussiana del reiluminado anterior, este camino mantiene residentes **42.4 B** (planos más tablas) y **16 B por splat dibujado** (los buffers del sort reutilizados como registros de dibujado). El coste fijo baja de 29 MiB a unos 3 MiB. Con 1M splats son 58.6 MiB, frente a 326 MiB antes (Dawn, tamaños de buffer).

7.5 Páginas por bandas de error y el tier compacto

7.5.1 El corte por error

Cada nodo fusionado n lleva las cotas ε_n (error) y ρ_n (radio de la esfera) en f16 (`LODE`). Con p los píxeles por unidad a distancia unidad, la traversal refina n cuando

$$P_n = \frac{\varepsilon_n\, p}{\max(d_n - \rho_n,\ \text{near})} > \tau.$$

τ está en píxeles. athenea pagina su árbol de fusión por **bandas de error**, como Nanite. Un nodo hace falta exactamente cuando el error proyectado de su padre supera τ, así que cada nodo va a la banda (una octava) del error de su padre. Las bandas se ordenan de gruesa a fina, cada una en orden de hojas, y el orden se trocea en páginas de $P = 3000$ nodos. Una fila de la tabla de páginas guarda una caja alrededor de las esferas de los padres y el error de padre máximo y mínimo. Una página hace falta cuando

$$\frac{\varepsilon^{\max}_{\text{page}}\, p}{\max(\text{dist}(\text{eye}, \text{box}),\ \text{near})} > \tau \quad \text{(la página 0 siempre).}$$

Como la caja acota las esferas de los padres, este test es conservador respecto a la traversal. En las piezas t42, el corte paginado ya asentado fue idéntico al corte con todo residente; lo residente fue 2.27–2.53 veces el corte.

En la traversal se aplican dos culls:

- **Caras traseras.** Un nodo de una pieza cerrada se poda con su subárbol cuando $\boldsymbol{n} \cdot \boldsymbol{v} < -\sin(\theta + \arcsin(\rho/d))$, con θ el semiángulo de su cono de normales.
- **Oclusión.** Con el ojo fuera de la caja del asset, un nodo se poda cuando las cuatro celdas bilineales hacia el ojo $(\mathrm{oct}(\boldsymbol{v}) \cdot 16 - 0.5)$ están todas cerradas en su entrada del codebook de oclusión.

El codebook de athenea para el coche tiene 8192 entradas (el 84 % del ideal, sin falsos positivos), y el de v8 tiene 8182. Con centros cuantizados, la oclusión se prueba sobre la caja de centros posibles (partida por octantes, donde las dos coordenadas octaédricas son monótonas). Sobre los 10.7M nodos del coche, en las cámaras de athenea y 24 ojos aleatorios, esto ocultó 0 nodos que el centro exacto conserva, a costa de conservar como mucho el 0.06 % de los ocultos. Con todos los culls activos, los dibujados bajan un -51 % frente a todos apagados, con 36.9 dB entre las dos imágenes.

7.5.2 El tier compacto y sus versiones

La sección `CMPC` del tier 4 guarda los 40 bytes de GPU más los bytes de la traversal, así que un móvil no lee nada de los tiers 1–3. Las tablas que comparte una pieza (`CMPT`: diccionario, materiales, base) y un asset (`.cmpa`: base, un diccionario, una tabla de materiales, y el codebook de oclusión con la caja del asset; identificado por FNV-1a 64) se leen una sola vez.

`CMPC`	hilo	bytes por nodo (fichero)	traversal	cambio
v1	CN	64 = 40 + 24	las 4 palabras del árbol LoD + las 2 de `LODE`	primeras páginas compactas; paginado = residente bit a bit
v2	CQ	56 = 40 + 16	16 B en el fichero, 14 B en WASM	`CMPT` v2 y la tabla de asset `.cmpa`; mismo corte nodo a nodo; descarga de cerca 23.7 → 16.8 MB, tablas 3.62 → 0.97 MB
v3	CS	64 = 40 + 24	+ palabra de cono, índice de oclusión de 13 bits; 30 B de entrada, 20 B en WASM	back-cull y oclusión activos por defecto; solo se planifican las páginas que pide la traversal
v4	CT	contenedor `CMP4`	registros cuantizados de 12 bytes	carga en **tramos raw-deflate de 64 nodos**, plano tras plano; pool de nodos; tier 4 +10 % (314 → 344 MB)

La **página v4** es un contenedor: una cabecera, el final de cada tramo, los planos de la traversal en gzip, y luego cada tramo de 64 nodos comprimido con raw deflate. Los tramos se pueden pedir, guardar y desalojar uno a uno, así que la CPU guarda la carga de una página comprimida y la GPU solo guarda registros del corte **dibujado**. Lo motivó el censo del pool: con páginas enteras residentes, solo se dibujaba un ∼26 % de los nodos de cada página, y las páginas más pequeñas no ayudaban (residente/corte 4.54× con $P = 3000$, 4.46× con $P = 500$).

7.5.3 Registros de la traversal

El coste por nodo de la traversal en WASM bajó por pasos, todos con el mismo corte:

etapa	bytes por nodo	cómo
`LodSplat` de Spark	24	árbol completo
`CMPC` v2 / v3 en WASM	14 / 20	centro f16 como paso en orden f16 desde la posición compacta (exacto), ε, ρ f16, hijos

etapa	bytes por nodo	cómo
`QuantNode` de v4	12	centro 3×10 bits en la caja de centros de la página; ε 8 y ρ 7 bits de código log sobre el rango de la página; θ 6; eje 5+5; oclusión 13; hijos como posición + delta de página
ronda 3 (CU), por clase	5, 7, 9 u 11	2 bits eligen hoja/fusión × con/sin cono; centros de 9 bits; **índice de 9 bits a la paleta de oclusión de la página** (511: ninguna); un offset de byte cada 32 nodos

El redondeo solo refina más y cullea menos: $\varepsilon, \rho, \theta$ se redondean hacia arriba, el error del eje de una fusión se suma a su θ, y el mayor error de centro de la página se suma a cada esfera. En el coche entero, 1.15M nodos residentes ocupan 10.4 MB como registros por clase, frente a 13.8 MB como palabras de 12 bytes. El presupuesto de páginas supone 8 B por nodo. Otro paso sin pérdida hizo que las fusiones de cada página apunten a la página de sus hijos a través de una **paleta por página** de 9 bits. Eso dejó 436 nodos en forma completa en vez de 181k en el coche de 1280, y quitó un mapa WASM de 2.5 MB.

7.5.4 Raíces sintéticas de varios niveles

Una ranura de página 0 cabe 3072 nodos, pero los bosques car5120 de athenea conservan a propósito miles de raíces sueltas (las fusiones nunca cruzan piezas ni sus guardas): fine5 tiene 29 213 y fine6 21 065. Pasado un fan-out (2048), el packer corta las raíces, en orden de Morton de sus centros, en tramos, cada uno hijo de un nodo sintético, y repite hasta que la página 0 cabe el resto. Cada nodo sintético tiene error infinito (siempre se refina), una esfera alrededor de sus hijos, sin cono ni índice de oclusión, así que el corte es el del bosque plano (probado con 5 ojos × 5 valores de τ). fine5 pasó de 10 ficheros a 3 y fine6 de 8 a 1.

7.6 Streaming y el corte de cada frame

El lector (`CompactBandStreamer`) solo lee las tablas de páginas al abrir. Tras cada traversal:

1. El **conjunto necesario** sale solo de la tabla de páginas de cada árbol, con el ojo en el espacio de objeto del árbol (el giro del coche desde el escenario Z-up de athenea) y con foveación.
2. Las **páginas que faltan** se piden por orden del error que quita una página por cada byte que cuesta; en caso de empate, la más cercana. Las páginas consecutivas se juntan en una sola petición Range (hasta 8 MiB), dentro del hueco libre del pool.
3. El pool **desaloja** las páginas no necesarias desde la banda más fina hacia arriba. La página 0 y cualquier página de la que dependa una residente nunca se desalojan.
4. τ sube hasta que el conjunto necesario cabe (`tauToFit`). La versión actual busca por bisección el τ más bajo que cabe, lo que quitó la oscilación de cerca (4.61–5.76 → 5.16).

Un nodo cuyos hijos no están se dibuja como hoja, o como su padre o sus hijos mientras se decodifica un tramo. Con los tramos de v4, los tramos del propio corte y los de sus padres e hijos se guardan siempre, y el resto se desaloja primero bajo un presupuesto de tramos. También se recorre una cámara extrapolada (2.5 actualizaciones por delante), para precargar sus tramos. Hay hasta 12 peticiones Range en vuelo.

El trabajo se reparte entre el hilo principal, la traversal en WASM y dos workers:

dónde	qué	medido
worker de carga (WASM)	traversal de todos los árboles, con el corte enviado por trozos; pila en vez de heap cuando el presupuesto no limita	Camera, $\tau = 3$: 128 → 40 ms (mismo corte)

dónde	qué	medido
worker de nodos (`NodeCutter`)	corte a ranuras, inflado de tramos (~0.3 μs por nodo), empaquetado de registros, lista de dibujado con sustitutos, precarga	empaquetado JS 32.8 → ~9 ms; máx / p99 en la órbita de 31–35 / 26–31 a 17.5–19.0 / 17.4–18.7 ms
hilo principal	peticiones Range, escritura de registros y listas de dibujado (adoptadas, sin GC)	—
compute de GPU	movimiento de registros cuando el pool encoge (un dispatch, un hilo por palabra)	primer frame tras un frame quieto 88–100 → 34 ms

En una órbita rápida de 120° a 1° por frame en el móvil emulado, la parte del corte que falta en un frame bajó del 26 % (CT) al 4.3–6.4 %, que es el 0.24–0.30 % del área de pantalla. Los frames se mantuvieron a 60 fps con un máximo de 17.3–19.6 ms. El corte se asentó 2.6–3.1 s después de la carga.

7.7 El dibujado

El camino compacto (`CompactRelitPath`, `compact.slang`) ejecuta cinco pases por frame:

1. **Proyección** (un hilo por cluster, solo cuando cambia la luz): las proyecciones de la 094.
2. **Cull** (un hilo por nodo de la lista): test de frustum; una clave de distancia radial de 24 bits y el índice añadidos con un atómico.
3. **Sort**: radix sort en GPU de las claves dibujadas, de atrás adelante (6 pasadas sobre 24 bits).
4. **Shade** (un hilo por ranura dibujada, agrupado por cluster): decodificar, reiluminar (§7.8), proyectar. Se escribe un registro de 16 bytes en los propios buffers del sort: centro como u16 a 1/16 px, σ_1 (half) con el ángulo del eje mayor, σ_2 (half) con α (half; el signo negativo marca opacidad de LoD), y color en RGB9E5.
5. **Dibujado**: quads instanciados, premultiplicados sobre un target rgba16float.

Proyección (EWA). Con W la rotación de la vista, R, S la rotación y la escala del splat, y J el jacobiano de la perspectiva en el centro en espacio de vista (x, y, z):

$$J = \begin{pmatrix} f_x/z & 0 & -f_x x/z^2 \\ 0 & f_y/z & -f_y y/z^2 \end{pmatrix}, \qquad \Sigma_{2D} = JWRS\,(JWRS)^\top = \begin{pmatrix} a & b \\ b & d \end{pmatrix}.$$

Compensación de opacidad. El desenfoque de anti-aliasing suma $\beta = 0.3\ \mathrm{px}^2$ a la diagonal, y la opacidad se escala para conservar la masa del splat:

$$\Sigma' = \Sigma_{2D} + \beta I, \qquad \alpha' = \alpha_0 \sqrt{\det\Sigma_{2D} / \det\Sigma'}\,.$$

Un nodo fusionado con opacidad de LoD $\alpha_0 > 1$ se dibuja con $\alpha_0 \leftarrow \min(4\alpha_0 - 3, 5)$, y su quad se ensancha $0.7(\alpha_0 - 1)$ desviaciones. Un splat tenue ($\alpha' < 4\,\alpha_{\min}$) conserva su masa $\alpha_0\sqrt{\det\Sigma_{2D}}$: se elige su desenfoque para que su opacidad llegue al objetivo tenue, o se dibuja a ese objetivo con probabilidad $\alpha'/(4\alpha_{\min})$ mediante un dither de hash estable. El suelo de 0.1 de una lámina fina de cristal se aplica **después** del desenfoque y se atenúa por su área en pantalla. Aplicado antes del desenfoque, había dejado la tapa del faro 1.63 veces demasiado clara. El fragmento evalúa $\alpha = \alpha' e^{-z^2/2}$ dentro del radio de corte, descarta por debajo de $\alpha_{\min}$ y suma los lóbulos nítidos por píxel (§7.8).

Un hilo por ranura dibujada reutiliza los buffers del sort como registros, así que no hay nada más por splat. El camino compacto no tiene registros relit (84 B), ni registros de píxel (48 B), ni acumulador (32 B), ni splats ext (32 B). En el móvil tampoco tiene textura de profundidad: nadie la lee. Con la vista quieta solo

se repite el dibujado. Escribir directamente en 8 bits no ahorró tiempo de GPU (Apple mezcla en memoria de tile) y cambiaba la imagen hasta unos 100 códigos en el 1–3 % de los canales, así que el target sigue siendo rgba16float.

7.8 Reiluminado en el móvil

El shade evalúa `relitByDome` de athenea (rama TX) para una cúpula. El código es una copia literal con las lecturas del campo tomadas de registros, porque WebGPU no puede enlazar un mismo buffer como lectura y escritura en un dispatch. Los términos conservados salen de las proyecciones; el campo reflejado (16 coeficientes × RGB) se lee a lo largo del espejo. Se portaron o añadieron los siguientes refinamientos, y cada uno se midió.

El campo reflejado. Es el 42 % del shade, y el coste es aritmético, no de memoria. Dos cambios lo reducen: el shade agrupado por cluster (un counting sort de las ranuras por cluster, con histogramas por workgroup), -6 a -9 %; y el campo limitado a grado 1 (4 coeficientes) cuando una huella mide menos de 3 px, -23 % del shade con 215k dibujados. Frente al path trace, el cambio en puerta, umbral, parachoques y capó es menor de 0.01 dB.

Lóbulos nítidos por píxel. Un barniz o un pulido por debajo de los umbrales de rugosidad nítida no se responde una sola vez en el centro. El shade resta la lectura del cielo en el centro y le pasa al dibujado un registro de 5 palabras: la dirección del espejo en el centro (unorm16 × 2), sus pasos un píxel a la derecha y un píxel abajo (pares f16), el peso del barniz y las dos rugosidades como raíces cuadradas, y el peso del pulido en RGB9E5. El fragmento lee entonces el cielo a lo largo de $\mathrm{oct}^{-1}(\boldsymbol{a}_0 + \boldsymbol{s}_x\,\delta x + \boldsymbol{s}_y\,\delta y)$. La rugosidad con que se lee se construye en tres pasos:

- *Desensanchado.* El horneado ensanchó la rugosidad guardada por Toksvig, $r'^4 = r^4 + 2s^2$ con s la dispersión de normales. Como el giro por píxel ya cuenta esa dispersión, el lóbulo lee $r = \max(r'^4 - 2s^2, 0)^{1/4}$.
- *Regla del pliegue* (athenea f9c9a2b). Con $t = 2s$ el giro sobre $\pm 2\sigma$ y $w = \mathrm{smoothstep}(0.7, 1.5, t)$, el giro del píxel se escala por $1 - w$ y el lóbulo se ensancha, $\alpha' = \sqrt{\alpha^2 + w\,(t/2)^2}$, por debajo de 0.5.
- *Dispersión del píxel.* La normal gira $\boldsymbol{d}_x, \boldsymbol{d}_y$ a lo ancho de un píxel. Esa caja de pendientes, el doble para el espejo, ensancha el $\alpha = r^2$ de GGX:

$$\alpha^2 \leftarrow \alpha^2 + \frac{2\left(\left\|\boldsymbol{d}_x\right\|^2 + \left\|\boldsymbol{d}_y\right\|^2\right)}{12} = \alpha^2 + \frac{\left\|\boldsymbol{d}_x\right\|^2 + \left\|\boldsymbol{d}_y\right\|^2}{6}.$$

Sin la dispersión del píxel, v7 mostraba motas brillantes en los pliegues. Con τ 1 / 3, las motas pasaron de 24 / 27 a 6 / 2 en el umbral y de 19 / 25 a 2 / 1 en el pliegue de la puerta; la puerta pasó de 34.21 a 35.53 dB. Un lóbulo cuyo espejo se queda dentro de 2 texels de su nivel de cielo a lo largo de 3σ se lee una vez en el centro (el dibujado -10 a -12 % de cerca, sin cambio frente al path trace dentro de 0.001 dB).

El entorno como atlas filtrable. Al principio el fragmento decodificaba dos niveles de cuatro texels cada uno desde un storage buffer. Ahora los ocho niveles prefiltrados de la cúpula se copian en un único atlas rgba16float: el nivel 0 en el origen, los demás en una columna a su derecha (0.9 MB con un lado base de 256, 3.1 MB con 512). El atlas se muestrea con un sampler bilineal de hardware, dos muestras por lóbulo. La imagen queda a 77 dB de las lecturas del storage. El pase compacto a 393 × 852, $\tau = 3$ (M5 Pro, tiempo de reloj) pasó de 11.8 a 4.0 ms con lóbulos nítidos desde 1 px; a 1920, $\tau = 1$, de 75 a 17.2 ms. El shade también lee sus propios texels del entorno desde el atlas, lo que liberó el buffer del entorno (-2.67 MB).

polishField y coatFold (athenea 35ae3cb). Cada material lleva su propia rugosidad en los bits 26–31 de pbr, sin coste por nodo. Donde esa rugosidad es ≥ 0.3, el pulido consulta las celdas de su propio lóbulo, y el campo reflejado responde lo que cierran (el cociente de la puerta frente al path trace pasó de 0.786 a 0.857). Donde la superficie gira deprisa, el barniz se sujeta: $r_{\text{coat}} = \max(\text{unwidened}(c, s),\ \min(c, 0.3))$. Solo se aplica donde la rugosidad del material es ≥ 0.3. Aplicado en todas partes, iluminaba cromo, cristal y radios de las llantas (ruedas +63 % de relMSE). En el horneado de 1280 el barniz sale a 0, así que el término ya no cambia el coche.

7.9 La sombra de contacto

Un set compacto se dibuja sin el renderer de Spark, así que la sombra de contacto es un pase propio (`ground-shadow.js`), y usa solo gaussianas (sin malla y sin AO). El shadow catcher de athenea son gaussianas negras sobre el plano del suelo, cada una con sus celdas de direcciones abiertas. Se splatean sobre la cúpula antes del coche: EWA con el mismo desenfoque de 0.3 px² y la misma compensación de opacidad, negras con opacidad $\alpha_i = o_i\,\kappa_i$. Negro sobre negro no necesita sort. El oscurecimiento es

$$\kappa_i = \frac{-\ln r_i}{2\pi}, \qquad r_i = \frac{\sum_{\text{celdas abiertas } j} L_{\text{dome}}(\omega_j)\cos\theta_j\,\Delta\Omega_j \;+\; \text{sol a través de las celdas}}{\text{irradiancia de todo el cielo sobre el suelo}},$$

recalculado en la CPU a partir de los 16 armónicos de la cúpula cada vez que cambia la cúpula. El catcher de día de athenea (309k gaussianas a 1.26 cm) cuesta 9.9 MB de GPU dibujado entero, y aclarado a 1 de cada 4 muestra un grano ondulado. En su lugar, `floor_shadow --merge 2` ajusta por momentos grupos de 2×2 vecinos (centro y covarianza ponderados por opacidad × área, masa conservada, celdas por mayoría). El resultado son 78k gaussianas, 0.65 MB en gzip, 2.5 MB de GPU, y es el más cercano al path trace en todas las regiones:

región (factor de sombra)	PT	s65 77k	completo 309k	fusión 2 × 2, 78k
rueda delantera	0.607	0.565	0.568	**0.594**
rueda trasera	0.713	0.617	0.618	**0.638**
suelo delantero	0.211	0.168	0.164	**0.224**
bajo el umbral	0.437	0.390	0.387	**0.430**

Sin la sombra el coche flotaba, y el umbral salía 1.66 veces el path trace: era el suelo de la nave en el infinito visto por el borde del coche. Con la sombra el umbral sale 1.08 (escritorio: 1.07).

7.10 Control por tiempo de frame y el frame quieto

El objetivo de dibujado pasó por cuatro versiones el 12 de octubre:

1. **Objetivo de dibujados** (CU 02666e9). Empezaba en 260k dibujados y bajaba en el acto cuando la mediana de GPU pasaba de 16.5 ms. Subía un 6 % por segundo mientras estaba por debajo de 14 ms.
2. **~227k dibujados** (2dd3a24). Con el ojo quieto, el suelo de τ se buscaba por bisección hasta dibujar unos 227k (el corte a 60 fps de v6).
3. **Suelo de τ por tiempo de frame, más un frame quieto** (f02dc1b, 16.5 ms).
4. **Primero la calidad** (dfdcc17, tras el «aunque empeore el rendimiento quiero calidad» del usuario). Es la versión actual.

En la versión actual, el tiempo de GPU de un frame se mide desde su inicio hasta `onSubmittedWorkDone`. Cada 0.5 s, con $\bar{t}$ la mediana de al menos 8 frames en movimiento y $T = 33$ ms:

$$\tau_{\text{floor}} \leftarrow \begin{cases} \tau_{\text{floor}} \cdot \min\left(1.6,\ \sqrt{\bar{t}/(0.91\,T)}\right) & \bar{t} > T, \\ \tau_{\text{floor}}/1.04 & \bar{t} < 0.85\,T \text{ durante dos periodos,} \\ \tau_{\text{floor}} & \text{en otro caso,} \end{cases} \qquad \tau_{\text{floor}} \in [1.5,\ 8] \text{ px.}$$

La raíz cuadrada sale de que el número de dibujados escala más o menos como $1/\tau^2$.

El frame quieto. Tras 300 ms sin entrada ni cambio de cámara, el corte en movimiento se fija en el pool de nodos, τ baja a 0.5 px y el límite de páginas sube del pool en movimiento (14 MB de traversal) al pool quieto (28 MB). Se dibuja un corte fino cueste lo que cueste, y se vuelve a dibujar según llegan más páginas. Después no se redibuja nada hasta un toque, que trae de vuelta el corte en movimiento fijado en ese mismo frame y desaloja hasta el presupuesto en movimiento a 32 páginas por actualización. Los datos del coche en reposo pueden llegar a unos 100 MB, y a unos 55 MB en movimiento (decisión del usuario). Medido en el móvil emulado, de cerca y quieto: 529k dibujados con τ 1.23 y 95.6 MB de datos del coche (registro). `pieces.md` §4 da 531k con τ 1.22 en 118 ms, simulado a 190 ms por millón, con el primer frame tras un toque en 39–47 ms.

7.11 Contabilidad de memoria

El 12 de octubre athenea fijó la contabilidad: el objetivo de 50–60 MB se aplica solo a los **datos del coche**. Los datos del coche son las páginas y los tramos residentes, la traversal en WASM, el heap y los buffers del worker de nodos, y los registros de la GPU. El **estado de render** se cuenta aparte: registros nítidos, atlas, sombra, targets, sort. Es lo que da la calidad pedida, y una pestaña del iPhone 15 llegó a unos 1.2 GB en las sondas de estrés. La **página** en sí (heap de JS, three.js, cúpulas) es una tercera cifra.

estado	datos	datos del coche	estado de render	página
seis piezas t48 (CR), τ 1, emulado	CMPC v1	—	GPU 53 MB en total (pico 59.6): datos 24.7, fx 9, target + cúpula 7.9, env 2.7, sort 4.6, canvas 3.8	—
coche entero, τ 3, CT	v5	≈60 MB de CPU + GPU (pool de páginas al mismo τ : ≈87)	—	—
coche entero, τ 3, CU ronda 3	v6	Camera 55.1, cerca 59.0, lejos 49.1 (todo junto)		
coche entero, τ 3, tras el atlas	v6	Camera **39.0**, cerca **43.8**, lejos **33.0**	24.1 / 26.0 / 21.7	∼39 → 25 (tanda A) → ∼17 (tanda B)
coche entero, τ 3	v7	31–36	—	—
frame quieto, de cerca	v7	**95.6** (529k, τ 1.23)	—	—

Varias reducciones fueron idénticas al byte en la imagen:

- los floats del cielo del reiluminado se reconstruyen desde el RGBE al subirlo (1.6 MB retenidos en vez de 8 MB para una cúpula de 1k);
- la cúpula visible en rg11b10ufloat en el móvil;
- un único codebook de oclusión compartido (antes, 4.3 MB de arrays de JS);
- los registros nítidos dimensionados a las ranuras dibujadas (-0.3 a -2.2 MB);
- el WASM servido como fichero aparte, con el layout parseado en un worker.

La contabilidad anterior por gaussiana explica por qué hacía falta el camino compacto. El reiluminado previo guardaba 308 B por gaussiana más 29 MiB fijos: 147 / 206 / 323 MiB con 0.4 / 0.6 / 1.0M (BY). El camino compacto guarda 42.4 B + 16 B por splat dibujado + unos 3 MiB.

7.12 Resultados medidos

7.12.1 El iPhone real

Página de estrés, 10–11 de octubre (rampas sintéticas sobre splats reales del Corvette; medianas):

camino	200k	400k	600k	800k	~1M	notas
3DGS plano, fps	58.8	58.8	45.5	34.5	26.3–27	móvil frío; ~37 ms por millón
transfer, camino "by" anterior, fps	38.5	12.3	7.2	5.2	3.7	362–469 MB de GPU
compacto, base sintética, ms (fps)	30 (33.3)	63 (15.9)	101 (9.9)	137 (7.3)	154 (6.5)	20.8–79.8 MB de GPU
compacto, base t16 real, tras CF, ms (fps)	27 (37)	47 (21.3)	70 (14.3)	95 (10.5)	108 (9.3)	35 MB de GPU con 400k

Modelo tras CF: unos 115 ms por millón dibujado + 4 ms, así que 30 fps aguantan unos 250k dibujados y 60 fps unos 110k. El primer segundo de cada escalón corría 2–3 veces más rápido que el ritmo sostenido (reloj o estado térmico), y una pasada hecha tras cuatro pasadas en 20 minutos fue la mitad de rápida a partir de 600k. Para planificar se usan las cifras sostenidas.

Las pruebas del usuario, 12 de octubre (coche entero, datos v7):

prueba	vista	dibujados	τ	fps	observación
primera	por defecto	106k	8.00	53	fluido; el τ 8.00 era la "respiración" vieja por tiempo de frame atascada en su techo (arreglado)
primera	de cerca	160k	—	32	logo y rejilla blandos
segunda	de cerca	—	—	—	«quiero más calidad de cerca»: aún blando

athenea estima unos 190 ms por millón de shade en ese iPhone. Lo blando de cerca se atribuyó a la nube, no al reproductor: el logo horneado a 1280 px sale borroso incluso con todas sus hojas, y nítido a 5120 px (`close/close_mos.png`). De ahí salió el nivel de primer plano car5120, que es v8.

7.12.2 El iPhone emulado

datos	vista	dibujados / residentes	τ	descarga	fps
t48mass-paged (6 piezas)	por defecto	174k / 384k	1.55	25 MB	61
t48-compact-tx, CMPC v1	por defecto	199k / 555k	1	24.6 MB	60
el mismo, CMPC v2 + `.cmpa`	por defecto	199k / 555k	1	17 MB	60
coche entero, v3 (pool de 24 MB, fov = h)	por defecto	147k / 580k	6.75	20 MB	60
coche entero, v5 (CMPC v4)	por defecto	226k / 1016k	3	36 MB leídos	60
coche entero, v7	por defecto	216k / 1318k	2.22	—	60

datos	vista	dibujados / residentes	τ	descarga	fps
coche entero, v7 tras CU ronda 9	por defecto	234k	1.95	—	60
coche entero, v7, frame quieto	de cerca	529k	1.23	—	(un frame)

7.12.3 Frente al path trace, por versión de datos

Las versiones de datos de `car-compact-tx` son distintas de las versiones del formato `CMPC`:

set	contenido
v3	occl8k, CMPC v3
v4	árboles closed2 de athenea
v5	tramos de CMPC v4
v6	árboles mrough de athenea, rugosidad por material, 1.35 GB
v7	car1280, 1.81 GB
v8	car5120: 24 árboles, 74.5M hojas, 148.9M nodos, oclusión compartida de 8182 entradas (§6.9)

Salvo que se indique, las cifras de abajo son PSNR en la cámara de athenea a 1920 × 1080, por las regiones del harness validate-corvette.

versión	coche	puerta	umbral	parachoques	capó	fuente / condición
v6	19.33	33.77	31.87	20.21	14.16	fde7880, camino compacto
v6, desensanchado de Toksvig (CV)	19.68 → 20.58	+1.0	—	—	+1.3	registro; con detalle por píxel
v7 (primera conversión)	+0.5 frente a v6	−2.3	−2.7	—	+0.9	motas en los pliegues
v7 + dispersión del píxel, τ 1	—	**35.53**	**32.36**	relMSE 0.199	14.46	93955a0
v7, valores por defecto actuales	19.03	35.52	32.36	22.28	14.68	94026b7 (campo de grado 1)
v8	—	—	—	—	—	conversión en curso (CX ronda 4); sin medir

La métrica “car crop” de `pieces.md` (recorte de 900 × 420) es otra región. Con ella, el lóbulo nítido por píxel en el camino compacto llevó el coche de 16.49 a 17.46 dB (17.41 con lóbulos desde 4 px), que es el nivel del camino de escritorio. En el móvil, τ 3 frente a τ 1 difiere en 40 dB, así que los defectos que quedan (pelos, rayas, manchas) se atribuyeron a la nube, no al corte. Los pasos sin pérdida se comprobaron frente a la imagen anterior: los registros nítidos en 5 palabras, 60–80 dB; el shade agrupado, 84 dB (ruido entre ejecuciones 78 dB); los términos solo de cristal, 188 dB.

7.13 Limitaciones y trabajo pendiente

- **v8** (car5120 con la base de 512 clusters y el codebook de oclusión compartido de 8182 entradas) se estaba convirtiendo para el reproductor cuando se escribió esto; la parte de athenea (los 24 árboles y su oclusión compartida, §6.9) está completa. Su calidad frente al path trace, su descarga y su comportamiento en el móvil real están sin medir.

- Solo hay tiempos del dispositivo real para las rampas de estrés y las dos pruebas del usuario. La velocidad del móvil emulado no es la del móvil.
- La tira cromada del parachoques delantero sale un 38 % más clara que el path trace. Las lámparas no tienen emisión en el horneado. Los reflejos abiertos de la puerta (1.13) vienen de celdas de 16 × 16 de 11°.
- La traversal compacta no guarda el tamaño de LoD de una fusión, así que el corte combinado tamaño/error no tiene con qué protegerse. El corte por error por sí solo no ganó al corte por tamaño en todas las vistas con estos datos (`research/errcut.md`).

8. Evaluación

Esta sección reúne las medidas de las secciones 2–7 en un solo relato, eje por eje. No introduce cifras nuevas: cada cifra se cita de la sección que se indica a su lado, que a su vez nombra su fuente (`docs/decisions.md`, `docs/pieces.md`, las notas de investigación, los registros de render en `~/luc/athenea-renders/` y el registro del reproductor). Donde dos fuentes dan cifras distintas para la misma magnitud, la §8.10 dice cuál es cuál.

8.1 Marco

Las medidas de escritorio se tomaron en un Apple M5 Pro con Metal; los resultados de Vulkan y CUDA proceden de una NVIDIA L4 (§2.15). El reproductor se midió de tres maneras que no son intercambiables (§7): marcas de tiempo de GPU de Dawn sin ventana en el M5 Pro, el sitio publicado en Chrome de escritorio en su modo móvil a 393 × 852 (el *iPhone emulado*, más o menos el doble de rápido que el real), y un iPhone real (iOS 18.7, Safari 26.6.1, 393 × 852 @3). La referencia del Corvette es el path trace de la malla desde `/World/Camera` a 1920 × 1080 (§1.6). Muchos tiempos se tomaron en una máquina compartida con otros trabajos de GPU; donde una sección lo dice, no son medidas de rendimiento.

8.2 La corrección y la velocidad del motor

Cada comprobación de abajo es un kernel de GPU frente a una forma cerrada, un estimador independiente, MaterialX o Storm.

comprobación	**resultado**	**§**
dos discos semiopacos, raster / referencia / trazador de rayos frente al operador over analítico	p99 0 (mezcla codificada: p99 26)	2.4
planos convertidos de cobertura 0.25 / 0.5 / 0.75, raster a tamaño completo	0.251 / 0.501 / 0.752	2.5.3
paso de obturador 0.03 frente a N instantes	p99 2, cociente de energía 1.0000	2.5.3
trazador de rayos frente a su referencia de orden exacto	p99 ≤ 1, como mucho el 0.05 % de los píxeles por encima de 2	2.15
visibilidad de mallas frente a Storm (Kitchen_set), raster / rayos / bvh	segmentación: 18 / 41 / 36 píxeles distintos de ∼57 k	2.8
grafos MaterialX frente al genglsl del propio MaterialX, 65 536 direcciones	peor $3.2 \cdot 10^{-5}$ en once grafos	2.9
luces frente a formas cerradas; χ^2 frente a `lightPdf`	0 de 8281 píxeles por encima del 2 %; $\lvert z \rvert \leq 1.61$	2.10
path tracer: cáscara emisora cerrada; exponente de convergencia	0 de 3072 por encima de 10^{-4}; -0.499	2.11
OIDN, 16 caminos frente a una referencia de 4096 caminos	relMSE $3.65 \cdot 10^{-4} \rightarrow 7.17 \cdot 10^{-5}$	2.11
batería de pruebas en tres dispositivos (188 casos)	Metal 188 pasan; Vulkan 180 pasan, 8 omitidas; CUDA 96 pasan, 1 falla, 91 omitidas	2.15

carga	**cifra**	**§**
radix sort, 10 M pares de 32 bits	12.4 ms (39.9 ms antes)	2.3
clips del gorrión, 5 887 323 gaussianas a 1600 × 1000, reproduciendo	102 → 49 ms por frame	2.5.3

carga	cifra	§
train_7k (742 k), 1080p: raster por teselas / ComputeBvh / trazador por hardware	13 / 343 / 435 ms	2.6
visibilidad de Kitchen_set a 1920 × 1080: raster / rayos / bvh	31.7 / 19.2 / 29.2 ms	2.8
Kitchen_set iluminado a 1920 × 1080: raster / path tracer (1 camino, 1 rebote)	96.0 / 358.9 ms	2.11
Corvette TX entero (14.7 M gaussianas) a 1920 × 1080, rasterizador de escritorio	158 ms (unos 83 ms para una nube reiluminada de su tamaño)	4.10.2

8.3 La conversión frente al path trace

asset o prueba	medida	resultado	§
cuadrilátero unidad iluminado de frente (píxel de la malla 0.5032)	píxel convertido con $\sigma = 1.0$ / con los valores del material	0.456 / 0.5009	3.4, 3.6
Corvette, 640 px, bordes sujetos frente al control	relMSE / PSNR / SSIM, fotograma de día	0.0887 / 21.93 dB / 0.8567 frente a 0.0902 / 21.86 dB / 0.8554	3.5
shadow catcher del Corvette frente al path trace	bajo el coche / contacto del parachoques / suelo abierto	0.192 / 0.030 / 0.220 frente a 0.182 / 0.043 / 0.217	3.9
Corvette, horneado de 640 → 1280 px	recorte del coche / fotograma	16.55 → 17.88 dB / 22.18 → 23.90 dB	3.11
peón blanco, cuerpo / cabeza de vidrio (2048^2)	relMSE	0.0218 / 0.1418 (ejecuciones posteriores: vidrio 0.149, 0.081, 0.043)	5.11
gorrión (conversión web) frente al path trace de la malla	relMSE, raster	0.0025	5.10
ala del gorrión, raster / trazada	relMSE con la cobertura siguiendo a la opacidad	0.1635 / 0.1247	5.10

8.4 El coche entero a lo largo de la cadena

El mismo fotograma, medido a medida que se añadía cada paso. Filas de la §4.10.3 y la §6.10; los nodos son gaussianas dibujadas.

prueba	horneado	qué	nodos	recorte del coche (dB)	fotograma (dB)
T63b	640 px	todas las hojas, antes de los pasos posteriores de la §4	5.38 M	16.37	21.92
T63b	640 px	corte por error de 1 px, ambos culls	675 760	16.28	21.80
T63c	640 px	corte por error de 0.25 px, cull de caras traseras	2 313 541	16.29	21.81
T64	640 px	hojas, lóbulos nítidos sin ensanchar	5.38 M	16.58	22.15
T65 cut_a	640 px	+ coat sujeto en los pliegues	—	16.51	22.15
T65 cut_b	640 px	+ campo del polish en materiales rugosos	—	16.55	22.18
T66	640 px	hojas (0.01 px), sin cull / con cull	4.35 M / 3.70 M	16.73 / 16.75	22.40 / 22.43
T67	640 px	grupo 0 con derrame $w_s = 2$	561 242 (grupo 0)	16.79	22.38

prueba	horneado	qué	nodos	recorte del coche (dB)	fotograma (dB)
T69 / T70	640 px	giro limitado / pliegue del coat solo en la pintura	—	16.57 / 16.57	22.20 / 22.20
T71	1280 px	corte 0.25 px, cull de caras traseras	4 998 976	**17.88**	**23.90**
T72 / T73	1280 px	sin el límite / polish nítido condicionado	—	17.88 / 17.88	23.91 / 23.90
T74	1280 px	condición + separación tubo/pliegue	—	17.90	23.93
T75	1280 px	separación tubo/pliegue (commiteada)	—	17.88	23.91

De aquí salen tres conclusiones. El nivel de detalle cuesta unos 0.1 dB frente al path trace (T63b, 16.37 → 16.28 con un octavo de las hojas; §6.10). Los pasos de sombreado de la §4 movieron el coche de 640 px unos 0.2–0.4 dB. La resolución del horneado lo movió 1.3 dB en el recorte y 1.7 dB en el fotograma (T65 cut_b → T71). El PSNR sobre un recorte se mueve poco, así que los cocientes por clase de abajo muestran dónde actúan los cambios.

región o clase (media sobre la del path trace)	antes	después	condición	§
puerta, espejo al suelo a < 3 m	0.78	1.02	hojas de t63b → t65 cut_b	4.10.4
puerta, espejo al suelo a 3–30 m	0.69	0.88	t63b → t65 cut_b	4.10.4
puerta, espejo al HDRI (abierto)	0.92	1.13	t63b → t65 cut_b	4.10.4
juntas de la puerta	0.69	1.04	t63b → t65 cut_b	4.10.4
reflejos abiertos de la puerta, contraste local (σ 1.5 / 4 px)	0.35 / 0.53	0.56 / 0.74	t65 cut_b → T71	4.10.4
brillos del capó, desplazamiento p_{50}	3.2 px	2.0 px	t65 cut_b → T71	4.10.5
media del capó	0.89	0.98	640 → 1280 px	4.10.5
huecos de la rejilla en el corte grueso	2.24	1.04	640 → 1280 px	3.11, 6.5
pelos del umbral (píxeles)	987	459	640 → 1280 px	3.11
umbral, bajo el coche (móvil)	1.66	1.08	sin → con la sombra de contacto	7.9
paso alto de la tira cromada / PT	0.41	0.41	T71 → T75 (T73: 1.12)	4.10.6
motas brillantes en el pliegue del umbral	10	1	T71 → T75	4.10.6

material (relMSE frente al path trace)	cromo	metal rugoso	pintura	§
primer transfer	0.536	0.0896	0.705	4.10.1
TX, celdas y campo en todas partes	1.64	0.221	0.393	4.10.1
TX con la condición del polish	0.591	0.0835	0.341	4.10.1
+ coat nítido y polish nítido por píxel	0.545	0.083	0.281	4.10.1

8.5 Niveles de detalle

nivel	hojas	fusiones	raíces	páginas	codebook de oclusión	celdas reabiertas	§
coche a 640 px	5.38 M	—	17 133	3 582	8192	11.9 %	6.2.5, 6.7

nivel	hojas	fusiones	raíces	páginas	codebook de oclusión	celdas reabiertas	§
coche a 1280 px	8.69 M	8.67 M	20 723	5 789	8187	12.3 %	6.7, 6.10
nivel de primer plano a 5120 px	74 536 126	74 328 031	208 095	49 634	8 182	0–23.6 % por árbol	6.7, 6.9

El nivel de primer plano son 24 árboles y 148 864 157 nodos, 87.4 GB en disco, con su oclusión compartida escrita en el sitio (645 s, 10.2 GB de pico residente; §6.9).

prueba	corte, al tamaño del móvil (393 px), frente a las hojas a 4×	nodos de cerca	dB de cerca	nodos de lejos	dB de lejos	§
T50	error 1 px, cotas	666 585	33.29	343 541	39.15	6.10
T56	+ cull de caras traseras de cáscaras cerradas	522 931	33.63	—	—	6.10
T57	+ oclusión	360 333	33.58	195 049	39.53	6.10
T58	ambos culls, regla $O < 2\sqrt{E}$	378 028	33.53	—	—	6.10

El corte de lejos cabe en las ~250 k a 30 fps del móvil, y el de cerca es más o menos 1.4× eso (§6.10). En el reproductor, con todos los culls activos se dibujan un 51 % menos de nodos que con todos apagados, a 36.9 dB entre las dos imágenes, y el codebook de oclusión alcanza el 84 % del ideal (§7.5.1). La residencia es el punto débil: un recorrido necesita unas 2.2× el corte, y las páginas tienen 3.6–4.5× el corte sea cual sea su tamaño (§6.8), y por eso el móvil separa el recorrido de la carga útil (§7.5.2).

8.6 Compresión

paso	antes	después	coste en la imagen	§
planos de bytes (sin pérdida), Corvette ligero / HD	68.1 / 288.7 MB	60.9 / 248.6 MB	ninguno	7.1
codificación 3 del transfer (CPCA en el fichero), set ligero del Corvette / página del peón	67.9 / 281 MB	49.5 / 84 MB	relMSE por splat 4×10^{-8}–3×10^{-5}	7.2.1
CPCA conjunto en la GPU, base del coche entero (256 clusters)	112 valores	1 id + 24 pesos int4	relMSE del transfer $1.03{\cdot}10^{-2}$; imagen a relMSE 10^{-3}	7.2.2
base de 512 clusters (prueba v8), fine3_0	0.0181	0.0136	el mismo tiempo de shade en el M5 Pro	7.2.2
bits de direcciones abiertas por VQ, 1024 centroides por pieza	256 bits	índice u16	62.9–65.4 dB	7.3
registro compacto (residente)	308 B por gaussiana + 29 MiB	42.4 B + 16 B por splat dibujado + ~3 MiB	40.2–41.9 dB frente al reiluminado completo	7.4
registro de recorrido	24 B (Spark)	5–11 B por clase	el mismo corte	7.5.3

8.7 El móvil

Frente al path trace, por versión de datos (regiones de validate-corvette, PSNR en dB a 1920 × 1080; §7.12.3):

versión	coche	puerta	umbral	parachoques	capó
v6 (árboles de 640 px)	19.33	33.77	31.87	20.21	14.16
v7 + dispersión del píxel, τ 1 (1280 px)	—	35.53	32.36	relMSE 0.199	14.46
v7, valores por defecto actuales	19.03	35.52	32.36	22.28	14.68
v8 (nivel de primer plano a 5120 px)	—	—	—	—	—

v8 se estaba convirtiendo para el reproductor cuando se escribió esto (CX ronda 4); no se ha medido. En el recorte del coche, el lóbulo nítido por píxel del camino compacto llevó el coche de 16.49 a 17.46 dB (§7.12.3).

Velocidad (§7.12.1, §7.12.2):

dispositivo	datos	dibujados	resultado
iPhone real, página de estrés	3DGS simple	200 k / ~1 M	58.8 / 26.3–27 fps
iPhone real, página de estrés	compacto, base real	200 k / 400 k / ~1 M	27 / 47 / 108 ms
iPhone real, modelo	compacto	—	~115 ms por millón + 4 ms: ~250 k a 30 fps, ~110 k a 60 fps
iPhone real, primera prueba del usuario	v7, vista por defecto	106 k	53 fps (τ 8.00)
iPhone real, primera prueba del usuario	v7, de cerca	160 k	32 fps, logo y rejilla blandos
iPhone emulado	v5 / v7 / v7 tras CU ronda 9	226 k / 216 k / 234 k	60 fps con τ 3 / 2.22 / 1.95
iPhone emulado, fotograma quieto	v7, de cerca	529 k	un fotograma con τ 1.23
Dawn, M5 Pro, pasada compacta	393 × 852 τ 3 / 1920 τ 1	—	4.0 / 17.2 ms

La segunda prueba del usuario encontró la vista de cerca todavía blanda. Lo blando se atribuyó a la nube de 1280 px, no al reproductor, y eso llevó al nivel de primer plano a 5120 px (§7.12.1, §6.9). Para el gorrión, el corte móvil (488 656 splats, 31.7 MB) iba a 60 fps sin ventana en modo móvil, animado (§5.12).

8.8 Memoria

magnitud	cifra	§
datos del coche, v6 tras el atlas, τ 3: Camera / de cerca / de lejos	39.0 / 43.8 / 33.0 MB	7.11
estado de render, mismas vistas	24.1 / 26.0 / 21.7 MB	7.11
página (heap de JS, three.js, cúpulas)	~39 → 25 → ~17 MB	7.11
datos del coche, v7, τ 3	31–36 MB	7.11
datos del coche, fotograma quieto de cerca (presupuesto ~100 MB)	95.6 MB	7.10, 7.11
por gaussiana, reiluminado anterior / camino compacto	308 B + 29 MiB / 42.4 B + 16 B por splat dibujado + ~3 MiB	7.4, 7.11
con 1 M splats, anterior / compacto	326 / 58.6 MiB	7.4

magnitud	cifra	§
escritura de un árbol, puerta de 4.08 M hojas (tras el arreglo)	10.5 GB de pico residente	6.9
oclusión compartida sobre 24 árboles	10.2 GB de pico residente	6.9

El objetivo de 50–60 MB se aplica solo a los datos del coche; el estado de render compra la calidad pedida y se cuenta aparte (§7.11). En movimiento, los datos del coche eran 49.1–59.0 MB en v6 antes del atlas y 31–44 MB en v6 después de él y en v7, dentro del objetivo (§7.11).

8.9 Costes

paso	cifra	§
coche a 1280 px, 8 grupos, transfer de cada uno	212–304 s, unos 34 min en total	3.11
paneles a 5120 px, transfer de cada uno (máquina compartida)	2–25 min (capó 2 264 198 en 120 s; Front_Bumper_001 en 1 487 s)	3.11
shadow catcher, 309 k gaussianas	79 s	3.9
campos de visibilidad por parte del gorrión (12 partes)	22 s	5.8
decimación del gorrión conservando el rig (5.89 M)	28 s	5.9
codebook de oclusión compartido sobre el nivel de primer plano	645 s	6.9
construcción del LOD original, train_7k	47 ms	6.1

8.10 Donde las fuentes difieren

- **Hojas a 1280 px.** Los ocho horneados escribieron 8 703 429 gaussianas (§3.11, «8.70 M»); los árboles tienen 8.69 M hojas (§6.10). La primera cifra es la del registro de la conversión, la segunda la de los árboles.
- **Front_Bumper_001.** El registro del transfer da 5 009 059 gaussianas, el árbol 5 008 986 hojas (§3.11); los totales usan la cifra del árbol.
- **El tamaño en disco del nivel de primer plano.** 87.4 GB para los 24 árboles; «unos 86 GB» era la estimación antes de escribir el último árbol (§6.9).
- **El total de oclusión del nivel de primer plano.** Un primer informe marcó un 132.4 % reabierto; era un desbordamiento de 32 bits del total de celdas cerradas, corregido en 4a8f8c2. Las cifras por árbol (0–23.6 %) y el codebook eran correctos (§6.7).
- **El fotograma quieto.** El registro del reproductor da 529 k dibujados con τ 1.23 y 95.6 MB de datos del coche; `pieces.md` da 531 k con τ 1.22 en 118 ms (§7.10). Son dos ejecuciones de la misma política.
- **Juntas de la puerta a 1280 px.** La §6.10 cita a research como «juntas de 0.90 a 1.02» entre cut_b y T71, mientras que la tabla por clases de la §4.10.4 da las juntas de cut_b en 1.04 (0.90 es T66 sin cull). Las dos cifras vienen de ejecuciones de research distintas; ambas dejan las juntas a 1280 px a un 2–4 % del path trace.
- **Recorte del coche en el móvil.** Los 17.46 dB del camino compacto (§7.12.3) y los 17.88 dB de T71 (§4.10.3) son medidas separadas sobre caminos y datos distintos, y aquí no se comparan.
- **Velocidad real frente a emulada.** Las cifras emuladas van más o menos al doble que las del iPhone real; los dibujados se juzgan en el dispositivo (§7).

9. Limitaciones y trabajo futuro

Las limitaciones de abajo se reúnen de las listas de las propias secciones (§2.16, §3.12, §4.11, §5.13, §6.11, §7.13) y se agrupan por dónde actúan. Cada una se enuncia con la cifra que la mide, cuando la hay.

9.1 Defectos que quedan en el Corvette

Esto es lo que el coche de 1280 px aún muestra frente al path trace, los defectos que quien mira el móvil ve primero.

defecto	medido	causa	remedio propuesto	§
faros oscuros	faros oscuros frente al path trace	el horneado del transfer no lleva la emisión de las lámparas	hornear la emisión de las lámparas y su luz sobre el coche	3.12, 4.11
tira cromada del parachoques delantero	un 38 % más clara que el path trace, con el pico más bajo	un tubo de unos pocos píxeles de ancho: cada gaussiana lee su propio espejo con una sola normal	— (abierto)	3.12, 4.11
reflejo del suelo cercano en la llanta	0.72 del path trace	— (no analizado más)	— (abierto)	4.11
reflejos abiertos de la puerta	1.13 del path trace; el 68 % del exceso donde el espejo da contra el coche	celdas de 16 × 16 de unos 11° dejan pasar cielo detrás de un retrovisor o un pilar	celdas de 32 × 32 (+96 B por gaussiana) lo reducirían más o menos a la mitad	4.10.4, 4.11
labio inferior del parachoques	irregular	la huella de una celda por gaussiana en un borde enrollado	— (abierto)	3.12
brillos del capó	a 2.0 px de su posición en el path trace	el giro por píxel extrapola el operador de forma del triángulo central	una curvatura ajustada sobre la huella con muestras de la malla (p_{90} 9.8° frente a 46°) lo reduciría más o menos a la mitad	4.10.6, 4.11
huecos de la rejilla en los cortes gruesos	2.24 de las hojas a 640 px, 1.04 a 1280 px	fusiones de tiras de menos de un píxel; el hueco es una propiedad de la proyección	hojas más finas (hecho a 1280 px); ninguna medida por nodo independiente de la vista lo ve	6.5, 6.11
molduras finas lejanas	trazos gruesos en todos los cortes	las cotas de error no ven este error	— (abierto)	6.11

9.2 La representación y el transfer

- **Una normal, una huella.** Una gaussiana tiene una normal y la huella de una celda; las tiras finas curvadas y los bordes enrollados son donde se nota (§9.1). La correspondencia de cobertura supone el solapamiento $2\pi\sigma^2$ de la retícula de celdas, que no tienen los triángulos menores que una celda ni las gaussianas sujetas por bordes (§3.12).
- **Vidrio.** Un vidrio fino tintado se convierte como sólido, y una nube conserva un único índice de refracción salvo que lleve capas (§3.12). Las gaussianas de lámina fina se suben a un suelo de opacidad de 0.1 por gaussiana, así que fusionar menos láminas y más grandes hace variar la transmisión de celda a celda; queda pendiente un suelo por píxel (§6.11).

- **Campo de rango uno.** El acoplamiento del campo reflejado al cielo pone en un reflejo el color de un suelo iluminado, no su forma, bajo un cielo iluminado desde un lado; con grado 3 resuelve un horizonte a unos veinte grados (correlación 0.56 con grado 3, 0.63 con grado 5, 0.85 con un mapa octaédrico de 32×32, en simulación) (§4.11).
- **Luces dentro de la escena.** Se extrae un sol por cúpula. Las lámparas dentro de una escena se sombrean con bits que dicen qué direcciones salen de la escena, no cuáles llegan a la lámpara (§4.11).
- **Tamaño en el dispositivo.** La memoria del dispositivo limita una pieza a unos 5 M hojas para el árbol de fusión; el tope de 4.5 M hizo que las piezas más grandes se convirtieran a una resolución de cámara más gruesa (Bottom a 3392 px, Part_3 y Front_Bumper_001 a 4480 px; §3.11, §6.9).
- **Instanciado.** Los point instancers anidados solo se expanden en el nivel más interno, y el movimiento creado como velocidades se lee en la muestra (§3.12).

9.3 Assets animados

- Cuatro influencias por gaussiana truncan las puntas de las alas del gorrión (el 27 % de los vértices del ala llevan más); la proporción de gaussianas truncadas no se ha medido (§5.3, §5.13).
- No hay skinning por cuaterniones duales para las nubes, ni retargeting, ni mezcla de clips; de varias mallas con skinning con distintas transformaciones de prim solo se conserva la de la primera (§5.13).
- La ruta trazada posa una vez por frame, así que no desenfoca una nube con skinning, y las instancias de LoD no llevan movimiento (§5.13).
- La visibilidad por parte deja fuera el autosombreado de la propia parte, supone partes rígidas que ocluyen de forma independiente, y no se ha horneado por encima de unos 3 M gaussianas por parte; los transfers zonales conservan solo la mitad directa y el autosombreado de la pose de bind (§4.11, §5.8, §5.13).
- El error del ajuste zonal en el gorrión, y una medida geométrica de la cobertura en una flexión cerrada, no se han medido (§5.13).

9.4 Niveles de detalle, streaming y el reproductor

- **Residencia.** Las páginas tienen unas 4× el corte porque los nodos necesarios y los descartados por la vista comparten cada página; el móvil separa el recorrido de la carga útil en lugar de encoger las páginas (§6.8, §6.11).
- **Ponderación de la oclusión.** El codebook compartido se reparte por entradas usadas, no por la frecuencia con que una vista descarta un nodo; ponderar por una órbita de vistas en el horneado debería llevar el cull por encima del 84 % del ideal (§6.11).
- **Opacidad de las fusiones en el móvil.** El reproductor dibuja la masa de cada fusión, mientras que el rasterizador de athenea conserva la cobertura por mínimos cuadrados; el ajuste de cobertura es la mejor opción salvo en la rejilla (§6.4).
- **El corte en el móvil.** El recorrido compacto no guarda el tamaño de LoD de una fusión, así que un corte combinado de tamaño y error no tiene con qué protegerse, y el corte por error solo no ganó al corte por tamaño en todas las vistas (§7.13).
- **Sin portar.** La regla de caras traseras de cáscaras cerradas y el corte por error viven en `athenea-pieces`; el `CutSelector` del motor principal no ha cambiado (§6.11).

9.5 El motor

- Los splats todavía no están dentro de los rayos secundarios del path tracer: con mallas, los splats se componen por profundidad sobre las superficies trazadas y proyectan sombras sobre ellas, pero un rayo de rebote no los ve (§2.16).
- El desenfoque de movimiento y de lente del rasterizador son gaussianas ajustadas por momentos, hasta un 38 % demasiado picudas para estelas largas (§2.16).
- Las gaussianas de unas centésimas de píxel desaparecen del raster; los niveles de detalle son el remedio (§2.16).
- La visibilidad de mallas por raster paga unos 8 ms de tiempo de host por cada 1800 draws; la compilación de materiales es síncrona; la TLAS se reconstruye cada frame; la distorsión de lente más allá de k_1, k_2 no se aplica (§2.16).

9.6 Huecos en la validación

- **v8 en el móvil.** Los árboles y la oclusión compartida del nivel de primer plano a 5120 px están escritos (§6.9), pero su conversión para el reproductor estaba en curso cuando se escribió esto. Su calidad frente al path trace, su descarga y su comportamiento en el iPhone real están sin medir (§7.13). En particular, lo blando de cerca que lo motivó (§7.12.1) solo se ha mostrado resuelto en una prueba del logo y la rejilla (§3.11), no en el coche entero en el dispositivo.
- **La base de 512 clusters** no se ha medido en el móvil real (§7.2.2).
- **Los tiempos en dispositivo real** existen solo para las rampas de estrés y las dos pruebas del usuario (§7.13); la velocidad del móvil emulado no es la del móvil.
- **Un único asset de producción.** Las cifras a nivel de coche son de un asset, el Corvette, con el gorrión y el peón como controles animado y rígido. La generalidad a otros assets no se ha medido.
- **Máquina compartida.** Los tiempos de horneado y de árboles se tomaron en una máquina compartida con otros trabajos de GPU y no son medidas de rendimiento (§3.11, §8.9).

9.7 Trabajo futuro

En el orden que sugieren las medidas:

1. Medir v8 frente al path trace y en el iPhone real, de cerca y de lejos, con la base de 512 clusters.
2. Hornear la emisión de las lámparas en el transfer, para que los faros y su luz sobre el coche coincidan con el path trace.
3. Ajustar la curvatura sobre la huella de cada gaussiana con muestras de la malla, para reducir a la mitad el desplazamiento de los brillos del capó y estabilizar el giro por píxel.
4. Pasar a celdas de direcciones abiertas de 32×32 donde un coat nítido refleja el propio coche (la clase abierta de la puerta).
5. Tratar los tubos de cromo finos y los labios enrollados (la tira y el labio inferior del parachoques), el reflejo del suelo cercano en la llanta y las molduras finas lejanas.
6. Ponderar el codebook de oclusión por una órbita de vistas, y portar el corte por error y la regla de cáscaras cerradas al `CutSelector` del motor principal.
7. Para los assets animados: más de cuatro influencias, oclusión entre partes en poses distintas de la de bind, y horneados de visibilidad por encima de 3 M gaussianas por parte.

10. Referencias

Las referencias se dan por autores, título y año tal como se citan en el texto. Cuando las fuentes en las que se basa este artículo nombran una obra sin sus datos bibliográficos completos, se enumera tal como se cita y se marca así.

10.1 Splatting de gaussianas y render basado en puntos

1. Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics (SIGGRAPH).
2. Kerbl, B., Meuleman, A., Kopanas, G., Wimmer, M., Lanvin, A., Drettakis, G. (2024). A Hierarchical 3D Gaussian Representation for Real-Time Rendering of Very Large Datasets. ACM Transactions on Graphics (SIGGRAPH).
3. Zwicker, M., Pfister, H., van Baar, J., Gross, M. (2001). EWA Volume Splatting / Surface Splatting. IEEE Visualization y SIGGRAPH; y (2002) EWA Splatting, IEEE Transactions on Visualization and Computer Graphics.
4. Yu, Z., Chen, A., Huang, B., Sattler, T., Geiger, A. (2024). Mip-Splatting: Alias-free 3D Gaussian Splatting. CVPR.
5. Huang, B., Yu, Z., Chen, A., Geiger, A., Gao, S. (2024). 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. SIGGRAPH.
6. Pfister, H., Zwicker, M., van Baar, J., Gross, M. (2000). Surfels: Surface Elements as Rendering Primitives. SIGGRAPH.
7. Moënne-Loccoz, N., et al. (2024). 3D Gaussian Ray Tracing: Fast Tracing of Particle Scenes (3DGRT). ACM Transactions on Graphics (SIGGRAPH Asia).
8. Ren, K., et al. (2024). Octree-GS: Towards Consistent Real-time Rendering with LOD-Structured 3D Gaussians.
9. Xiong, et al. (2026). NanoGS: fusión por pares de vecinos más cercanos mutuos de gaussianas. Tal como se cita en la §6.2.
10. GaussianPOP: una puntuación de poda por gaussiana según el error que deja su eliminación. Tal como se cita en la §2.5.2.
11. Spark: un renderizador de splatting de gaussianas 3D para la web (World Labs). El reproductor de la §7 es un fork con backend WebGPU.

10.2 De mallas a gaussianas

12. Electronic Arts (SEED). mesh2splat: conversión de mallas en splats de gaussianas 3D por rasterización (código abierto, BSD-3). El origen de la conversión de la §3.
13. Mesh2GS. arXiv 2606.21898 (2026). El tamaño de celda derivado de una cámara de la §3.3.

10.3 Transporte de luz, transfer precalculado y compresión

14. Sloan, P.-P., Kautz, J., Snyder, J. (2002). Precomputed Radiance Transfer for Real-Time Rendering in Dynamic, Low-Frequency Lighting Environments. SIGGRAPH.
15. Sloan, P.-P., Hall, J., Hart, J., Snyder, J. (2003). Clustered Principal Components for Precomputed Radiance Transfer. SIGGRAPH.

16. Sloan, P.-P. (2008). Stupid Spherical Harmonics (SH) Tricks. Game Developers Conference. (Ajuste de armónicos zonales.)
17. Ramamoorthi, R., Hanrahan, P. (2001). An Efficient Representation for Irradiance Environment Maps. SIGGRAPH.
18. Křivánek, J., Colbert, M. (2008). Real-time Shading with Filtered Importance Sampling. Computer Graphics Forum (EGSR).
19. Lagarde, S., de Rousiers, C. (2014). Moving Frostbite to Physically Based Rendering. Notas de curso de SIGGRAPH.
20. Toksvig, M. (2005). Mipmapping Normal Maps. Journal of Graphics Tools.
21. Veach, E., Guibas, L. (1995). Optimally Combining Sampling Techniques for Monte Carlo Rendering. SIGGRAPH. (La heurística de potencia de la §2.11.)
22. Conty Estevez, A., Kulla, C. (2018). Importance Sampling of Many Lights with Adaptive Tree Splitting. HPG. (La BVH de luces de la §2.10.)
23. Lloyd, S. (1982). Least Squares Quantization in PCM. IEEE Transactions on Information Theory; Arthur, D., Vassilvitskii, S. (2007). k-means++: The Advantages of Careful Seeding. SODA. (Los ajustes k-means de las §7.2–7.3.)

10.4 Materiales y sombreado

24. Walter, B., Marschner, S., Li, H., Torrance, K. (2007). Microfacet Models for Refraction through Rough Surfaces. EGSR.
25. Heitz, E. (2018). Sampling the GGX Distribution of Visible Normals. Journal of Computer Graphics Techniques.
26. Oren, M., Nayar, S. (1994). Generalization of Lambert's Reflectance Model. SIGGRAPH; Portsmouth, J., Kutz, P., Hill, S. (2025). EON: A Practical Energy-Preserving Rough Diffuse BRDF.
27. Burley, B. (2012). Physically-Based Shading at Disney. Notas de curso de SIGGRAPH.
28. Estevez, A. C., Kulla, C. (2017). Production Friendly Microfacet Sheen BRDF. Sony Pictures Imageworks; Zeltner, T., Burley, B., Chiang, M. J.-Y. (2022). Practical Multiple-Scattering Sheen Using Linearly Transformed Cosines.
29. Lazarov, D. (2013). Getting More Physical in Call of Duty: Black Ops II. Notas de curso de SIGGRAPH. (El ajuste del albedo de entorno comparado en la §4.6.4.)
30. Andersson, Z., Kutz, P., et al. (2024). Especificación de OpenPBR Surface. Academy Software Foundation.
31. MaterialX: un estándar abierto para contenido de materiales y de look development (versión 1.39). Academy Software Foundation.

10.5 Geometría, estructuras de aceleración y niveles de detalle

32. Karras, T. (2012). Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees. High Performance Graphics.
33. Möller, T., Trumbore, B. (1997). Fast, Minimum Storage Ray–Triangle Intersection. Journal of Graphics Tools.
34. Runnalls, A. R. (2007). Kullback–Leibler Approach to Gaussian Mixture Reduction. IEEE Transactions on Aerospace and Electronic Systems.

35. Karis, B., Stubbe, R., Wihlidal, G. (2021). Nanite: A Deep Dive. SIGGRAPH Advances in Real-Time Rendering.

10.6 Color

36. Academy of Motion Picture Arts and Sciences (2024). Transformaciones de salida de ACES 2.0 (referencia en CTL).
37. Hellwig, L., Stolitzka, D., Fairchild, M. D. (2022). El modelo de apariencia de color conocido como Hellwig 2022, una revisión de CAM16, que usan las transformaciones de salida de ACES 2.0.
38. Wrensch, B. Un ajuste analítico de la transformación de vista AgX. Nota en línea, tal como se cita en la §2.12.
39. Krystek, M. (1985). An Algorithm to Calculate Correlated Colour Temperature. Color Research and Application. (El ajuste del cuerpo negro de la §2.10.)
40. OpenColorIO. Academy Software Foundation.

10.7 Animación y visibilidad

41. Lin, et al. MLP de visibilidad direccional por parte del cuerpo para avatares articulados. Tal como se cita en la §5.8.
42. DNF-Avatar: una rejilla escalar de oclusión ambiental por hueso. Tal como se cita en la §5.8.

10.8 Sistemas y estándares

43. Pixar Animation Studios. OpenUSD (Universal Scene Description) e Hydra, incluidos UsdSkel, UsdLux, UsdShade y el esquema `ParticleField`.
44. He, Y., Fatahalian, K., Foley, T. (2018). Slang: Language Mechanisms for Extensible Real-time Shading Systems. ACM Transactions on Graphics; slang-rhi, la interfaz de hardware de render que usa el motor.
45. Intel. Open Image Denoise (OIDN) 2.5.
46. W3C. WebGPU y el WebGPU Shading Language (WGSL).
47. Friedman, J., Jones, A. C. (2015). Fully Automatic ID Mattes with Support for Motion Blur and Transparency (Cryptomatte). SIGGRAPH Posters.
48. O'Neill, M. E. (2014). PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation; Halton, J. H. (1960). On the efficiency of certain quasi-random sequences of points.

Apéndice A. Repositorios, comandos y figuras

A.1 Repositorios

repositorio	dónde	qué contiene
athenea	github.com/jesusluque/athenea	el motor y renderizador de la §2: módulos, shaders, el delegado `hdAthenea`, la CLI `athenea`, los manuales `docs/operations.md` y `docs/development.md` con sus traducciones al español, y `docs/decisions.md`
athenea-pieces	github.com/jesusluque/athenea-pieces, rama `pieces` (privado)	un fork de athenea con el trabajo de pieces: árboles de fusión por pieza, el corte por error, las cáscaras cerradas, la oclusión compartida, los horneados de 1280 y 5120 px (§3.11, §6); `docs/pieces.md` y este artículo en `docs/paper/`
sparkwebGPU	`~/luc/sparkwebGPU`, rama `webgpu`	el fork WebGPU de Spark de la §7: el lector de `.athc` v3, el camino compacto, el streamer, las páginas del reproductor; su registro `CLAUDE.md` y sus notas `research/`

Los hashes de commit citados en las secciones 3–6 son commits de athenea-pieces; el 2e6e183 de la §2.15 es un commit de lucabRTrender, el predecesor de athenea antes de que se renombrara con una historia nueva. Los de la §7 son commits de sparkwebGPU salvo que el texto nombre athenea.

A.2 Comandos y opciones

El manual de cada subcomando y opción es `docs/operations.md` (en español: `docs/operations.es.md`), que da el valor por defecto, la unidad y la restricción de cada opción. Los comandos en los que se apoya este artículo:

comando	se usa para	sección del manual	§
`athenea mesh2splat STAGE (--cell-from-camera, --camera-pixels, --transfer, --hide, --max-cells, --specular-filter, --edge-reach, --shadow-catcher, --light-catcher, --validate, --skinned, --lod-levels)`	conversión y horneado	2.8, 3.1	3, 4, 5
`athenea decimate (--tree, --passes, --merge-glass, --object-groups, --tree-coverage, --tree-pages, --occlusion-mask, --occlusion-group, --cut-budget, --cut-cull-back)`	árboles de fusión, cortes, páginas, oclusión	2.4.1, 3.3	5.9, 6
`athenea visibility`	campos de visibilidad por parte	2.9	5.8
`athenea stage`, `athenea view`	tiempos (`--frames`) y el visor	2.5, 2.6	2
`athenea compare`	comparación de imágenes en la GPU	2.11	2.15
`athenea aofx`	los efectos AOFX (mesh2splat, Measure)	2.10, 7	2.14, 3.1
`athc-convert (--gzip --planes, --shadow-vq)`, `fit-compact-basis`, `floor_shadow --merge 2`, `usd-athc`	los formatos del reproductor (sparkwebGPU)	—	7

Las variables de entorno (`ATHENEA_GPU_BUDGET`, `ATHENEA_STAGES`) se enumeran en la §8.1 de `docs/operations.md`, los formatos (`.athc`, los esquemas sin código) en la §8.2 y la §4.3. Los trabajos de GPU se ejecutaron de uno en uno mediante `~/luc/athenea-gpu/shared.sh`, bajo `guard.sh` y `memguard.sh` (§5 de `docs/pieces.md`).

A.3 Figuras

Las figuras a las que se refieren las secciones 2–7 se guardan fuera del directorio del artículo:

figura	fichero	§
el gorrión, malla trazada y convertida	`athenea-pieces/docs/images/sparrow-mesh-cloud.png`, `sparrow-cloud.png`, `sparrow-film.png`	2, 5.10
bordes sujetos frente al control (coche de 640 px)	`athenea-renders/simplify3/edges/edges_cmp.png`, `day_new_Camera.png`, `day_ctl_Camera.png`	3.5
la prueba de horneado A / B / C a 1280 px	`athenea-renders/pieces/car1280/test/A.png`, `B.png`, `C.png`	3.11
rugosidad del coat guardada, path trace frente a hojas frente a corte	`athenea-renders/pieces/car/t63b/research_coat_roughness.png`, `t63b/mos.png`	4.10
cut_a / cut_b; la clase abierta de la puerta	`athenea-renders/pieces/car/t65/mos.png`, `t66/research_open_split.png`	4.10.4
el horneado de 1280 px frente al path trace; los trazos del pliegue del umbral	`athenea-renders/pieces/car/t71/mos.png`, `t71/research_sill.png`	4.10, 6.10
polish nítido condicionado; separación tubo/pliegue	`athenea-renders/pieces/car/t73/bump_mos.png`, `t74/sill_mos.png`	4.10.6
el gorrión en la web, path trace frente a raster; el ala corregida	`athenea-renders/sparrow-web/ref_pt_raster.png`, `sparrow/fixed_wing_athenea.png`	5.10
el peón blanco y su anillo	`athenea-renders/pawn-hq/pawn_gt_gs.png`, `ring_gt_gs.png`	5.11
la red de grietas y el cull de caras traseras (T50)	`athenea-renders/pieces/car/t50/cut_Camera.png`, `cull_Camera.png`	6.6, 6.10
logo y rejilla a 1280 frente a 5120 px	`athenea-renders/pieces/close/r_1280.png`, `r_5120.png`, `close_mos.png`	3.11, 6.9, 7.12

Todas las rutas de render están bajo `~/luc/athenea-renders/`; cada render del trabajo se guarda allí por tarea.